

Analyse et prévision des accidents corporels dans les Hauts-de-Seine

M

2026-07-06

Contents

1	Importation et préparation des données	3
1.1	Répertoire de travail	3
1.2	Importation des données	4
1.3	Contrôle initial	4
1.4	Conversion des dates	8
1.5	Recodage des variables cibles	8
1.6	Connexion à la base MySQL	9
1.7	Vérification de la connexion	9
2	Nettoyage et préparation des données	10
2.1	Définition des variables d'étude	10
2.2	Analyse de la qualité des données	10
2.3	Nettoyage de la variable Luminosité	13
2.4	Nettoyage de la variable Conditions atmosphérique	14
2.5	Nettoyage de la variable Lieu	16
2.6	Nettoyage de la variable Catégorie de route	18
2.7	Vérification des variables binaires	20
2.8	Contrôle des valeurs manquantes	22
2.9	Contrôle des modalités « Non renseigné »	23
2.10	Évolution annuelle de la qualité des données	23
2.11	Contrôle final des données	25
3	Préparation de la base SQL	26
3.1	Construction de la table finale	26
3.2	Contrôle final	28
3.3	Export vers MySQL	29
3.4	Vérification de l'import	30
3.5	Aperçu des données importées	31
4	Importation de la table mensuelle	31
4.1	Lecture de la table agrégée	32
4.2	Aperçu des noms des variables	32
4.3	Contrôle des données mensuelles	32
4.4	Validation des données	33
5	Préparation des données pour le modèle SARIMAX	36
5.1	Préparation des variables numériques	36
5.2	Contrôle des variables	36
5.3	Validation finale des données préparées	37
6	Analyse exploratoire de la série temporelle (EDA)	37

6.1	Construction de la série temporelle	37
6.2	Vérification de la série temporelle	37
6.3	Évolution de la série	38
6.4	Décomposition de la série	39
6.5	Distribution de la variable cible	40
6.6	Boxplot de la variable cible	42
6.7	Statistiques descriptives	43
6.8	Distribution mensuelle	44
6.9	Lag plots	45
6.10	Graphique de saisonnalité par année	46
6.11	Graphique mensuel	47
6.12	Analyse des fonctions d'autocorrélation	48
7	Analyse exploratoire des variables exogènes	50
7.1	Sélection des variables exogènes	50
7.2	Création des matrices d'apprentissage et de test	51
7.3	Contrôle de cohérence des données	51
7.4	Evolution temporelle des variables exogènes	52
7.5	Distribution des variables exogènes	55
7.6	Analyse des corrélations	57
8	Analyse de la stationnarité	60
8.1	Représentation graphique	60
8.2	Test de Dickey-Fuller augmenté (ADF)	61
8.3	Test KPSS	62
8.4	Détermination du nombre de différenciations	62
9	Construction et sélection des modèles SARIMA	62
9.1	Constitution des jeux d'apprentissage et de test	62
9.2	Identification automatique d'un modèle de référence	63
9.3	Estimation du modèle de référence	63
9.4	Diagnostic des résidus	63
9.5	Comparaison des modèles candidats	65
9.6	Validation sur l'échantillon test	66
10	Construction des modèles SARIMAX	67
10.1	Recherche des variables exogènes	67
10.2	Comparaison des structures SARIMAX	69
10.3	Comparaison des modèles selon l'AIC	71
10.4	Comparaison des modèles selon le RMSE	72
10.5	Modèles candidats retenus	73
11	Validation du modèle retenu	74
11.1	Validation croisée (Expanding Window)	74
11.2	Estimation du modèle final	76
11.3	Vérification des résidus	77
12	Validation externe sur les données BAAC 2022	79
12.1	Importation des fichiers BAAC 2022	79
12.2	Construction de la base de validation	80
12.3	Construction de la variable exogène mensuelle 2022	80
12.4	Standardisation de la variable	80
12.5	Vérification de la cohérence des données	81
12.6	Prévision des accidents en 2022	81
12.7	Comparaison entre les prévisions et les observations	82

12.8 Évaluation des performances prédictives	83
12.9 Visualisation des prévisions	83

```

required_packages <- c(
  "tidyverse",
  "forecast",
  "tseries",
  "ggplot2",
  "ggfortify",
  "DBI",
  "RMariaDB",
  "lubridate",
  "zoo",
  "readr",
  "reshape2",
  "GGally",
  "car",
  "dplyr",
  "patchwork",
  "gridExtra",
  "cowplot",
  "scales"
)

new_packages <- required_packages[
  !(required_packages %in% rownames(installed.packages()))
]

if(length(new_packages) > 0){
  install.packages(new_packages)
}

invisible(
  lapply(
    required_packages,
    library,
    character.only = TRUE
  )
)

```

1 Importation et préparation des données

Cette première étape consiste à mettre en place l'environnement de travail nécessaire à l'analyse. Les données sont importées, contrôlées puis préparées afin de garantir leur qualité avant les opérations de nettoyage, d'agrégation et de modélisation. Cette préparation constitue également une étape indispensable avant leur intégration dans la base de données MySQL, qui servira à centraliser les données et à faciliter les traitements relationnels réalisés dans les chapitres suivants.

1.1 Répertoire de travail

Cette étape permet de définir le répertoire dans lequel se trouvent les fichiers du projet. Le code fixe le dossier de travail afin que R puisse accéder automatiquement aux fichiers de données et produire les

différents résultats sans erreur de chemin d'accès.

```
setwd("/home/zosia/SINISTRES")
getwd()
#> [1] "/home/zosia/SINISTRES"
```

La sortie confirme que le répertoire de travail est correctement défini sur le dossier SINISTRES. Toutes les opérations d'importation et d'exportation réalisées dans la suite du projet utiliseront ce répertoire comme emplacement de référence.

##Création un dossier pour stocker les images

```
dir.create("graphics", showWarnings = FALSE)
```

1.2 Importation des données

Cette étape consiste à importer le fichier contenant les accidents corporels dans un objet R. Les paramètres utilisés permettent de lire correctement un fichier CSV séparé par des points-virgules tout en conservant les variables textuelles sous forme de caractères afin de faciliter leur nettoyage ultérieur.

```
accidents <- read.csv(
  "accidents-corporels-de-la-circulation-routiere.csv",
  sep = ";",
  stringsAsFactors = FALSE,
  encoding = "UTF-8"
)
```

1.3 Contrôle initial

Après l'importation, un contrôle préliminaire est effectué afin de vérifier que les données ont été correctement chargées. Le code affiche les dimensions de la base, le nombre de variables ainsi que les premières observations, ce qui permet de contrôler rapidement la structure générale du jeu de données.

```
cat("Dimensions :", dim(accidents), "\n")
#> Dimensions : 41266 139
cat("Nombre de variables :", ncol(accidents), "\n")
#> Nombre de variables : 139

head(accidents)
#>
#> 1  48.87802591666302, 2.1630524920390553      CRS 0198021 02036
#> 2  48.89007831550073, 2.2561923868882268 Sécurité publique 0092026 00817
#> 3  48.78741212485848, 2.2266826006293705 Sécurité publique 0092048 00579
#> 4  48.902630210540934, 2.2825802032500278 Sécurité publique 0092026 01672
#> 5  48.894264224562306, 2.191356397065447 Sécurité publique 0092050 00033
#> 6  48.81070194674014, 2.307955704735035 Sécurité publique 0092049 00046
#>
#>      date heure code_insee      commune      lieu
#> 1 2010-02-19 16:45      92063 Rueil-Malmaison Hors intersection
#> 2 2010-02-24 10:00      92026 Courbevoie Intersection en T
#> 3 2010-02-27 12:20      92048 Meudon Hors intersection
#> 4 2010-04-28 18:00      92026 Courbevoie Intersection en T
#> 5 2010-05-01 14:15      92050 Nanterre Hors intersection
#> 6 2010-05-03 23:45      92049 Montrouge Intersection en X
#>
#>      luminosite cond_atmos      TYPE_COLL
#> 1      Plein jour      Normales Collisions en chaîne
#> 2      Plein jour      Autre Collision par le côté
```

```

#> 3          Plein jour      Normales Collision par le côté
#> 4          Plein jour      Normales Collisions en chaîne
#> 5          Plein jour      Normales      Autre collision
#> 6 Nuit avec éclairage public allumé      Normales Collision par le côté
#>          adresse          cat_route1 nom_route1      rev_route1
#> 1      A86N, PK76000, S2          Autoroute      A86      Chaussée normale
#> 2      DOUMER / ABREUVOIR Route départementale      D6      Chaussée mouillée
#> 3 18, AV DU GAL DE GAULLE      Voie communale      Chaussée normale
#> 4      JOFFRE / SAENS Route départementale      D7      Chaussée normale
#> 5      AV HENRI MARTIN Route départementale      D986      Chaussée normale
#> 6      DORMOY / ARNOUX Route départementale      D62      Chaussée normale
#>          cat_route2 nom_route2      rev_route2
#> 1
#> 2 Route départementale      D7 Chaussée mouillée
#> 3
#> 4      Voie communale      Chaussée normale
#> 5
#> 6      Voie communale      Chaussée normale
#>          cat_ve1 route_ve1
#> 1          Véhicule Léger      Route 1
#> 2      Motocyclette (cylindrée comprise entre 50 et 125 cm3)      Route 2
#> 3          Véhicule Léger      Route 1
#> 4 Véhicule Utilitaire seul (PTAC compris entre 1,5 et 3,5t)      Route 1
#> 5          Véhicule Léger      Route 1
#> 6          Véhicule Léger      Route 2
#>          sens_ve1          man_ve1 nb_t_ve1 nb_bh_ve1
#> 1 P.K. ou P.R. décroissants      Même sens, même file      0      0
#> 2          Sans changement de direction      0      0
#> 3 P.K. ou P.R. décroissants      Tournant à gauche      0      0
#> 4      P.K. ou P.R. croissants      Même sens, même file      0      0
#> 5          En marche arrière      0      0
#> 6      P.K. ou P.R. croissants Sans changement de direction      0      0
#>          nb_bnh_ve1          cat_ve2 route_ve2
#> 1      0          Véhicule Léger      Route 1
#> 2      1 Motocyclette (cylindrée comprise entre 50 et 125 cm3)      Route 2
#> 3      0      Scooter (cylindrée comprise entre 50 et 125 cm3)      Route 1
#> 4      0          Véhicule Léger      Route 1
#> 5      0
#> 6      1          Véhicule Léger      Route 1
#>          sens_ve2          man_ve2 nb_t_ve2 nb_bh_ve2
#> 1 P.K. ou P.R. décroissants      Même sens, même file      0      0
#> 2          Sans changement de direction      0      0
#> 3 P.K. ou P.R. décroissants      Dépassant par la gauche      0      0
#> 4      P.K. ou P.R. croissants      Même sens, même file      0      0
#> 5          0      0
#> 6      P.K. ou P.R. croissants Sans changement de direction      0      0
#>          nb_bnh_ve2          cat_ve3 route_ve3          sens_ve3
#> 1      1 Véhicule Léger      Route 1 P.K. ou P.R. décroissants
#> 2      0
#> 3      1
#> 4      1 Véhicule Léger      Route 1      P.K. ou P.R. croissants
#> 5      0
#> 6      1

```

```

#>      man_ve3 nb_t_ve3 nb_bh_ve3 nb_bnh_ve3 cat_ve4 route_ve4 sens_ve4
#> 1 Même sens, même file      0      0      1
#> 2      0      0      0
#> 3      0      0      0
#> 4 Même sens, même file      0      0      0
#> 5      0      0      0
#> 6      0      0      0
#>      man_ve4 nb_t_ve4 nb_bh_ve4 nb_bnh_ve4 usager1 age_usa1 sexe_usa1
#> 1      0      0      0 Conducteur      47 Féminin
#> 2      0      0      0 Conducteur      31 Masculin
#> 3      0      0      0 Conducteur      23 Masculin
#> 4      0      0      0 Conducteur      28 Masculin
#> 5      0      0      0 Conducteur      42 Masculin
#> 6      0      0      0 Conducteur      36 Féminin
#>      grav_usa1 veh_usa1 etat_usa1 loc_usa1 attit_usa1
#> 1 Blessé léger (non hospitalisé) Véhicule 2
#> 2      Indemne Véhicule 2
#> 3 Blessé léger (non hospitalisé) Véhicule 2
#> 4      Indemne Véhicule 2
#> 5      Indemne Véhicule 1
#> 6 Blessé léger (non hospitalisé) Véhicule 1
#>      acc_usa1 usager2 age_usa2 sexe_usa2      grav_usa2
#> 1      Conducteur      36 Masculin      Indemne
#> 2      Conducteur      38 Masculin Blessé léger (non hospitalisé)
#> 3      Conducteur      35 Masculin      Indemne
#> 4      Passager      47 Masculin Blessé léger (non hospitalisé)
#> 5      Piéton      55 Féminin      Blessé grave (hospitalisé)
#> 6      Conducteur      62 Masculin Blessé léger (non hospitalisé)
#>      veh_usa2 etat_usa2      loc_usa2
#> 1 Véhicule 1
#> 2 Véhicule 1
#> 3 Véhicule 1
#> 4 Véhicule 2
#> 5      Sur la chaussée à plus de 50m du passage piéton
#> 6 Véhicule 2
#>      attit_usa2 acc_usa2 usager3 age_usa3 sexe_usa3
#> 1      Conducteur      25 Féminin
#> 2      NA
#> 3      NA
#> 4      Conducteur      50 Masculin
#> 5 Traversant      Seul      NA
#> 6      Passager      58 Féminin
#>      grav_usa3 veh_usa3 etat_usa3 loc_usa3 attit_usa3
#> 1 Blessé léger (non hospitalisé) Véhicule 3
#> 2
#> 3
#> 4      Indemne Véhicule 3
#> 5
#> 6      Indemne Véhicule 2
#>      acc_usa3 usager4 age_usa4 sexe_usa4 grav_usa4 veh_usa4 etat_usa4 loc_usa4
#> 1      NA
#> 2      NA
#> 3      NA

```

```

#> 4      Passenger      54 Masculin Indemne Véhicule 3
#> 5      NA
#> 6      NA
#>   attit_usa4 acc_usa4   usager5 age_usa5 sexe_usa5 grav_usa5   veh_usa5
#> 1      NA
#> 2      NA
#> 3      NA
#> 4      Conducteur      20 Masculin Indemne Véhicule 1
#> 5      NA
#> 6      NA
#>   etat_usa5 loc_usa5 attit_usa5 acc_usa5   usager6 age_usa6 sexe_usa6 grav_usa6
#> 1      NA
#> 2      NA
#> 3      NA
#> 4      Passenger      18 Masculin Indemne
#> 5      NA
#> 6      NA
#>   veh_usa6 etat_usa6 loc_usa6 attit_usa6 acc_usa6 type_acci
#> 1      NA Léger
#> 2      NA Léger
#> 3      NA Léger
#> 4 Véhicule 1 NA Léger
#> 5      NA Grave
#> 6      NA Léger
#>                                     victime nb_usager nb_veh nb_pie nb_pie_bnh
#> 1      Automobiliste légèrement blessé      3      3      0      0
#> 2 Cyclomotoriste ou motard légèrement blessé      2      2      0      0
#> 3 Cyclomotoriste ou motard légèrement blessé      2      2      0      0
#> 4      Automobiliste légèrement blessé      7      3      0      0
#> 5      Piéton gravement blessé      2      1      1      0
#> 6      Automobiliste légèrement blessé      3      2      0      0
#>   nb_pie_bh nb_pie_t nb_vel nb_vel_bnh nb_vel_bh nb_vel_t nb_cyc nb_cyc_bnh
#> 1      0      0      0      0      0      0      0      0
#> 2      0      0      0      0      0      0      0      0
#> 3      0      0      0      0      0      0      0      0
#> 4      0      0      0      0      0      0      0      0
#> 5      1      0      0      0      0      0      0      0
#> 6      0      0      0      0      0      0      0      0
#>   nb_cyc_bh nb_cyc_t nb_mot nb_mot_bnh nb_mot_bh nb_mot_t nb_ul nb_ul_bnh
#> 1      0      0      0      0      0      0      3      2
#> 2      0      0      2      1      0      0      0      0
#> 3      0      0      1      1      0      0      1      0
#> 4      0      0      0      0      0      0      3      1
#> 5      0      0      0      0      0      0      1      0
#> 6      0      0      0      0      0      0      2      2
#>   nb_ul_bh nb_ul_t nb_pl nb_pl_bnh nb_pl_bh nb_pl_t nb_tc nb_tc_bnh nb_tc_bh
#> 1      0      0      0      0      0      0      0      0      0
#> 2      0      0      0      0      0      0      0      0      0
#> 3      0      0      0      0      0      0      0      0      0
#> 4      0      0      0      0      0      0      0      0      0
#> 5      0      0      0      0      0      0      0      0      0
#> 6      0      0      0      0      0      0      0      0      0
#>   nb_tc_t cat_ve5 route_ve5 sens_ve5 man_ve5 nb_t_ve5 nb_bh_ve5 nb_bnh_ve5

```

```

#> 1      0      NA      NA      NA
#> 2      0      NA      NA      NA
#> 3      0      NA      NA      NA
#> 4      0      NA      NA      NA
#> 5      0      NA      NA      NA
#> 6      0      NA      NA      NA
#>                                     geo_shape
#> 1 {"coordinates": [2.1630524920390553, 48.87802591666302], "type": "Point"}
#> 2 {"coordinates": [2.2561923868882268, 48.89007831550073], "type": "Point"}
#> 3 {"coordinates": [2.2266826006293705, 48.78741212485848], "type": "Point"}
#> 4 {"coordinates": [2.2825802032500278, 48.902630210540934], "type": "Point"}
#> 5 {"coordinates": [2.191356397065447, 48.894264224562306], "type": "Point"}
#> 6 {"coordinates": [2.307955704735035, 48.81070194674014], "type": "Point"}

```

Les résultats montrent que la base contient 41 266 observations décrites par 139 variables, ce qui confirme que l'importation s'est déroulée correctement. L'affichage des premières lignes permet de vérifier la présence des principales variables relatives aux caractéristiques des accidents, des véhicules, des usagers et de leur localisation. Ce contrôle constitue une première validation avant les opérations de transformation et de nettoyage qui seront réalisées par la suite.

1.4 Conversion des dates

Cette étape convertit la variable date au format Date de R afin de permettre les traitements chronologiques. Le code vérifie également la période couverte par les données, ce qui est indispensable pour les futures analyses temporelles et la construction des séries chronologiques.

```

accidents$date <- as.Date(accidents$date)

cat("Période couverte :\n")
#> Période couverte :
print(range(accidents$date, na.rm = TRUE))
#> [1] "2006-01-01" "2021-12-31"

```

La conversion de la variable date au format Date de R est réalisée avec succès. La période d'observation s'étend du 1er janvier 2006 au 31 décembre 2021, soit seize années complètes. Cette couverture temporelle est suffisamment longue pour construire les séries chronologiques nécessaires à la modélisation SARIMA et SARIMAX.

1.5 Recodage des variables cibles

Cette étape transforme les variables nombre de piétons impliqués et nombre de deux-roues motorisés impliqués en variables binaires indiquant simplement leur présence ou leur absence dans un accident. Ce recodage simplifie leur utilisation comme variables explicatives lors des analyses statistiques et de la modélisation.

```

# Accident impliquant au moins un piéton
accidents$nb_pie <- ifelse(
  accidents$nb_pie > 0,
  1L,
  0L
)

# Accident impliquant au moins un deux-roues motorisé
accidents$nb_mot <- ifelse(
  accidents$nb_mot > 0,

```

```

1L,
0L
)

table(accidents$nb_pie)
#>
#>      0      1
#> 31903  9363
table(accidents$nb_mot)
#>
#>      0      1
#> 26281 14985

```

Les distributions obtenues confirment que les variables `nb_pie` et `nb_mot` ont été correctement transformées en variables binaires indiquant respectivement la présence (1) ou l'absence (0) d'un piéton ou d'un deux-roues motorisé dans l'accident. Ces variables pourront être utilisées comme variables explicatives lors des analyses statistiques et de la modélisation.

1.6 Connexion à la base MySQL

Cette étape établit la connexion entre R et la base de données MySQL contenant les données du projet. Les paramètres de connexion sont récupérés depuis un fichier d'environnement afin de sécuriser les identifiants d'accès. Cette connexion permettra d'exporter les données préparées, d'exécuter des requêtes SQL et de construire les tables utilisées dans la modélisation.

```

readRenviron("~/Renviron")

con <- dbConnect(
  RMariaDB::MariaDB(),
  host = Sys.getenv("MYSQL_HOST"),
  user = Sys.getenv("MYSQL_USER"),
  password = Sys.getenv("MYSQL_PASSWORD"),
  dbname = "projet_actuariat"
)

```

1.7 Vérification de la connexion

Après l'établissement de la connexion, une requête SQL simple est exécutée afin de vérifier que la communication entre R et MySQL fonctionne correctement et que la base de données attendue est bien sélectionnée.

```

DBI::dbGetQuery(
  con,
  "SELECT DATABASE();"
)
#>      DATABASE()
#> 1 projet_actuariat

```

Le résultat de la requête confirme que la connexion entre R et MySQL est correctement établie et que la base de données `projet_actuariat` est bien sélectionnée. Cette vérification garantit que les opérations ultérieures d'exportation, de création de tables et d'exécution de requêtes SQL pourront être réalisées sans erreur de connexion.

2 Nettoyage et préparation des données

Cette étape a pour objectif d'améliorer la qualité des données avant leur agrégation temporelle et leur utilisation dans les modèles statistiques. Les variables présentant des modalités incohérentes, des valeurs manquantes ou des différences de codification entre les années sont identifiées puis harmonisées afin de constituer une base de données fiable. Cette phase garantit également la cohérence des données qui seront ensuite stockées et exploitées dans la base MySQL pour la construction des tables d'analyse et des séries temporelles.

2.1 Définition des variables d'étude

Cette étape consiste à définir les variables qui seront retenues pour les analyses descriptives, les traitements statistiques et la modélisation. Ces variables sont sélectionnées en raison de leur disponibilité sur l'ensemble de la période d'observation et de leur pertinence pour expliquer les accidents corporels. Cette sélection permet également de concentrer les opérations de nettoyage sur les variables réellement utiles et prépare les données qui seront ensuite exportées vers MySQL pour les traitements relationnels et les agrégations temporelles.

```
vars_etude <- c(  
  "luminosite",  
  "cond_atmos",  
  "lieu",  
  "cat_route1",  
  "nb_pie",  
  "nb_mot"  
)
```

2.2 Analyse de la qualité des données

Cette étape consiste à examiner les modalités présentes dans chacune des variables d'étude afin de détecter les éventuelles incohérences, les anciennes nomenclatures, les valeurs manquantes ainsi que les modalités non renseignées. Ce diagnostic constitue une étape essentielle avant toute opération de nettoyage et garantit que les données stockées ultérieurement dans MySQL seront homogènes et adaptées aux analyses statistiques.

2.2.1 Exploration des modalités

Le code parcourt automatiquement l'ensemble des variables retenues pour afficher la distribution de leurs modalités. Cette exploration permet d'identifier rapidement les anomalies de codification, les doublons de libellés ainsi que les modalités rares ou incomplètes qui devront être harmonisées.

```
for(v in vars_etude){  
  
  cat("\n===== \n")  
  cat("Variable :", v, "\n")  
  cat("===== \n")  
  
  print(  
    accidents %>%  
      count(.data[[v]], sort = TRUE)  
  )  
  
}  
#>  
#> =====  
#> Variable : luminosite
```

```

#> =====
#>                                luminosite      n
#> 1                                Plein jour 27071
#> 2      Nuit avec éclairage public allumé 7938
#> 3                                Non renseignée 2504
#> 4                                Crépuscule ou aube 2380
#> 5                                Nuit sans éclairage public 967
#> 6      Nuit avec éclairage public non allumé 373
#> 7                                          33
#>
#> =====
#> Variable : cond_atmos
#> =====
#>                cond_atmos      n
#> 1                Normales 25623
#> 2                  1 5326
#> 3          Pluie légère 4153
#> 4      Non renseignées 2504
#> 5          Temps couvert 930
#> 6                  2 912
#> 7          Pluie forte 597
#> 8                  8 345
#> 9                  Autre 221
#> 10      Temps éblouissant 172
#> 11                  3 132
#> 12      Neige / grêle 125
#> 13                  7 54
#> 14      Brouillard / fumée 52
#> 15      Vent fort / tempête 47
#> 16                  33
#> 17                  4 18
#> 18                  9 11
#> 19                  6 6
#> 20                  5 5
#>
#> =====
#> Variable : lieu
#> =====
#>                                lieu      n
#> 1          Hors intersection 20003
#> 2          Intersection en X 8237
#> 3          Intersection en T 6805
#> 4              Non renseigné 2504
#> 5              Giratoire 1321
#> 6          Intersection en Y 891
#> 7      Intersection à plus de 4 branches 651
#> 8              Place 553
#> 9          Autre intersection 251
#> 10              Hors-Int 17
#> 11          Passage à niveau 12
#> 12              Autre 10
#> 13              En X 6
#> 14              En T 5

```

```

#>
#> =====
#> Variable : cat_route1
#> =====
#>
#> cat_route1      n
#> 1      Route départementale 19762
#> 2      Voie communale 8108
#> 3      Autoroute 3778
#> 4      3 3750
#> 5      4 1824
#> 6      Route nationale 1381
#> 7      1373
#> 8      1 982
#> 9      2 215
#> 10     9 16
#> 11     Autre 12
#> 12 Parc de stationnement ouvert à la circulation publique 12
#> 13     5 8
#> 14     6 8
#> 15     Hors réseau public 5
#> 16     D19 2
#> 17     D906 2
#> 18     A14 1
#> 19     A86 1
#> 20     D106 1
#> 21     D17 1
#> 22     D182 1
#> 23     D20 1
#> 24     D60 1
#> 25     D63 1
#> 26     D83 1
#> 27     D987 1
#> 28     D993 1
#> 29     DBROSSOLETTE AVENUE PIERRE 1
#> 30     DCHARLES PASQUA (QUAI) 32 A 58 1
#> 31     DEUROPE (AVENUE DE L') 1
#> 32     DGRANDE RUE 1
#> 33     DMARCEL DASSAULT 1
#> 34     DVERDUN (AVENUE DE) DU 115 AU 149 1
#> 35     DVERDUN (AVENUE DE) DU 152 AU 226 1
#> 36     N118 1
#> 37     NBRIAND AVENUE ARISTIDE N°2 A 114 1
#> 38     NDE LATTRE DE TASSIGNY (BOULEVARD DU MARE 1
#> 39     VC 1
#> 40     VC19 1
#> 41     VCGABRIEL PERI (RUE) 1
#> 42     VCGUY MOQUET (RUE) 1
#> 43     VCPAUL BERT (RUE) 1
#> 44     VCSALVADOR ALLENDE (RUE) 1
#> 45     VCSCHNEIDER (AVENUE) 1
#>
#> =====
#> Variable : nb_pie

```

```
#> =====
#>   nb_pie      n
#> 1      0 31903
#> 2      1  9363
#>
#> =====
#> Variable : nb_mot
#> =====
#>   nb_mot      n
#> 1      0 26281
#> 2      1 14985
```

L'exploration des variables met en évidence plusieurs types d'incohérences susceptibles d'affecter les analyses.

La variable Luminosité est principalement composée des modalités Plein jour (27 071 observations) et Nuit avec éclairage public allumé (7 938 observations). Elle contient également 2 504 modalités « Non renseignée » ainsi que 33 chaînes vides qui devront être harmonisées.

La variable Conditions atmosphériques présente une coexistence entre des libellés textuels (Normales, Pluie légère, Temps couvert, etc.) et d'anciennes codifications numériques (1 à 9), révélant une évolution de la nomenclature au cours de la période étudiée.

Pour la variable Lieu, plusieurs modalités décrivent une même situation mais avec des libellés différents, notamment Hors intersection et Hors-Int, ou encore Intersection en X et En X.

La variable Catégorie de route comporte également des anciennes codifications numériques ainsi que quelques identifiants particuliers de routes (A14, A86, D19, N118, VC19, etc.) qui devront être regroupés dans des catégories homogènes.

Enfin, les variables binaires nb_pie et nb_mot ne comportent que les modalités 0 et 1, ce qui confirme que leur codage est déjà cohérent.

Cette exploration permet ainsi d'identifier précisément les opérations de nettoyage nécessaires avant la constitution de la base finale.

2.3 Nettoyage de la variable Luminosité

Cette étape vise à vérifier la qualité de la variable Luminosité en recherchant les valeurs manquantes et les modalités incomplètes. Les éventuelles chaînes vides sont ensuite remplacées par une modalité explicite afin d'obtenir une codification homogène sur l'ensemble de la période d'étude.

```
accidents %>%
  count(luminosite, sort = TRUE)
#>
#>           luminosite      n
#> 1              Plein jour 27071
#> 2  Nuit avec éclairage public allumé  7938
#> 3              Non renseignée  2504
#> 4      Crépuscule ou aube  2380
#> 5          Nuit sans éclairage public   967
#> 6  Nuit avec éclairage public non allumé   373
#> 7
#> 33
```

```
sum(accidents$luminosite == "", na.rm = TRUE)
#> [1] 33
```

```

sum(is.na(accidents$luminosite))
#> [1] 0

accidents <- accidents %>%
  mutate(
    luminosite_clean = case_when(
      luminosite == "" ~ "Non renseignée",
      TRUE ~ luminosite
    )
  )

# Contrôle final
table(accidents$luminosite_clean)
#>
#>                Crépuscule ou aube                Non renseignée
#>                        2380                        2537
#>  Nuit avec éclairage public allumé Nuit avec éclairage public non allumé
#>                        7938                        373
#>      Nuit sans éclairage public                Plein jour
#>                        967                27071

# Vérification de cohérence
stopifnot(
  sum(table(accidents$luminosite_clean)) == nrow(accidents)
)

vars_etude[vars_etude == "luminosite"] <- "luminosite_clean"

```

L'exploration montre que la variable ne contient aucune valeur manquante (NA) mais présente 33 chaînes vides. Ces observations sont recodées sous la modalité « Non renseignée », ce qui permet d'uniformiser les informations manquantes.

Après harmonisation, la variable comporte désormais 2 537 observations classées dans la catégorie « Non renseignée », tandis que les autres modalités conservent leurs effectifs initiaux. La vérification de cohérence confirme que le nombre total d'observations reste identique à celui de la base d'origine, garantissant qu'aucune donnée n'a été perdue lors du nettoyage.

La nouvelle variable `luminosite_clean` constitue désormais la version de référence qui sera utilisée dans les analyses statistiques et qui sera utilisée lors de la création de la table `accidents_propre` dans MySQL.

2.4 Nettoyage de la variable Conditions atmosphérique

Cette étape consiste à harmoniser la variable Conditions atmosphériques afin de supprimer les différences de codification observées entre les différentes années. Les anciennes nomenclatures numériques sont regroupées avec leurs libellés actuels, tandis que les modalités manquantes sont uniformisées pour produire une variable cohérente.

```

# Exploration des modalités
accidents %>%
  count(cond_atmos, sort = TRUE)
#>      cond_atmos      n
#> 1      Normales 25623
#> 2              1   5326

```

```

#> 3      Pluie légère 4153
#> 4      Non renseignées 2504
#> 5      Temps couvert 930
#> 6              2 912
#> 7      Pluie forte 597
#> 8              8 345
#> 9              Autre 221
#> 10     Temps éblouissant 172
#> 11              3 132
#> 12     Neige / grêle 125
#> 13              7 54
#> 14     Brouillard / fumée 52
#> 15     Vent fort / tempête 47
#> 16              33
#> 17              4 18
#> 18              9 11
#> 19              6 6
#> 20              5 5

# Vérification des valeurs manquantes
sum(accidents$cond_atmos == "", na.rm = TRUE)
#> [1] 33
sum(is.na(accidents$cond_atmos))
#> [1] 0

# Harmonisation des modalités
accidents <- accidents %>%
  mutate(
    cond_atmos_clean = case_when(
      cond_atmos %in% c("Normales", "1") ~ "Normales",
      cond_atmos %in% c("Pluie légère", "2") ~ "Pluie légère",
      cond_atmos %in% c("Pluie forte", "3") ~ "Pluie forte",
      cond_atmos %in% c("Neige / grêle", "4") ~ "Neige / grêle",
      cond_atmos %in% c("Brouillard / fumée", "5") ~ "Brouillard / fumée",
      cond_atmos %in% c("Vent fort / tempête", "6") ~ "Vent fort / tempête",
      cond_atmos %in% c("Temps éblouissant", "7") ~ "Temps éblouissant",
      cond_atmos %in% c("Temps couvert", "8") ~ "Temps couvert",
      cond_atmos %in% c("Autre", "9") ~ "Autre",
      cond_atmos %in% c("Non renseignées", "") ~ "Non renseignées",
      TRUE ~ "Non renseignées"
    )
  )

```

```

# Contrôle final
table(accidents$cond_atmos_clean)
#>
#>          Autre  Brouillard / fumée      Neige / grêle  Non renseignées
#>          232          57          143          2537
#>      Normales      Pluie forte      Pluie légère      Temps couvert
#>      30949          729          5065          1275
#>      Temps éblouissant Vent fort / tempête
#>          226          53

# Vérification de cohérence
stopifnot(
  sum(table(accidents$cond_atmos_clean)) == nrow(accidents)
)

vars_etude[vars_etude == "cond_atmos"] <- "cond_atmos_clean"

```

L'analyse confirme l'absence de valeurs NA, mais met en évidence 33 chaînes vides ainsi que plusieurs anciennes codifications numériques coexistantes avec les libellés actuels.

Le recodage permet d'associer chaque ancienne valeur numérique à son équivalent textuel (Normales, Pluie légère, Temps couvert, etc.) tout en regroupant les modalités vides sous « Non renseignées ».

Après harmonisation, la variable présente une nomenclature unique composée de dix catégories cohérentes. Les conditions Normales représentent la modalité la plus fréquente avec 30 949 observations, tandis que les modalités Non renseignées totalisent 2 537 observations.

La vérification finale confirme que toutes les observations ont été conservées. La variable `cond_atmos_clean` remplace désormais la variable initiale dans la liste des variables d'étude et sera utilisée pour alimenter la table `accidents_propre`, à partir de laquelle seront construites les vues SQL du projet.

2.5 Nettoyage de la variable Lieu

Cette étape consiste à harmoniser la variable `Lieu` afin d'éliminer les différences de codification observées entre plusieurs années de collecte. Certaines modalités décrivent une même configuration d'accident mais utilisent des libellés différents. Le nettoyage permet de regrouper ces modalités sous une nomenclature unique, garantissant ainsi une meilleure cohérence des analyses statistiques et des traitements réalisés ultérieurement dans MySQL.

```

# Exploration de la variable lieu
accidents %>%
  count(lieu, sort = TRUE)
#>
#>          lieu      n
#> 1 Hors intersection 20003
#> 2 Intersection en X  8237
#> 3 Intersection en T  6805
#> 4 Non renseigné    2504
#> 5 Giratoire        1321
#> 6 Intersection en Y   891
#> 7 Intersection à plus de 4 branches  651
#> 8 Place            553
#> 9 Autre intersection  251
#> 10 Hors-Int         17
#> 11 Passage à niveau  12
#> 12 Autre            10

```

```

#> 13                               En X      6
#> 14                               En T      5

# Vérification des valeurs manquantes
sum(accidents$lieu == "", na.rm = TRUE)
#> [1] 0

sum(is.na(accidents$lieu))
#> [1] 0

# Harmonisation des modalités
accidents <- accidents %>%
  mutate(
    lieu_clean = case_when(

      lieu %in% c(
        "Hors intersection",
        "Hors-Int"
      ) ~ "Hors intersection",

      lieu %in% c(
        "Intersection en X",
        "En X"
      ) ~ "Intersection en X",

      lieu %in% c(
        "Intersection en T",
        "En T"
      ) ~ "Intersection en T",

      TRUE ~ lieu
    )
  )

# Contrôle final
table(accidents$lieu_clean)
#>
#>                Autre                Autre intersection
#>                10                251
#>                Giratoire            Hors intersection
#>                1321            20020
#> Intersection à plus de 4 branches    Intersection en T
#>                651                6810
#>                Intersection en X      Intersection en Y
#>                8243                891
#>                Non renseigné        Passage à niveau
#>                2504                12
#>                Place
#>                553

# Vérification de cohérence
stopifnot(

```

```
sum(table(accidents$lieu_clean)) == nrow(accidents))
```

L'exploration de la variable montre que les observations sont principalement classées dans la catégorie "Hors intersection", suivie des intersections en X puis des intersections en T. Elle met également en évidence quelques différences d'écriture telles que Hors-Int au lieu de Hors intersection, ou encore En X et En T à la place de Intersection en X et Intersection en T.

Les contrôles montrent qu'aucune valeur manquante ni chaîne vide n'est présente dans cette variable. Le nettoyage consiste donc uniquement à fusionner les modalités équivalentes afin d'obtenir une nomenclature homogène.

Après harmonisation, les effectifs des principales catégories sont correctement regroupés : les modalités Hors intersection, Intersection en X et Intersection en T intègrent désormais les anciennes dénominations utilisées dans certaines années. La vérification finale confirme que le nombre total d'observations reste inchangé, ce qui garantit qu'aucune donnée n'a été perdue au cours du recodage.

La nouvelle variable lieu_clean constitue désormais la variable de référence qui sera utilisée dans les analyses statistiques ainsi que dans les traitements réalisés au sein de la base MySQL.

2.6 Nettoyage de la variable Catégorie de route

Cette étape vise à harmoniser la variable Catégorie de route, qui présente plusieurs systèmes de codification selon les années d'observation. Le code identifie les anciennes nomenclatures, les identifiants de routes et les modalités atypiques afin de les regrouper dans des catégories homogènes. Cette standardisation est indispensable pour garantir la cohérence des analyses et faciliter les agrégations effectuées dans MySQL.

```
# Vérification des modalités atypiques
accidents %>%
  filter(
    !cat_route1 %in% c(
      "Autoroute",
      "Route nationale",
      "Route départementale",
      "Voie communale",
      "Hors réseau public",
      "Parc de stationnement ouvert à la circulation publique",
      "Autre",
      "1", "2", "3", "4", "5", "6", "9",
      ""
    )
  ) %>%
  count(cat_route1, sort = TRUE)
```

#>	cat_route1	n
#> 1	D19	2
#> 2	D906	2
#> 3	A14	1
#> 4	A86	1
#> 5	D106	1
#> 6	D17	1
#> 7	D182	1
#> 8	D20	1
#> 9	D60	1
#> 10	D63	1
#> 11	D83	1
#> 12	D987	1
#> 13	D993	1

```

#> 14          DBROSSOLETTE AVENUE PIERRE 1
#> 15          DCHARLES PASQUA (QUAI) 32 A 58 1
#> 16          DEUROPE (AVENUE DE L') 1
#> 17          DGRANDE RUE 1
#> 18          DMARCEL DASSAULT 1
#> 19          DVERDUN (AVENUE DE) DU 115 AU 149 1
#> 20          DVERDUN (AVENUE DE) DU 152 AU 226 1
#> 21          N118 1
#> 22          NBRIAND AVENUE ARISTIDE N°2 A 114 1
#> 23 NDE LATRE DE TASSIGNY (BOULEVARD DU MARE 1
#> 24          VC 1
#> 25          VC19 1
#> 26          VCGABRIEL PERI (RUE) 1
#> 27          VCGUY MOQUET (RUE) 1
#> 28          VCPAUL BERT (RUE) 1
#> 29          VCSALVADOR ALLENDE (RUE) 1
#> 30          VCSCHNEIDER (AVENUE) 1

# Recodage des anciennes nomenclatures et des valeurs aberrantes
accidents <- accidents %>%
  mutate(
    cat_route1_clean = case_when(

      # Libellés actuels et anciens codes
      cat_route1 %in% c("Autoroute", "1") ~ "Autoroute",
      cat_route1 %in% c("Route nationale", "2") ~ "Route nationale",
      cat_route1 %in% c("Route départementale", "3") ~ "Route départementale",
      cat_route1 %in% c("Voie communale", "4") ~ "Voie communale",
      cat_route1 %in% c("Hors réseau public", "5") ~ "Hors réseau public",
      cat_route1 %in% c(
        "Parc de stationnement ouvert à la circulation publique",
        "6"
      ) ~ "Parc stationnement",
      cat_route1 %in% c("Autre", "9") ~ "Autre",

      # Quelques observations contenant des identifiants de routes
      grepl("^A", cat_route1) ~ "Autoroute",
      grepl("^N", cat_route1) ~ "Route nationale",
      grepl("^D", cat_route1) ~ "Route départementale",
      grepl("^VC", cat_route1) ~ "Voie communale",

      # Chaînes vides
      TRUE ~ "Non renseigné"
    )
  )

# Regroupement des catégories très peu représentées
accidents <- accidents %>%
  mutate(
    cat_route_final = case_when(
      cat_route1_clean %in% c(
        "Autre",
        "Hors réseau public",

```

```

    "Parc stationnement"
  ) ~ "Autres",

  TRUE ~ cat_route1_clean
)
)

# Contrôle final
table(accidents$cat_route_final)
#>
#>      Autoroute      Autres      Non renseigné
#>      4762         61         1373
#> Route départementale  Route nationale  Voie communale
#>      23532        1599        9939

# Vérification de cohérence
stopifnot(
  sum(table(accidents$cat_route_final)) == nrow(accidents)
)

vars_etude[vars_etude == "cat_route1"] <- "cat_route_final"
vars_etude[vars_etude == "lieu"] <- "lieu_clean"

```

L'exploration met en évidence plusieurs modalités atypiques correspondant à des identifiants précis de routes, tels que A14, A86, N118, D19, VC19, ainsi que diverses adresses particulières. Ces modalités sont très peu représentées et ne correspondent pas à la nomenclature principale utilisée dans le reste de la base.

Le recodage permet d'abord d'associer les anciennes codifications numériques aux catégories actuelles (Autoroute, Route nationale, Route départementale, Voie communale, etc.). Les identifiants de routes sont ensuite automatiquement reclassés grâce à leurs préfixes (A, N, D ou VC), ce qui assure une harmonisation complète de la variable.

Enfin, les catégories très peu fréquentes telles que Autre, Hors réseau public et Parc stationnement sont regroupées dans une catégorie unique Autres, afin de limiter la dispersion des modalités et d'améliorer la robustesse des analyses statistiques.

Le contrôle final confirme que la variable harmonisée comporte principalement 23 532 routes départementales, 9 939 voies communales, 4 762 autoroutes et 1 599 routes nationales. Seules 1 373 observations restent classées comme Non renseigné, ce qui correspond aux informations absentes dans les données d'origine.

La variable `cat_route_final` remplace désormais la variable initiale dans la liste des variables d'étude. Cette harmonisation garantit une nomenclature unique de la catégorie de route et facilitera les agrégations réalisées dans la table `accidents_propre` ainsi que les vues SQL construites dans MySQL.

2.7 Vérification des variables binaires

Cette étape consiste à contrôler les variables binaires représentant la présence d'au moins un piéton et d'au moins un deux-roues motorisé dans un accident. Les vérifications portent sur la structure des variables, la présence éventuelle de valeurs manquantes ainsi que sur la cohérence de leur codage avant leur utilisation dans les analyses statistiques.

2.7.1 Vérification de la variable Nombre de piétons

Cette vérification permet de confirmer que la variable `nb_pie` possède une structure conforme à une variable binaire et qu'aucune anomalie susceptible d'affecter les analyses n'est présente.

```

# Exploration des modalités
accidents %>%
  count(nb_pie, sort = TRUE)
#>   nb_pie      n
#> 1      0 31903
#> 2      1  9363

# Structure
str(accidents$nb_pie)
#> int [1:41266] 0 0 0 0 1 0 1 0 0 0 ...

# Valeurs manquantes
sum(is.na(accidents$nb_pie))
#> [1] 0

# Résumé statistique
summary(accidents$nb_pie)
#>   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
#> 0.0000 0.0000  0.0000  0.2269 0.0000  1.0000

# Aucun nettoyage nécessaire
accidents$nb_pie_clean <- accidents$nb_pie

stopifnot(
  sum(!is.na(accidents$nb_pie_clean)) == nrow(accidents)
)

```

Les résultats montrent que la variable comporte uniquement les modalités 0 et 1, représentant respectivement les accidents n'impliquant aucun piéton et ceux impliquant au moins un piéton. La structure de la variable confirme qu'il s'agit bien d'une variable entière binaire.

Le contrôle des valeurs manquantes indique qu'aucune observation n'est absente. Les statistiques descriptives confirment que la moyenne de la variable est égale à 0,2269, ce qui signifie que 22,69 % des accidents impliquent au moins un piéton, tandis que la majorité des accidents n'en impliquent aucun.

Aucune opération de nettoyage supplémentaire n'étant nécessaire, la variable est simplement copiée dans `nb_pie_clean` afin de conserver une nomenclature homogène avec les autres variables nettoyées. Cette variable pourra être utilisée directement dans les analyses statistiques ainsi que dans les traitements réalisés sous MySQL.

2.7.2 Vérification de la variable Nombre de motos

Cette étape consiste à vérifier la qualité de la variable `nb_mot`, qui indique la présence ou l'absence d'au moins un deux-roues motorisé dans un accident. Les contrôles portent sur la structure de la variable, la présence de valeurs manquantes et la cohérence de son codage.

```

# Exploration des modalités
accidents %>%
  count(nb_mot, sort = TRUE)
#>   nb_mot      n
#> 1      0 26281
#> 2      1 14985

# Structure

```

```

str(accidents$nb_mot)
#>  int [1:41266] 0 1 1 0 0 0 0 0 1 1 ...

# Valeurs manquantes
sum(is.na(accidents$nb_mot))
#> [1] 0

# Résumé statistique
summary(accidents$nb_mot)
#>   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
#> 0.0000 0.0000 0.0000 0.3631 1.0000 1.0000

```

Les résultats montrent que la variable est correctement codée sous forme binaire avec uniquement les modalités 0 et 1. Aucune valeur manquante n'est observée, ce qui confirme l'intégrité de cette variable.

Les statistiques descriptives indiquent que La moyenne de 0,3631 indique que 36,31 % des accidents impliquent au moins un deux-roues motorisé, tandis que les autres accidents n'en concernent pas.

Ces contrôles confirment que la variable est directement exploitable dans les analyses descriptives, les modèles statistiques ainsi que dans les traitements SQL réalisés au sein de MySQL, sans nécessiter de nettoyage supplémentaire.

2.8 Contrôle des valeurs manquantes

Cette étape consiste à vérifier qu'aucune valeur manquante ne subsiste dans les variables retenues pour la modélisation après les différentes opérations de nettoyage et d'harmonisation. Ce contrôle permet de garantir la qualité de la base de données avant son utilisation dans les analyses statistiques et son intégration dans la base MySQL.

```

sapply(
  accidents[, c(
    "cat_route_final",
    "luminosite_clean",
    "cond_atmos_clean",
    "lieu_clean",
    "nb_pie",
    "nb_mot"
  )],
  function(x) sum(is.na(x))
)
#>  cat_route_final luminosite_clean cond_atmos_clean    lieu_clean
#>           0           0           0           0
#>      nb_pie      nb_mot
#>           0           0

```

Les résultats montrent qu'aucune des variables d'étude ne contient de valeur manquante. Les variables `cat_route_final`, `luminosite_clean`, `cond_atmos_clean`, `lieu_clean`, `nb_pie` et `nb_mot` présentent toutes un nombre de valeurs manquantes égal à zéro.

Ce contrôle confirme que les opérations de nettoyage ont permis d'obtenir un jeu de données complet. L'absence de valeurs manquantes évite les biais lors des analyses statistiques et garantit le bon fonctionnement des traitements réalisés dans MySQL.

2.9 Contrôle des modalités « Non renseigné »

Après la vérification des valeurs manquantes, cette étape consiste à quantifier les modalités explicitement codées comme « Non renseigné » ou « Non renseignée ». Contrairement aux valeurs manquantes, ces observations sont conservées afin de préserver l'information disponible tout en permettant d'évaluer la qualité globale de la base de données.

```
accidents %>%
  summarise(
    route_non_renseignee =
      sum(cat_route_final == "Non renseigné"),

    luminosite_non_renseignee =
      sum(luminosite_clean == "Non renseignée"),

    cond_atmos_non_renseignee =
      sum(cond_atmos_clean == "Non renseignées"),

    lieu_non_renseigne =
      sum(lieu_clean == "Non renseigné")
  )
#>   route_non_renseignee luminosite_non_renseignee cond_atmos_non_renseignee
#> 1                1373                    2537                    2537
#>   lieu_non_renseigne
#> 1                2504
```

Les résultats indiquent que 2 537 observations restent classées comme « Non renseignée » pour les variables Luminosité et Conditions atmosphériques, tandis que 2 504 observations sont codées « Non renseigné » pour la variable Lieu. La variable Catégorie de route comporte quant à elle 1 373 observations appartenant à cette modalité.

Ces modalités proviennent principalement des informations absentes dans les données d'origine et ne résultent pas des opérations de nettoyage. Leur conservation permet de maintenir l'intégralité des observations tout en identifiant explicitement les informations indisponibles.

Dans la base MySQL, ces modalités homogènes faciliteront les requêtes SQL permettant de mesurer la qualité des données, de calculer les proportions d'informations non renseignées ou encore d'exclure ces observations lors d'analyses spécifiques si nécessaire.

2.10 Évolution annuelle de la qualité des données

Cette étape analyse l'évolution de la qualité des données sur l'ensemble de la période d'étude. Le code calcule, pour chaque année, la proportion de modalités non renseignées dans les principales variables d'étude afin d'identifier les éventuelles différences de qualité entre les années de collecte.

```
qualite_annuelle <- accidents %>%
  mutate(annee = format(date, "%Y")) %>%
  group_by(annee) %>%
  summarise(
    n = n(),

    pct_non_rens_lumi =
      mean(luminosite_clean == "Non renseignée"),

    pct_non_rens_meteo =
      mean(cond_atmos_clean == "Non renseignées"),
```

```

pct_non_rens_lieu =
  mean(lieu_clean == "Non renseigné"),

pct_non_rens_route =
  mean(cat_route_final == "Non renseigné")
)

print(qualite_annuelle)
#> # A tibble: 16 x 6
#>   annee      n pct_non_rens_lumi pct_non_rens_meteo pct_non_rens_lieu
#>   <chr> <int>          <dbl>          <dbl>          <dbl>
#> 1 2006  2504            1            1            1
#> 2 2007  2762            0            0            0
#> 3 2008  2730            0            0            0
#> 4 2009  2690            0            0            0
#> 5 2010  2529            0            0            0
#> 6 2011  2431            0            0            0
#> 7 2012  2576            0            0            0
#> 8 2013  2635            0            0            0
#> 9 2014  2805            0            0            0
#> 10 2015  2724            0            0            0
#> 11 2016  2882            0            0            0
#> 12 2017  2644            0            0            0
#> 13 2018  2512            0            0            0
#> 14 2019  2448            0            0            0
#> 15 2020  1950          0.0108          0.0108          0
#> 16 2021  2444          0.00491          0.00491          0
#> # i 1 more variable: pct_non_rens_route <dbl>

accidents %>%
  filter(cat_route_final == "Non renseigné") %>%
  count(annee = format(date, "%Y"))
#>   annee      n
#> 1  2006  644
#> 2  2007  722
#> 3  2019    6
#> 4  2020    1

```

Les résultats mettent en évidence une amélioration de la qualité des données au cours de la période étudiée.

L'année 2006 présente une proportion très élevée de modalités « Non renseigné » pour les variables Luminosité, Conditions atmosphériques et Lieu. En effet, le tableau montre que 100 % des observations de 2006 sont classées comme « Non renseignée » ou « Non renseigné » pour ces trois variables (proportion égale à 1). Cette situation traduit une qualité de renseignement plus faible au début de la période d'observation.

À partir de 2007, ces proportions deviennent nulles pour les variables Luminosité, Conditions atmosphériques et Lieu, ce qui témoigne d'une amélioration importante de la qualité des informations collectées. Les seules exceptions concernent les années 2020 et 2021, où l'on observe de très faibles proportions de modalités « Non renseignée » pour les variables Luminosité et Conditions atmosphériques, respectivement de 1,08 % et 0,49 %.

Pour la variable Catégorie de route, les modalités « Non renseigné » sont principalement concentrées sur les premières années de la période. Les comptages montrent 644 observations en 2006 et 722 en 2007, auxquelles s'ajoutent seulement 6 observations en 2019 et 1 observation en 2020. Ces résultats indiquent que la quasi-totalité des informations manquantes concernant cette variable provient des premières années de la

base.

Cette analyse met ainsi en évidence une amélioration progressive de la qualité des données au fil du temps. Elle permet également de mieux interpréter les analyses temporelles qui seront réalisées à partir des données agrégées dans MySQL.

2.11 Contrôle final des données

Cette dernière étape valide l'ensemble des opérations de préparation réalisées au cours du chapitre. Plusieurs contrôles sont effectués afin de vérifier les dimensions de la base, la période couverte, l'absence de valeurs manquantes, la cohérence des variables d'étude ainsi que la distribution finale des variables binaires. Cette validation garantit que les données sont prêtes à être utilisées pour les traitements SQL, les analyses statistiques et la modélisation.

```
cat("===== CONTRÔLE FINAL =====\n\n")
#> ===== CONTRÔLE FINAL =====

# Dimensions
cat("Nombre d'observations :", nrow(accidents), "\n")
#> Nombre d'observations : 41266
cat("Nombre de variables   :", ncol(accidents), "\n\n")
#> Nombre de variables   : 145

# Période couverte
cat("Période couverte :\n")
#> Période couverte :
print(range(accidents$date, na.rm = TRUE))
#> [1] "2006-01-01" "2021-12-31"

cat("\n")

# Valeurs manquantes
cat("Valeurs manquantes par variable d'étude :\n")
#> Valeurs manquantes par variable d'étude :

sapply(
  accidents[vars_etude],
  function(x) sum(is.na(x))
)
#> luminosite_clean cond_atmos_clean      lieu_clean  cat_route_final
#>                0                0                0                0
#>          nb_pie          nb_mot
#>                0                0

cat("\n")

# Modalités restantes "Non renseigné"
cat("Modalités 'Non renseigné' restantes :\n")
#> Modalités 'Non renseigné' restantes :

sapply(
  accidents[vars_etude],
  function(x)
    sum(grepl("Non renseign", x, ignore.case = TRUE),
      na.rm = TRUE)
)
```

```

)
#> luminosite_clean cond_atmos_clean      lieu_clean  cat_route_final
#>           2537           2537           2504           1373
#>           nb_pie           nb_mot
#>           0           0

cat("\n")

# Vérification des variables binaires

table(accidents$nb_pie)
#>
#>      0      1
#> 31903  9363

table(accidents$nb_mot)
#>
#>      0      1
#> 26281 14985

```

Les contrôles finaux confirment que la base préparée contient 41 266 observations décrites par 145 variables, couvrant la période du 1er janvier 2006 au 31 décembre 2021. L'augmentation du nombre de variables par rapport à la base initiale (139 variables) résulte de la création des variables harmonisées utilisées pour les analyses.

Les vérifications montrent qu'aucune des variables d'étude (*luminosite_clean*, *cond_atmos_clean*, *lieu_clean*, *cat_route_final*, *nb_pie* et *nb_mot*) ne contient de valeur manquante (NA). Les seules informations incomplètes correspondent désormais aux modalités explicitement codées « Non renseigné » ou « Non renseignée », conservées afin de préserver l'ensemble des observations.

Les variables binaires présentent uniquement les modalités 0 et 1. La variable *nb_pie* recense 31 903 accidents sans piéton et 9 363 avec au moins un piéton, tandis que *nb_mot* distingue 26 281 accidents sans deux-roues motorisé et 14 985 avec au moins un deux-roues motorisé, confirmant la cohérence du recodage.

L'ensemble de ces contrôles valide la qualité de la base de données préparée. Les données sont désormais prêtes à être intégrées dans la table *accidents_propre* de la base *projet_actuariat*, qui servira ensuite à la création des vues SQL (*accidents_mensuels*, *accidents_annuels*, etc.) utilisées pour les analyses descriptives et la modélisation présentées dans les chapitres suivants.

3 Préparation de la base SQL

Cette étape consiste à préparer une version finale des données nettoyées avant leur intégration dans la base de données MySQL. Les variables retenues sont sélectionnées, harmonisées et converties dans un format compatible avec la base relationnelle. Cette préparation garantit la cohérence des données stockées dans MySQL et facilite les requêtes SQL nécessaires à la création des tables agrégées ainsi qu'à la construction des séries temporelles utilisées dans la modélisation.

3.1 Construction de la table finale

Cette étape consiste à construire la table finale qui sera importée dans la base MySQL. Les variables nettoyées sont sélectionnées, renommées afin de retrouver leurs noms d'origine, puis converties dans un format adapté aux traitements statistiques et aux opérations réalisées dans la base de données relationnelle.

```

accidents_propre <- accidents %>%
  select(

```

```

    date,
    luminosite_clean,
    cond_atmos_clean,
    lieu_clean,
    cat_route_final,
    nb_pie,
    nb_mot
  ) %>%
  rename(
    luminosite = luminosite_clean,
    cond_atmos = cond_atmos_clean,
    lieu = lieu_clean,
    cat_route1 = cat_route_final
  ) %>%
  mutate(
    date = as.Date(date),
    nb_pie = as.integer(nb_pie),
    nb_mot = as.integer(nb_mot)
  )

str(accidents_propre)
#> 'data.frame':   41266 obs. of  7 variables:
#> $ date      : Date, format: "2010-02-19" "2010-02-24" ...
#> $ luminosite: chr  "Plein jour" "Plein jour" "Plein jour" "Plein jour" ...
#> $ cond_atmos: chr  "Normales" "Autre" "Normales" "Normales" ...
#> $ lieu      : chr  "Hors intersection" "Intersection en T" "Hors intersection" "Intersection en T" ...
#> $ cat_route1: chr  "Autoroute" "Route départementale" "Voie communale" "Route départementale" ...
#> $ nb_pie    : int   0 0 0 0 1 0 1 0 0 0 ...
#> $ nb_mot    : int   0 1 1 0 0 0 0 0 1 1 ...

table(accidents_propre$nb_pie)
#>
#>      0      1
#> 31903  9363

table(accidents_propre$nb_mot)
#>
#>      0      1
#> 26281 14985

```

Les résultats montrent que la table accidents_propre contient 41 266 observations réparties sur 7 variables.

La structure de la table indique que la variable date est enregistrée au format Date, les variables luminosite, cond_atmos, lieu et cat_route1 sont de type caractère (character), tandis que nb_pie et nb_mot sont de type entier (integer).

Les distributions des variables binaires confirment que le recodage réalisé lors des étapes précédentes a été conservé. La variable nb_pie comporte 31 903 accidents sans piéton et 9 363 accidents impliquant au moins un piéton. De même, la variable nb_mot comporte 26 281 accidents sans deux-roues motorisé et 14 985 accidents impliquant au moins un deux-roues motorisé.

Cette table constitue la version finale des données qui sera exportée dans MySQL.

3.2 Contrôle final

Avant l'exportation vers MySQL, un contrôle final est réalisé afin de vérifier la structure de la table, les types des variables, l'absence de valeurs manquantes ainsi que la cohérence des différentes modalités. Cette étape garantit que les données sont conformes aux exigences des traitements réalisés dans la base de données relationnelle.

```
str(accidents_propre)
#> 'data.frame': 41266 obs. of 7 variables:
#> $ date : Date, format: "2010-02-19" "2010-02-24" ...
#> $ luminosite: chr "Plein jour" "Plein jour" "Plein jour" "Plein jour" ...
#> $ cond_atmos: chr "Normales" "Autre" "Normales" "Normales" ...
#> $ lieu : chr "Hors intersection" "Intersection en T" "Hors intersection" "Intersection en T"
#> $ cat_route1: chr "Autoroute" "Route départementale" "Voie communale" "Route départementale" ...
#> $ nb_pie : int 0 0 0 0 1 0 1 0 0 0 ...
#> $ nb_mot : int 0 1 1 0 0 0 0 0 1 1 ...

summary(accidents_propre)
#> date luminosite cond_atmos lieu
#> Min. :2006-01-01 Length:41266 Length:41266 Length:41266
#> 1st Qu.:2009-11-12 Class :character Class :character Class :character
#> Median :2013-12-02 Mode :character Mode :character Mode :character
#> Mean :2013-11-11
#> 3rd Qu.:2017-09-01
#> Max. :2021-12-31
#> cat_route1 nb_pie nb_mot
#> Length:41266 Min. :0.0000 Min. :0.0000
#> Class :character 1st Qu.:0.0000 1st Qu.:0.0000
#> Mode :character Median :0.0000 Median :0.0000
#> Mean :0.2269 Mean :0.3631
#> 3rd Qu.:0.0000 3rd Qu.:1.0000
#> Max. :1.0000 Max. :1.0000

colSums(is.na(accidents_propre))
#> date luminosite cond_atmos lieu cat_route1 nb_pie nb_mot
#> 0 0 0 0 0 0 0

nrow(accidents_propre)
#> [1] 41266

table(accidents_propre$luminosite)
#>
#> Crépuscule ou aube Non renseignée
#> 2380 2537
#> Nuit avec éclairage public allumé Nuit avec éclairage public non allumé
#> 7938 373
#> Nuit sans éclairage public Plein jour
#> 967 27071

table(accidents_propre$cond_atmos)
#>
#> Autre Brouillard / fumée Neige / grêle Non renseignées
#> 232 57 143 2537
#> Normales Pluie forte Pluie légère Temps couvert
#> 30949 729 5065 1275
```

```

#>    Temps éblouissant Vent fort / tempête
#>                226                53

table(accidents_propre$lieu)
#>
#>                Autre                Autre intersection
#>                10                251
#>                Giratoire            Hors intersection
#>                1321            20020
#> Intersection à plus de 4 branches    Intersection en T
#>                651                6810
#>                Intersection en X      Intersection en Y
#>                8243                891
#>                Non renseigné          Passage à niveau
#>                2504                12
#>                Place
#>                553

table(accidents_propre$cat_route1)
#>
#>                Autoroute                Autres                Non renseigné
#>                4762                61                1373
#> Route départementale    Route nationale    Voie communale
#>                23532                1599                9939

table(accidents_propre$nb_pie)
#>
#>    0    1
#> 31903 9363

table(accidents_propre$nb_mot)
#>
#>    0    1
#> 26281 14985

```

Les résultats confirment que la table finale contient 41 266 observations et 7 variables, couvrant la période comprise entre le 1er janvier 2006 et le 31 décembre 2021.

Les statistiques descriptives montrent que les variables binaires conservent les mêmes caractéristiques que lors des étapes précédentes. La moyenne de nb_pie est de 0,2269, ce qui correspond à la proportion d'accidents impliquant au moins un piéton, tandis que celle de nb_mot est de 0,3631, correspondant à la proportion d'accidents impliquant au moins un deux-roues motorisé.

Le contrôle des valeurs manquantes indique qu'aucune des sept variables ne contient de valeur NA. Les tableaux de fréquences montrent également que les modalités des variables luminosite, cond_atmos, lieu et cat_route1 correspondent aux résultats obtenus après les opérations de nettoyage et d'harmonisation.

L'ensemble de ces vérifications confirme la cohérence de la table accidents_propre avant son exportation vers MySQL.

3.3 Export vers MySQL

Cette étape permet d'exporter la table finale depuis R vers la base de données MySQL. La fonction utilisée crée ou remplace automatiquement la table accidents_propre, qui devient la table de référence pour l'ensemble des traitements relationnels réalisés dans le projet.

```
dbWriteTable(
  con,
  "accidents_propre",
  accidents_propre,
  overwrite = TRUE,
  row.names = FALSE
)
```

L'exécution de la fonction `dbWriteTable()` lance l'exportation de la table `accidents_propre` vers la base de données `projet_actuariat`. L'argument `overwrite = TRUE` indique que toute table existante portant le même nom est remplacée par la nouvelle version.

La réussite de cette opération est vérifiée dans l'étape suivante à l'aide de plusieurs requêtes SQL exécutées directement dans MySQL.

3.4 Vérification de l'import

Après l'exportation, plusieurs requêtes SQL sont exécutées afin de vérifier que la table a été correctement créée dans MySQL. Les contrôles portent sur le nombre d'observations, la structure de la table ainsi que la cohérence des principales variables binaires.

```
DBI::dbGetQuery(
  con,
  "SELECT COUNT(*) AS n FROM accidents_propre"
)
#>      n
#> 1 41266

dbListFields(
  con,
  "accidents_propre"
)
#> [1] "date"      "luminosite" "cond_atmos" "lieu"      "cat_route1"
#> [6] "nb_pie"     "nb_mot"

DBI::dbGetQuery(
  con,
  "SELECT nb_mot, COUNT(*) AS n
   FROM accidents_propre
   GROUP BY nb_mot"
)
#>  nb_mot      n
#> 1      0 26281
#> 2      1 14985

DBI::dbGetQuery(
  con,
  "SELECT nb_pie, COUNT(*) AS n
   FROM accidents_propre
   GROUP BY nb_pie"
)
#>  nb_pie      n
#> 1      0 31903
#> 2      1  9363
```

Les résultats montrent que la table `accidents_propre` contient 41 266 observations, soit le même nombre que la table préparée sous R. Cette correspondance confirme que l'importation a conservé l'ensemble des observations.

La liste des champs retournée par `dbListFields()` montre que la table comporte les sept variables attendues : `date`, `luminosite`, `cond_atmos`, `lieu`, `cat_routel`, `nb_pie` et `nb_mot`.

Les requêtes SQL exécutées sur les variables `nb_pie` et `nb_mot` retournent les mêmes effectifs que ceux obtenus sous R (31 903 et 9 363 pour `nb_pie`, 26 281 et 14 985 pour `nb_mot`), ce qui confirme la cohérence des données après leur importation dans MySQL.

Ces contrôles valident la structure et le contenu de la table créée dans la base de données.

3.5 Aperçu des données importées

Cette dernière étape consiste à afficher un échantillon des premières observations directement depuis la base MySQL. Cette vérification visuelle permet de contrôler la bonne restitution des données après leur importation et de confirmer que les variables ont été correctement enregistrées.

```
DBI::dbGetQuery(
  con,
  "
  SELECT *
  FROM accidents_propre
  LIMIT 5
  "
)
#>      date luminosite cond_atmos      lieu      cat_routel
#> 1 2010-02-19 Plein jour  Normales Hors intersection  Autoroute
#> 2 2010-02-24 Plein jour   Autre Intersection en T Route départementale
#> 3 2010-02-27 Plein jour  Normales Hors intersection  Voie communale
#> 4 2010-04-28 Plein jour  Normales Intersection en T Route départementale
#> 5 2010-05-01 Plein jour  Normales Hors intersection Route départementale
#>   nb_pie nb_mot
#> 1      0      0
#> 2      0      1
#> 3      0      1
#> 4      0      0
#> 5      1      0
```

L'affichage des cinq premières observations montre que les variables `date`, `luminosite`, `cond_atmos`, `lieu`, `cat_routel`, `nb_pie` et `nb_mot` sont correctement restituées dans la table `accidents_propre`.

Les valeurs affichées sont cohérentes avec celles obtenues lors de la préparation des données sous R, ce qui confirme que la structure de la table a été conservée après l'importation dans MySQL.

Cette vérification constitue un contrôle visuel de la table avant son utilisation pour les requêtes SQL et les traitements réalisés dans les chapitres suivants.

4 Importation de la table mensuelle

Cette étape consiste à importer la table agrégée mensuellement depuis la base de données MySQL vers l'environnement R afin de procéder à la modélisation.

Note méthodologique concernant la période d'étude (Exclusion de l'année 2006)

Bien que la base de données brute s'étende initialement du 1er janvier 2006 au 31 décembre 2021, les analyses préliminaires de la qualité des données (détaillées dans la section 2.10) ont révélé que l'année 2006 présente

un taux de non-réponse de 100 % sur plusieurs variables clés, telles que les conditions atmosphériques, la luminosité et le lieu.

L'intégration de cette période fausserait l'analyse des facteurs exogènes et dégraderait la qualité des prévisions. Par conséquent, afin de garantir la robustesse et la cohérence de la modélisation SARIMAX, la série temporelle mensuelle a été restreinte à la période allant de janvier 2007 à décembre 2021.

4.1 Lecture de la table agrégée

Cette étape permet d'importer depuis MySQL la table `accidents_mensuels`, créée lors des traitements SQL d'agrégation. Les observations sont triées par année puis par mois afin d'obtenir une série temporelle parfaitement ordonnée pour les analyses ultérieures.

```
sarimax_sql <- dbGetQuery(  
  con,  
  "  
  SELECT *  
  FROM accidents_mensuels  
  ORDER BY annee, mois  
  "  
)
```

Cette requête récupère directement la table mensuelle produite dans MySQL. Le tri chronologique réalisé grâce à la clause `ORDER BY` garantit que les observations sont importées dans l'ordre temporel, condition indispensable pour les analyses de séries temporelles. Cette étape illustre le rôle de MySQL comme système de stockage et de prétraitement des données agrégées, tandis que R est utilisé pour les analyses statistiques.

4.2 Aperçu des noms des variables

Cette étape consiste à identifier les variables disponibles dans la table importée afin de vérifier que toutes les informations nécessaires à la modélisation ont bien été récupérées depuis MySQL.

```
names(sarimax_sql)  
#> [1] "annee"           "mois"            "nb_accidents"    "pct_nuit"  
#> [5] "pct_meteo"       "pct_intersection" "pct_autoroute"   "pct_pieton"  
#> [9] "pct_moto"
```

Les variables affichées correspondent aux indicateurs mensuels calculés lors des traitements SQL. On retrouve notamment l'année, le mois, le nombre mensuel d'accidents ainsi que les différentes proportions utilisées comme variables explicatives. Ce contrôle confirme que la structure de la table MySQL est conforme aux besoins de la modélisation.

4.3 Contrôle des données mensuelles

Avant de construire les modèles statistiques, il est nécessaire de vérifier la structure de la table importée. Les types des variables, leurs dimensions ainsi que leur organisation sont contrôlés afin de garantir leur compatibilité avec les fonctions de modélisation disponibles sous R.

4.3.1 Conversion du nombre d'accidents en numérique

Cette étape consiste à convertir explicitement la variable représentant le nombre mensuel d'accidents en format numérique afin d'assurer sa compatibilité avec les méthodes statistiques utilisées par la suite.

```
sarimax_sql <- sarimax_sql %>%  
  mutate(  
    nb_accidents = as.numeric(nb_accidents)  
  )
```

La conversion garantit que la variable réponse est bien reconnue comme une variable quantitative. Même si MySQL conserve déjà cette information sous un format numérique, cette vérification sous R sécurise les traitements ultérieurs et évite les erreurs liées à une mauvaise interprétation du type de données.

4.3.2 Vérification de la structur

Cette étape permet d'examiner la structure complète de la table afin de vérifier le type de chaque variable importée depuis MySQL.

```
str(sarimax_sql)
#> 'data.frame': 180 obs. of 9 variables:
#> $ annee : int 2007 2007 2007 2007 2007 2007 2007 2007 2007 2007 ...
#> $ mois : int 1 2 3 4 5 6 7 8 9 10 ...
#> $ nb_accidents : num 222 189 254 251 225 238 206 165 214 280 ...
#> $ pct_nuit : num 0.2883 0.3492 0.2008 0.1076 0.0667 ...
#> $ pct_meteo : num 0.203 0.228 0.169 0.012 0.218 ...
#> $ pct_intersection: num 0.545 0.529 0.425 0.55 0.538 ...
#> $ pct_autoroute : num 0.0541 0.0635 0.0906 0.0598 0.0844 ...
#> $ pct_pieton : num 0.261 0.191 0.252 0.251 0.209 ...
#> $ pct_moto : num 0.392 0.318 0.37 0.39 0.351 ...
```

La structure confirme que la base contient 180 observations mensuelles décrites par 9 variables. Les années et les mois sont codés en entiers tandis que les indicateurs agrégés sont numériques. Cette organisation est parfaitement adaptée aux analyses temporelles et confirme que l'importation depuis MySQL s'est déroulée correctement.

4.3.3 Dimensions de la table

Cette étape vérifie le nombre de lignes et de colonnes de la table mensuelle afin de confirmer que l'ensemble des observations attendues a bien été importé.

```
dim(sarimax_sql)
#> [1] 180 9
```

Le résultat indique que la table comporte 180 observations et 9 variables. Les 180 observations correspondent aux mois de la période d'étude, ce qui confirme que la table mensuelle construite dans MySQL a été intégralement récupérée sous R.

4.3.4 Noms des variables

Cette vérification permet de contrôler une nouvelle fois les variables présentes dans la base afin de confirmer que toutes les variables explicatives nécessaires sont disponibles avant les analyses statistiques.

```
names(sarimax_sql)
#> [1] "annee" "mois" "nb_accidents" "pct_nuit"
#> [5] "pct_meteo" "pct_intersection" "pct_autoroute" "pct_pieton"
#> [9] "pct_moto"
```

Les noms affichés confirment la présence de l'ensemble des variables issues des agrégations SQL. Cette vérification permet également de s'assurer que les intitulés des variables sont cohérents et pourront être utilisés directement dans les modèles SARIMAX sans modification supplémentaire.

4.4 Validation des données

Après l'importation, plusieurs contrôles sont réalisés afin de vérifier que les données agrégées sont complètes, cohérentes et directement exploitables pour la modélisation des séries temporelles.

4.4.1 Vérification de la période couverte

Cette étape consiste à afficher les premières et les dernières observations de la table afin de vérifier que la période d'étude est complète et correctement ordonnée.

```
head(
  sarimax_sql %>%
    select(annee, mois, nb_accidents),
  12
)
#>      annee mois nb_accidents
#> 1   2007    1         222
#> 2   2007    2         189
#> 3   2007    3         254
#> 4   2007    4         251
#> 5   2007    5         225
#> 6   2007    6         238
#> 7   2007    7         206
#> 8   2007    8         165
#> 9   2007    9         214
#> 10  2007   10         280
#> 11  2007   11         259
#> 12  2007   12         259

tail(
  sarimax_sql %>%
    select(annee, mois, nb_accidents),
  12
)
#>      annee mois nb_accidents
#> 169 2021    1         180
#> 170 2021    2         125
#> 171 2021    3         165
#> 172 2021    4         188
#> 173 2021    5         208
#> 174 2021    6         258
#> 175 2021    7         217
#> 176 2021    8         131
#> 177 2021    9         274
#> 178 2021   10         258
#> 179 2021   11         235
#> 180 2021   12         205
```

Les premières observations correspondent aux douze mois de l'année 2007, de janvier à décembre, tandis que les dernières observations couvrent les douze mois de l'année 2021, également de janvier à décembre. Les données sont présentées dans l'ordre chronologique, ce qui confirme que la table mensuelle importée est correctement ordonnée et couvre l'ensemble de la période disponible dans MySQL, de 2007 à 2021.

4.4.2 Statistiques descriptives

Cette étape produit un résumé statistique des principales variables de la base mensuelle afin d'évaluer leur distribution avant la modélisation.

```
summary(sarimax_sql)
#>      annee      mois      nb_accidents      pct_nuit
#> Min.   :2007   Min.   : 1.00   Min.   : 35.0   Min.   :0.0667
```

```

#> 1st Qu.:2010    1st Qu.: 3.75    1st Qu.:192.0    1st Qu.:0.1276
#> Median :2014    Median : 6.50    Median :219.5    Median :0.2040
#> Mean   :2014    Mean   : 6.50    Mean   :215.3    Mean   :0.2365
#> 3rd Qu.:2018    3rd Qu.: 9.25    3rd Qu.:246.0    3rd Qu.:0.3499
#> Max.   :2021    Max.   :12.00    Max.   :324.0    Max.   :0.5187
#>   pct_meteo      pct_intersection pct_autoroute      pct_pieton
#> Min.   :0.00000    Min.   :0.2727    Min.   :0.04610    Min.   :0.0970
#> 1st Qu.:0.08655    1st Qu.:0.3987    1st Qu.:0.09168    1st Qu.:0.1880
#> Median :0.13235    Median :0.4330    Median :0.12035    Median :0.2208
#> Mean   :0.15504    Mean   :0.4337    Mean   :0.11886    Mean   :0.2225
#> 3rd Qu.:0.22212    3rd Qu.:0.4768    3rd Qu.:0.14303    3rd Qu.:0.2562
#> Max.   :0.42250    Max.   :0.5987    Max.   :0.21630    Max.   :0.3761
#>   pct_moto
#> Min.   :0.2080
#> 1st Qu.:0.3333
#> Median :0.3571
#> Mean   :0.3588
#> 3rd Qu.:0.3919
#> Max.   :0.5355

```

Les statistiques descriptives confirment que les données couvrent la période allant de 2007 à 2021. La variable `nb_accidents` présente un minimum de 35 accidents mensuels, un maximum de 324 et une moyenne de 215,3 accidents par mois, avec une médiane de 219,5.

Les variables explicatives sont exprimées sous forme de proportions. Par exemple, la proportion d'accidents de nuit (`pct_nuit`) varie de 0,0667 à 0,5187, avec une moyenne de 0,2365. La proportion d'accidents impliquant un piéton (`pct_pieton`) est en moyenne de 0,2225, tandis que celle des accidents impliquant un deux-roues motorisé (`pct_moto`) atteint 0,3588 en moyenne. Les autres variables (`pct_meteo`, `pct_intersection` et `pct_autoroute`) présentent également des valeurs comprises entre leurs minimums et maximums, ce qui décrit la variabilité des indicateurs mensuels calculés dans MySQL.

4.4.3 Vérification des valeurs manquantes

Cette dernière vérification consiste à contrôler la présence éventuelle de valeurs manquantes dans chacune des variables de la table mensuelle.

```

sapply(
  sarimax_sql,
  function(x) sum(is.na(x))
)
#>      annee      mois  nb_accidents  pct_nuit
#>      0         0         0           0
#>  pct_meteo pct_intersection  pct_autoroute  pct_pieton
#>      0         0         0           0
#>      pct_moto
#>      0

```

Les résultats montrent que chacune des neuf variables de la table présente 0 valeur manquante. Les variables `annee`, `mois`, `nb_accidents`, `pct_nuit`, `pct_meteo`, `pct_intersection`, `pct_autoroute`, `pct_pieton` et `pct_moto` sont donc entièrement renseignées.

Ce contrôle confirme que la table mensuelle importée ne contient aucune valeur manquante et qu'aucun traitement complémentaire n'est nécessaire avant son utilisation dans les analyses descriptives et la modélisation des séries temporelles.

5 Préparation des données pour le modèle SARIMAX

Les données mensuelles importées depuis la base MySQL sont maintenant préparées pour la modélisation SARIMAX. Cette étape consiste à vérifier que les variables explicatives sont correctement typées et directement exploitables par les fonctions de modélisation de R. Les contrôles déjà réalisés lors de l'importation de la table mensuelle ne sont pas répétés afin d'éviter les redondances.

5.1 Préparation des variables numériques

Cette étape permet de définir les variables quantitatives qui seront utilisées dans le modèle SARIMAX. Toutes les variables explicatives ainsi que la variable réponse sont ensuite converties en format numérique afin d'assurer leur compatibilité avec les méthodes d'estimation.

5.1.1 Variables utilisées dans le modèle

```
vars_numeriques <- c(  
  "nb_accidents",  
  "pct_nuit",  
  "pct_meteo",  
  "pct_intersection",  
  "pct_autoroute",  
  "pct_pieton",  
  "pct_moto"  
)
```

La liste vars_numeriques regroupe les sept variables qui seront utilisées pour la modélisation SARIMAX. Elle comprend la variable réponse nb_accidents ainsi que les six variables explicatives pct_nuit, pct_meteo, pct_intersection, pct_autoroute, pct_pieton et pct_moto. Ces variables correspondent aux indicateurs présents dans la table mensuelle importée sous R.

5.1.2 Conversion des variables en type numérique

Cette étape garantit que toutes les variables quantitatives possèdent le format attendu par les fonctions de modélisation des séries temporelles.

```
sarimax_sql[vars_numeriques] <- lapply(  
  sarimax_sql[vars_numeriques],  
  as.numeric  
)
```

Le code applique la fonction as.numeric() aux sept variables définies dans vars_numeriques. Cette conversion garantit que la variable réponse ainsi que les variables explicatives sont stockées sous un format numérique compatible avec les fonctions de modélisation utilisées pour l'estimation du modèle SARIMAX.

5.2 Contrôle des variables

Avant la modélisation, les variables explicatives sont contrôlées afin de vérifier leur variabilité. Une variable sans variation ne pourrait pas contribuer à expliquer les fluctuations du nombre d'accidents.

5.2.1 Vérification de la variabilité des variables

```
sapply(  
  sarimax_sql[, vars_numeriques[-1]],  
  var,  
  na.rm = TRUE
```

```
)
#>      pct_nuit      pct_meteo pct_intersection      pct_autoroute
#>      0.016131852      0.008120072      0.003397376      0.001239072
#>      pct_pieton      pct_moto
#>      0.002485067      0.002283944
```

Les variances calculées sont toutes strictement positives. Elles sont respectivement égales à 0,0161 pour `pct_nuit`, 0,0081 pour `pct_meteo`, 0,0034 pour `pct_intersection`, 0,0012 pour `pct_autoroute`, 0,0025 pour `pct_pieton` et 0,0023 pour `pct_moto`.

Ces résultats montrent qu'aucune des variables explicatives n'est constante au cours de la période étudiée. Elles présentent toutes une variabilité et peuvent donc être utilisées comme variables explicatives dans le modèle SARIMAX.

5.3 Validation finale des données préparées

Cette dernière étape consiste à vérifier que les données préparées sont complètes et prêtes à être utilisées dans la phase de modélisation. Contrairement au chapitre précédent, seuls les contrôles indispensables à la modélisation sont conservés.

6 Analyse exploratoire de la série temporelle (EDA)

Les données étant désormais préparées pour la modélisation, une analyse exploratoire de la série temporelle (Exploratory Data Analysis – EDA) est réalisée afin de caractériser le comportement du nombre mensuel d'accidents corporels. Cette étape permet d'étudier l'évolution de la série, sa distribution, sa saisonnalité, sa variabilité ainsi que sa structure de dépendance temporelle. Les résultats obtenus orienteront le choix et la spécification du modèle SARIMAX.

6.1 Construction de la série temporelle

Cette étape consiste à convertir le nombre mensuel d'accidents en objet de type série temporelle (`ts`) afin de permettre l'utilisation des méthodes spécifiques aux séries temporelles disponibles sous R.

```
accidents_ts <- ts(
  sarimax_sql$nb_accidents,
  start = c(2007, 1),
  frequency = 12
)
```

La variable `nb_accidents`, issue des agrégations mensuelles réalisées dans MySQL, est transformée en une série temporelle mensuelle couvrant la période de janvier 2007 à décembre 2021. Cette représentation constitue la base des analyses exploratoires et des modèles SARIMAX.

6.2 Vérification de la série temporelle

Avant de réaliser les analyses graphiques, plusieurs contrôles sont effectués afin de vérifier les principales caractéristiques de la série temporelle.

```
start(accidents_ts)
#> [1] 2007    1

end(accidents_ts)
#> [1] 2021   12

frequency(accidents_ts)
```

```
#> [1] 12

length(accidents_ts)
#> [1] 180

summary(accidents_ts)
#>      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
#>    35.0   192.0   219.5   215.3   246.0   324.0
```

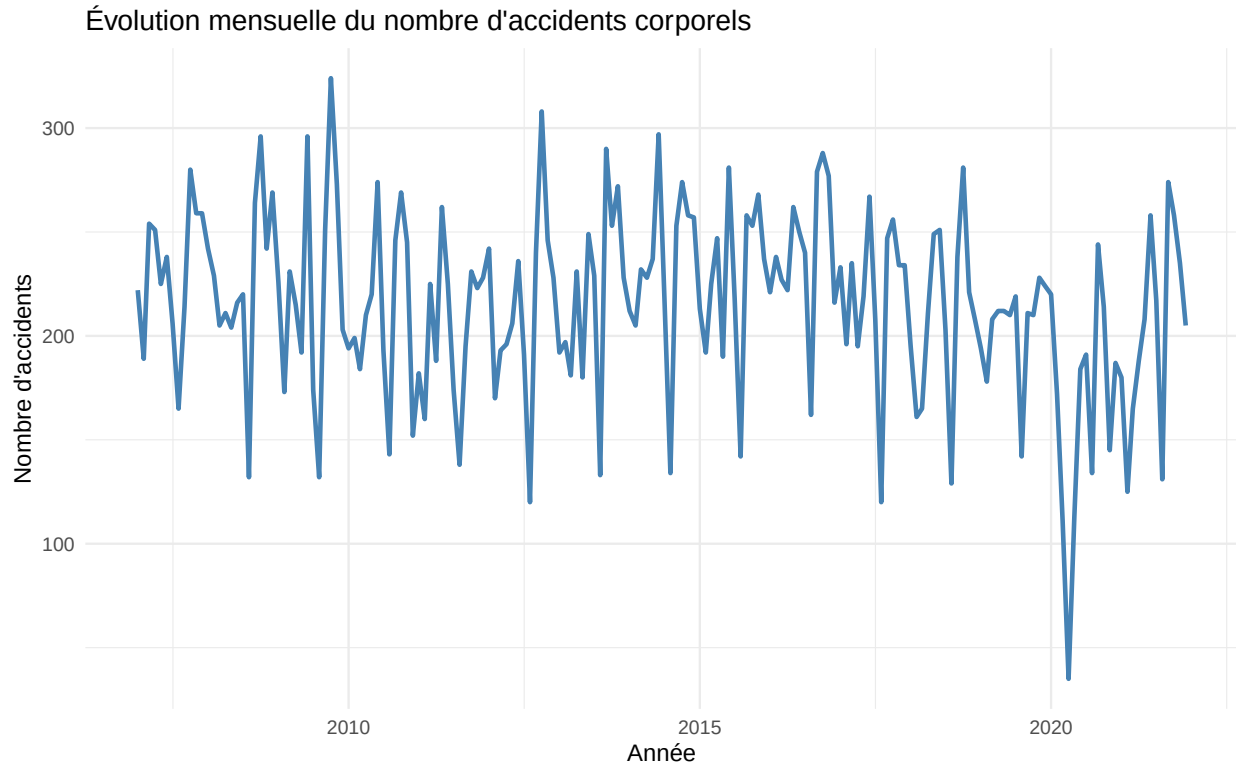
Les résultats montrent que la série débute en janvier 2007 et se termine en décembre 2021, avec une fréquence mensuelle (12 observations par an) pour un total de 180 observations. Les statistiques descriptives indiquent un nombre mensuel d'accidents compris entre 35 et 324, avec une moyenne d'environ 215 accidents par mois. Ces caractéristiques confirment que la série est correctement construite.

6.3 Évolution de la série

La représentation graphique permet d'observer l'évolution du nombre mensuel d'accidents au cours de la période d'étude et d'identifier d'éventuelles tendances ou ruptures.

```
df_series <- data.frame(
  Date = seq(
    from = as.Date("2007-01-01"),
    by = "month",
    length.out = length(accidents_ts)
  ),
  Accidents = as.numeric(accidents_ts)
)

ggplot(
  df_series,
  aes(
    x = Date,
    y = Accidents
  )
) +
  geom_line(
    colour = "steelblue",
    linewidth = 1
  ) +
  labs(
    title = "Évolution mensuelle du nombre d'accidents corporels",
    x = "Année",
    y = "Nombre d'accidents"
  ) +
  theme_minimal()
```



```
ggsave("graphics/01_evolution_mensuelle.png", width = 8, height = 5, dpi = 300)
```

Le graphique met en évidence une évolution irrégulière du nombre mensuel d'accidents corporels sur la période 2007–2021. Le nombre d'accidents oscille le plus souvent entre environ 150 et 300 accidents par mois, avec plusieurs pics dépassant ponctuellement 300 accidents, notamment autour des années 2009, 2013 et 2015.

Aucune tendance générale durable à la hausse ou à la baisse n'apparaît sur l'ensemble de la période. En revanche, une baisse exceptionnelle est observée en 2020, avec un minimum d'environ 35 accidents sur un mois, nettement inférieur aux niveaux observés durant le reste de la période. Après cet épisode, la série retrouve progressivement des niveaux plus proches des années précédentes.

Le graphique laisse également apparaître des fluctuations qui semblent se répéter au cours des années, suggérant la présence d'une composante saisonnière qui sera étudiée plus précisément dans les étapes suivantes.

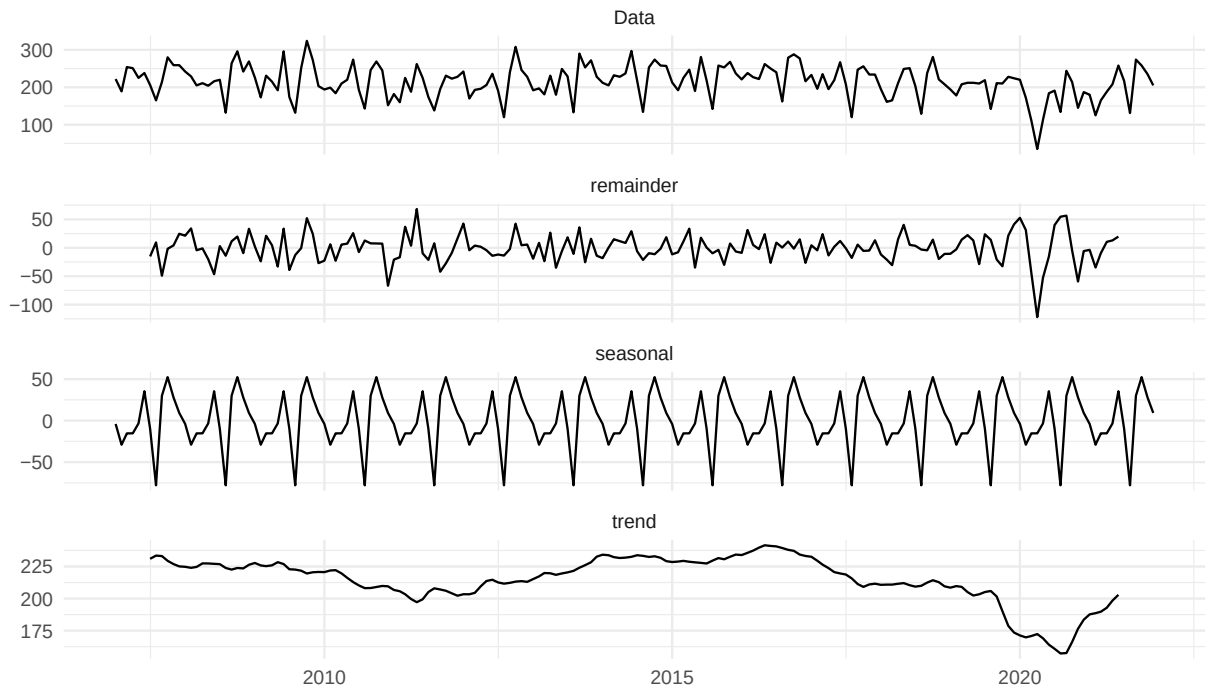
6.4 Décomposition de la série

La série est décomposée en ses composantes de tendance, de saisonnalité et de résidus afin de mieux comprendre sa structure.

```
dec <- decompose(accidents_ts)

autoplot(dec) +
  theme_minimal() +
  ggtitle(
    "Décomposition de la série mensuelle des accidents corporels"
  )
```

Décomposition de la série mensuelle des accidents corporels



```
ggsave("graphics/02_decomposition.png", width = 8, height = 5, dpi = 300)
```

La décomposition met en évidence les différentes composantes de la série temporelle.

La composante de tendance montre une légère diminution entre 2007 et 2011, suivie d'une hausse progressive jusqu'aux environs de 2016, avant une nouvelle baisse à partir de 2017. Cette diminution devient particulièrement marquée en 2020, puis la tendance remonte progressivement en fin de période.

La composante saisonnière présente un profil relativement stable d'une année à l'autre. Les mêmes variations mensuelles se répètent régulièrement, ce qui confirme l'existence d'une saisonnalité récurrente dans le nombre d'accidents corporels.

Enfin, les résidus sont globalement centrés autour de zéro, mais présentent des fluctuations plus importantes durant l'année 2020, traduisant un événement exceptionnel qui n'est pas entièrement expliqué par les composantes de tendance et de saisonnalité.

Cette décomposition confirme que la série comporte à la fois une composante saisonnière et une évolution de long terme, ce qui justifie l'utilisation d'un modèle SARIMAX intégrant une saisonnalité.

6.5 Distribution de la variable cible

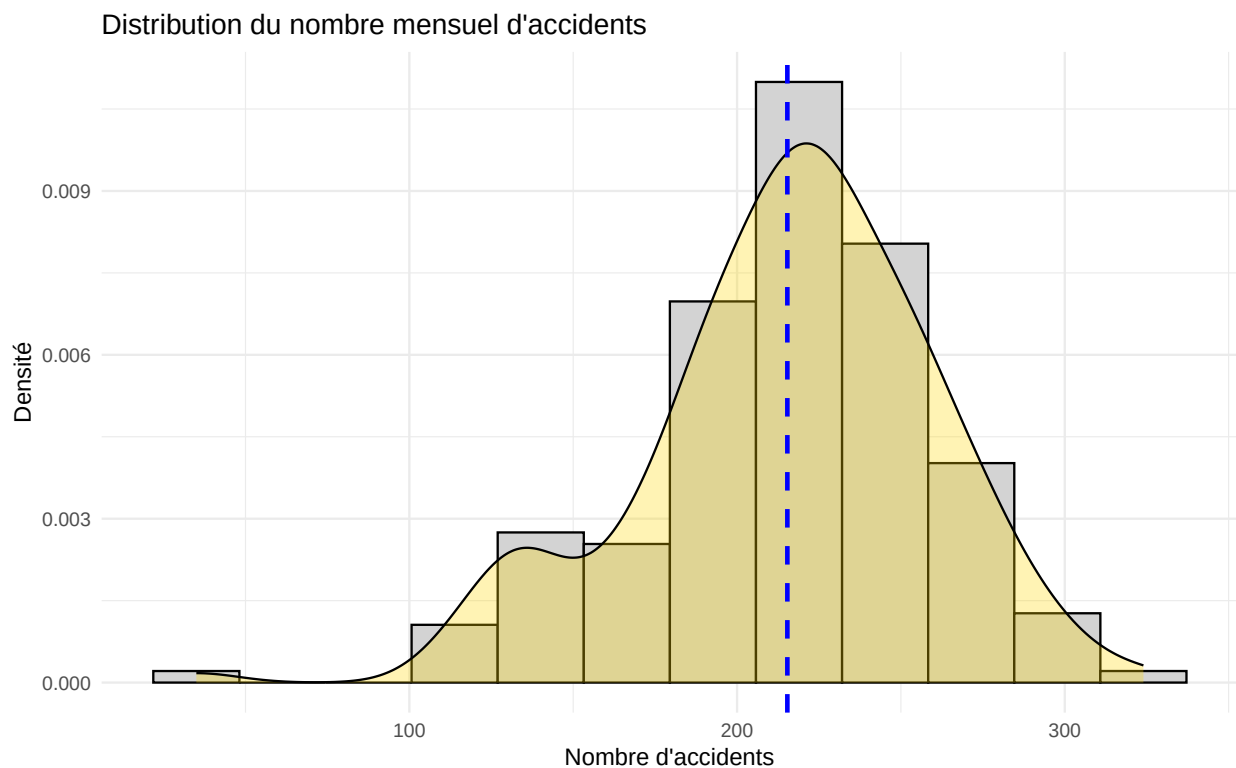
Cette étape permet d'étudier la distribution du nombre mensuel d'accidents afin d'apprécier sa forme générale et sa dispersion.

```
df_hist <- data.frame(  
  accidents = as.numeric(accidents_ts)  
)  
  
ggplot(  
  df_hist,  
  aes(  
    #
```

```

    x = accidents,
    y = after_stat(density)
  )
) +
  geom_histogram(
    bins = 12,
    fill = "lightgray",
    colour = "black"
  ) +
  geom_density(
    fill = "gold",
    alpha = 0.30
  ) +
  geom_vline(
    xintercept = mean(df_hist$accidents),
    colour = "blue",
    linetype = "dashed",
    linewidth = 1
  ) +
  labs(
    title = "Distribution du nombre mensuel d'accidents",
    x = "Nombre d'accidents",
    y = "Densité"
  ) +
  theme_minimal()

```



```

ggsave("graphics/03_distribution_de_la_variable_cible.png", width = 8, height = 5, dpi = 300)

```

L'histogramme montre que le nombre mensuel d'accidents est principalement concentré autour de 200 à 250 accidents par mois, avec une moyenne représentée par la ligne verticale située à environ 215 accidents.

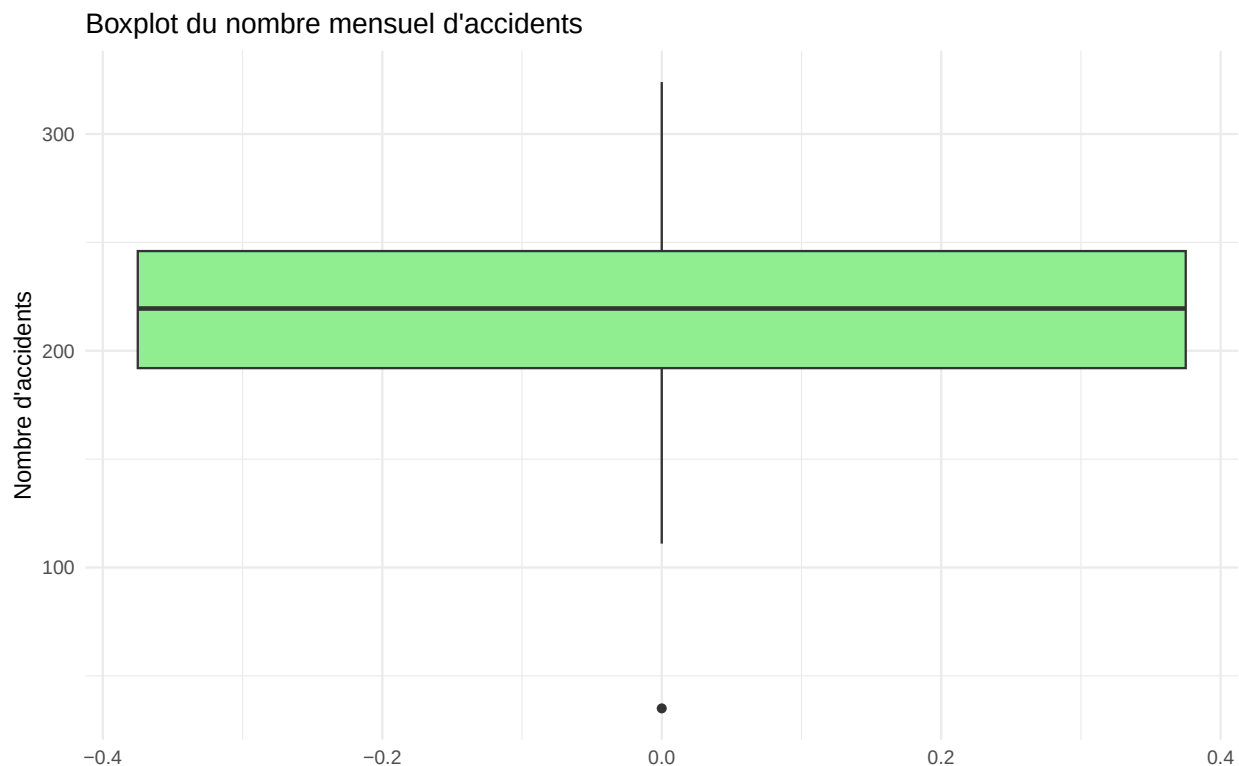
La distribution est unimodale et globalement centrée autour de cette moyenne. Elle présente toutefois une légère asymétrie vers les faibles valeurs, liée à quelques mois ayant enregistré un nombre d'accidents particulièrement faible, notamment durant l'année 2020.

Dans l'ensemble, la distribution reste relativement régulière malgré la présence de quelques observations extrêmes.

6.6 Boxplot de la variable cible

Le diagramme en boîte permet de visualiser la dispersion des observations ainsi que la présence éventuelle de valeurs atypiques.

```
ggplot(  
  data.frame(  
    accidents = as.numeric(accidents_ts)  
  ),  
  aes(y = accidents)  
) +  
  geom_boxplot(  
    fill = "lightgreen"  
  ) +  
  labs(  
    title = "Boxplot du nombre mensuel d'accidents",  
    x = NULL,  
    y = "Nombre d'accidents"  
  ) +  
  theme_minimal()
```



```
ggsave("graphics/04_boxplot_de_la_Variable_cible.png", width = 8, height = 5, dpi = 300)
```

Le diagramme en boîte montre que la moitié des observations est comprise approximativement entre 190 et 245 accidents par mois, avec une médiane proche de 220 accidents.

Le boxplot met également en évidence une observation atypique située autour de 35 accidents, nettement inférieure au reste des observations. Cette valeur correspond au minimum observé dans la série et est cohérente avec la forte baisse des accidents enregistrée durant l'année 2020.

En dehors de cette observation isolée, la dispersion de la série reste relativement homogène et ne révèle pas d'anomalies majeures susceptibles de compromettre la modélisation.

6.7 Statistiques descriptives

Les principales statistiques descriptives sont calculées afin de résumer la tendance centrale et la variabilité de la série.

```
cat("\n===== \n")
#>
#> =====
cat("ACCIDENTS CORPORELS\n")
#> ACCIDENTS CORPORELS
cat("===== \n")
#> =====

summary(accidents_ts)
#>      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
#>      35.0   192.0   219.5   215.3   246.0   324.0

cat(
  "\nVariance :",
  round(var(accidents_ts), 2)
)
#>
#> Variance : 1990.9

cat(
  "\nÉcart-type :",
  round(sd(accidents_ts), 2),
  "\n"
)
#>
#> Écart-type : 44.62
```

Les statistiques descriptives montrent que le nombre mensuel d'accidents varie entre 35 et 324, avec une moyenne de 215,3 accidents et une médiane de 219,5 accidents, ce qui indique que la majorité des observations est concentrée autour de cette valeur centrale.

L'écart-type est de 44,62 accidents et la variance atteint 1 990,90, traduisant une variabilité non négligeable du nombre d'accidents d'un mois à l'autre.

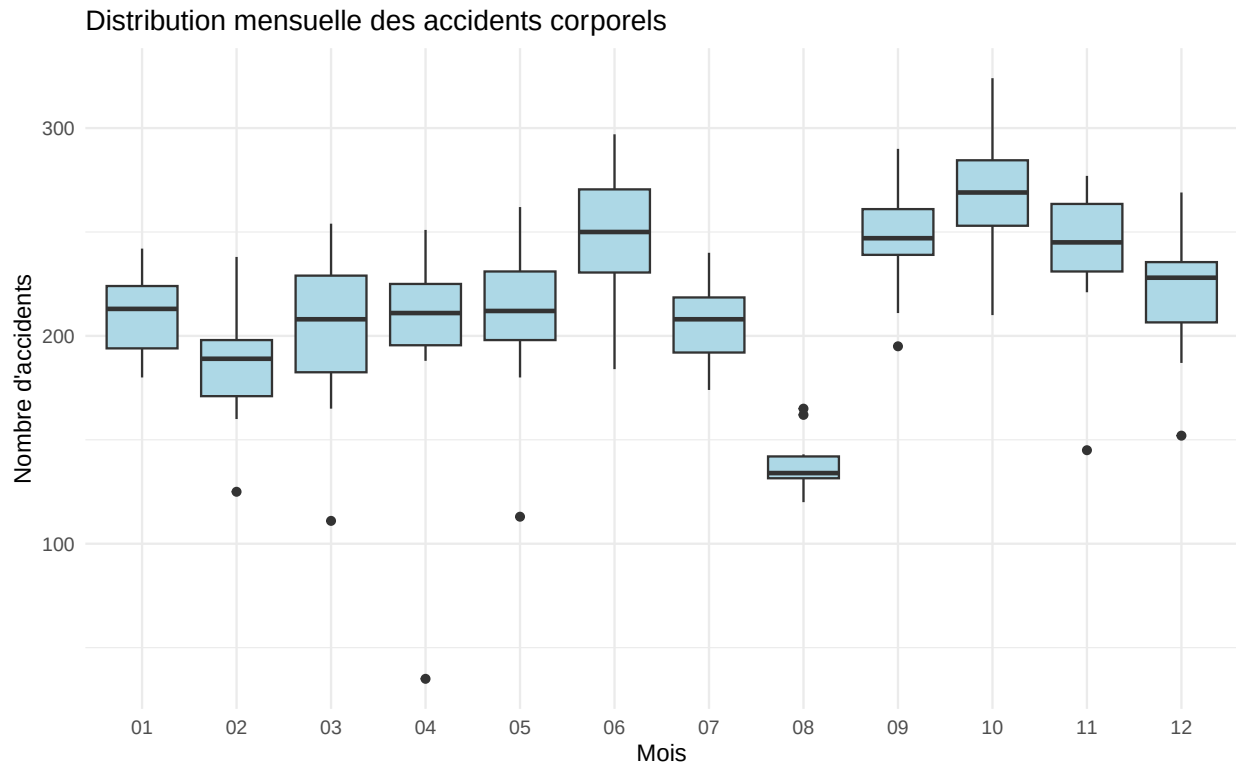
Les premier et troisième quartiles, respectivement égaux à 192 et 246 accidents, montrent que la moitié des observations est comprise dans cet intervalle.

Ces résultats confirment que la série présente une variabilité suffisante pour justifier une modélisation temporelle.

6.8 Distribution mensuelle

La distribution du nombre d'accidents est étudiée selon les mois de l'année afin de mettre en évidence une éventuelle saisonnalité.

```
df_ts <- data.frame(  
  Date = seq(  
    from = as.Date("2007-01-01"),  
    by = "month",  
    length.out = length(accidents_ts)  
  ),  
  Accidents = as.numeric(accidents_ts)  
)  
  
df_ts$Mois <- factor(  
  format(df_ts$Date, "%m"),  
  levels = sprintf("%02d", 1:12)  
)  
  
ggplot(  
  df_ts,  
  aes(  
    x = Mois,  
    y = Accidents  
  )  
) +  
  geom_boxplot(  
    fill = "lightblue"  
  ) +  
  labs(  
    title = "Distribution mensuelle des accidents corporels",  
    x = "Mois",  
    y = "Nombre d'accidents"  
  ) +  
  theme_minimal()
```



```
ggsave("graphics/05_distribution_mensuelle.png", width = 8, height = 5, dpi = 300)
```

Les boxplots mensuels mettent en évidence des différences selon les mois de l'année.

Les médianes sont généralement plus faibles en août, où le nombre mensuel d'accidents est le plus bas parmi les douze mois. À l'inverse, les mois de juin, septembre, octobre et novembre présentent des médianes plus élevées, traduisant un nombre d'accidents généralement supérieur durant ces périodes.

La dispersion varie également selon les mois. Certains mois, comme juin et octobre, présentent une variabilité plus importante, tandis que août affiche une distribution plus resserrée.

Quelques valeurs atypiques apparaissent également pour certains mois, notamment avril, novembre et décembre, correspondant à des mois où le nombre d'accidents a été exceptionnellement faible.

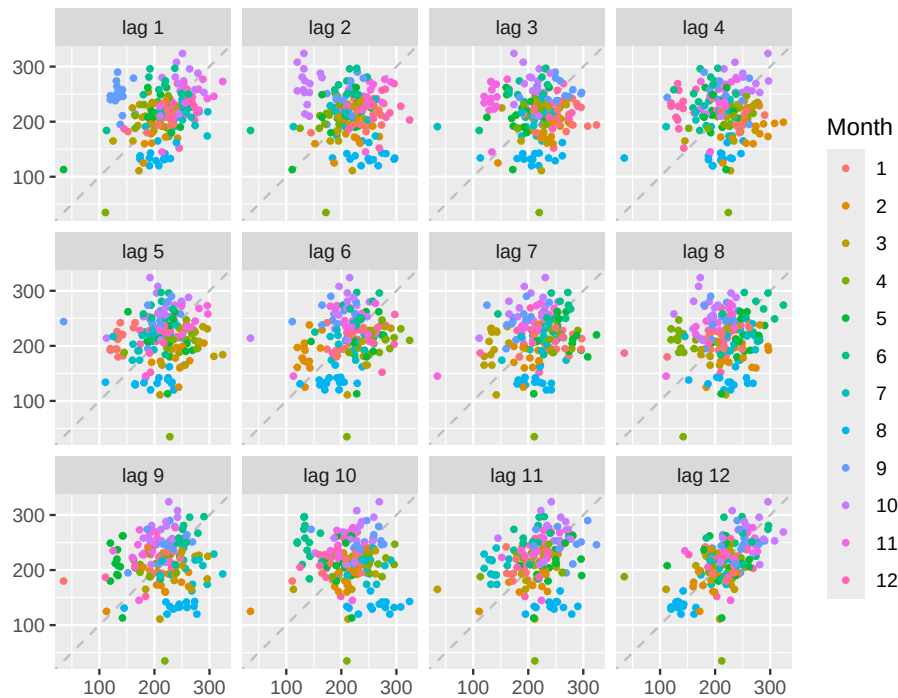
Dans l'ensemble, ces différences entre les distributions mensuelles confirment l'existence d'une saisonnalité dans la série temporelle, ce qui justifie l'intégration d'une composante saisonnière dans le modèle SARIMAX.

6.9 Lag plots

Les graphiques de retard permettent d'examiner les relations entre une observation et ses valeurs passées afin d'identifier une éventuelle dépendance temporelle.

```
gglagplot(
  accidents_ts,
  lags = 12,
  do.lines = FALSE
) +
  ggtitle("Lag plots de la série des accidents")
```

Lag plots de la série des accidents



Les graphiques de retard mettent en évidence une relation positive entre les observations pour plusieurs retards, en particulier pour les retards les plus élevés, où les nuages de points présentent une orientation croissante. Cette relation reste toutefois modérée, les observations demeurant relativement dispersées autour de la diagonale.

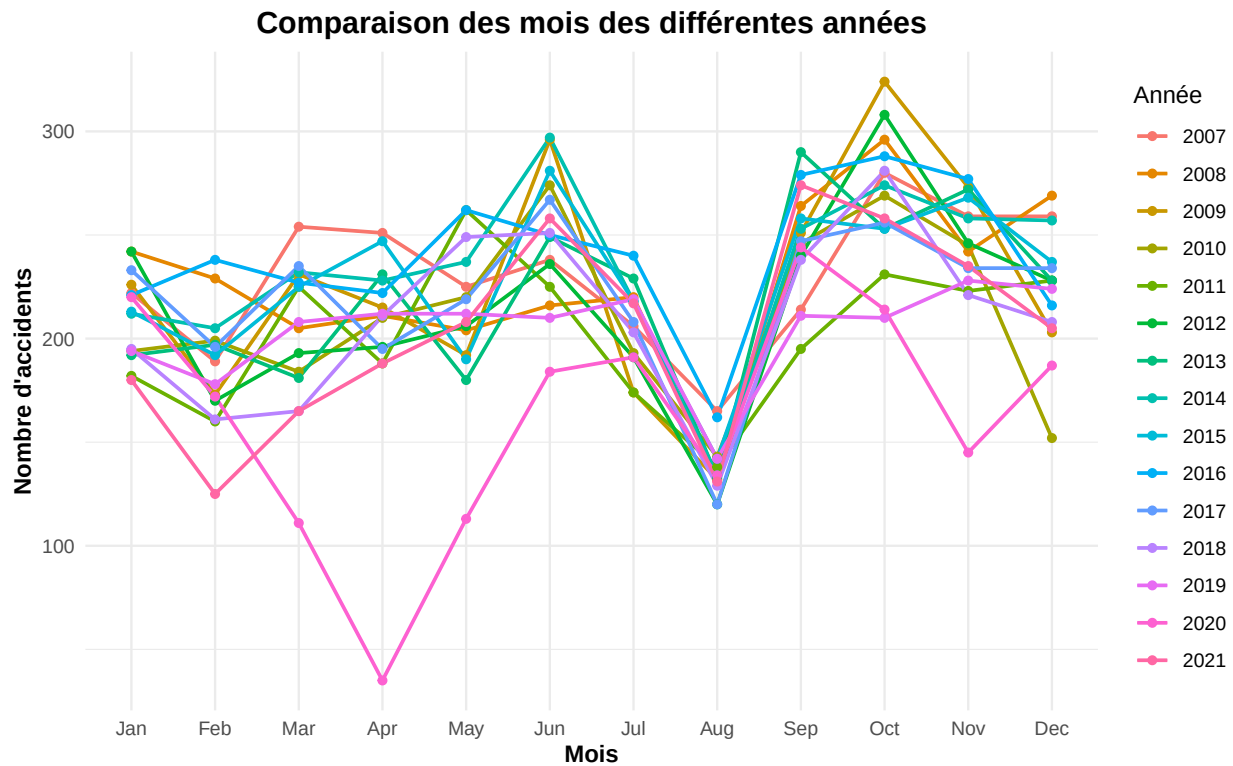
Les retards proches de 12 mois montrent une association plus marquée, ce qui suggère que le nombre d'accidents observé un mois donné est partiellement lié à celui enregistré au même mois de l'année précédente. Ces résultats mettent en évidence une dépendance temporelle compatible avec l'utilisation d'un modèle autorégressif comportant une composante saisonnière.

6.10 Graphique de saisonnalité par année

Afin de visualiser la reproductibilité de la composante saisonnière d'une année sur l'autre, une superposition des profils mensuels est réalisée.

```
ggseasonplot(
  accidents_ts,
  year.labels = FALSE,
  continuous = FALSE
) +
  geom_line(linewidth = 0.8) +
  geom_point(size = 1.5) +
  labs(
    title = "Comparaison des mois des différentes années",
    x = "Mois",
    y = "Nombre d'accidents",
    colour = "Année"
  ) +
  theme_minimal() +
  theme(
```

```
plot.title = element_text(face = "bold", hjust = 0.5, size = 14),
axis.title = element_text(face = "bold")
)
```



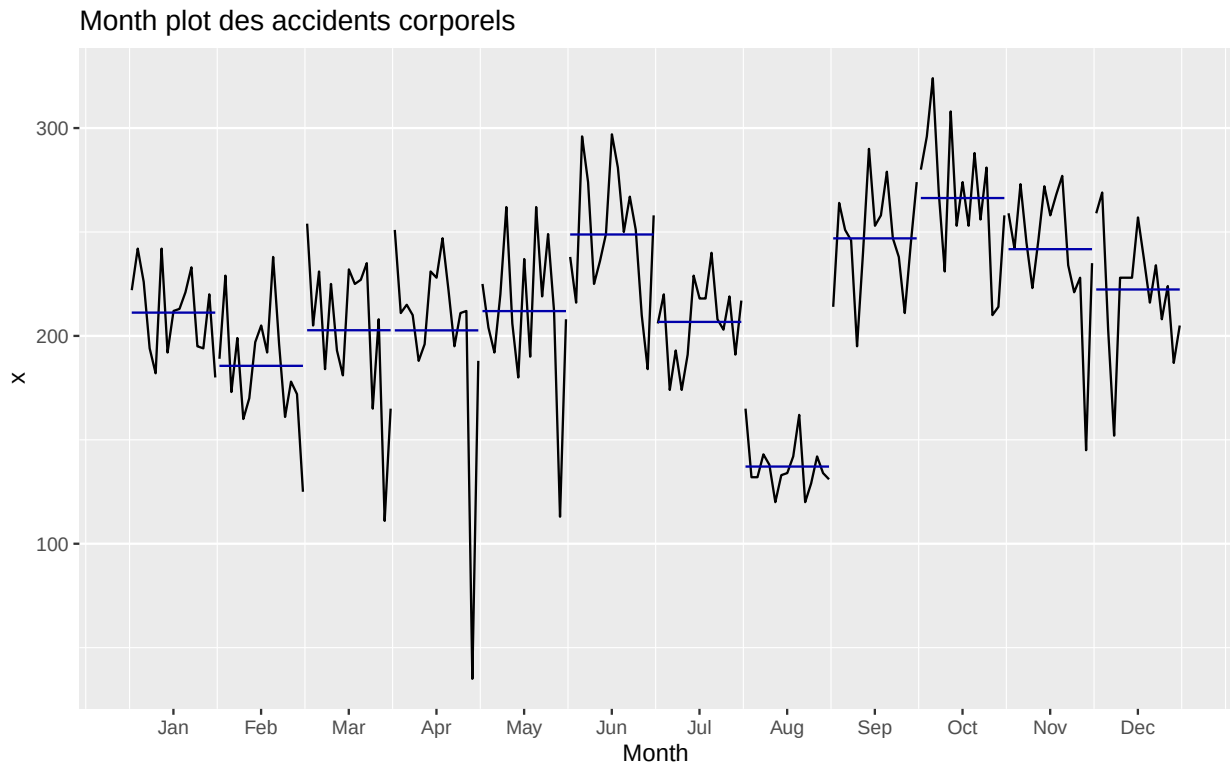
```
ggsave("graphics/06_Comparaison_interannuelle.png", width = 8, height = 5, dpi = 300)
```

La superposition des chroniques mensuelles met en évidence une régularité intra-annuelle stricte du nombre d'accidents sur la période historique. Deux dynamiques saisonnières majeures se répètent chaque année : un creux d'activité systématique et prononcé au mois d'août (coïncidant avec la baisse du trafic estival en milieu urbain), suivi d'un pic de sinistralité majeur à l'automne, particulièrement en octobre. Les trajectoires annuelles restent globalement resserrées dans un couloir allant de 150 à 300 accidents par mois, confirmant la stabilité de la saisonnalité avant la rupture structurelle observée lors de la crise sanitaire de 2020.

6.11 Graphique mensuel

Le graphique mensuel compare les observations d'un même mois sur plusieurs années afin d'évaluer la stabilité de la saisonnalité.

```
ggmonthplot(accidents_ts) +
  ggtitle("Month plot des accidents corporels")
```



Le graphique mensuel met en évidence des différences entre les mois de l'année. Les niveaux moyens sont généralement plus faibles en août, tandis qu'ils sont plus élevés entre juin et octobre, avec un maximum observé en octobre.

Une variabilité importante est également visible à l'intérieur de certains mois, notamment en avril, juin et octobre, où plusieurs observations s'écartent sensiblement de la moyenne mensuelle. La forte baisse observée en avril 2020 apparaît clairement comme une valeur atypique.

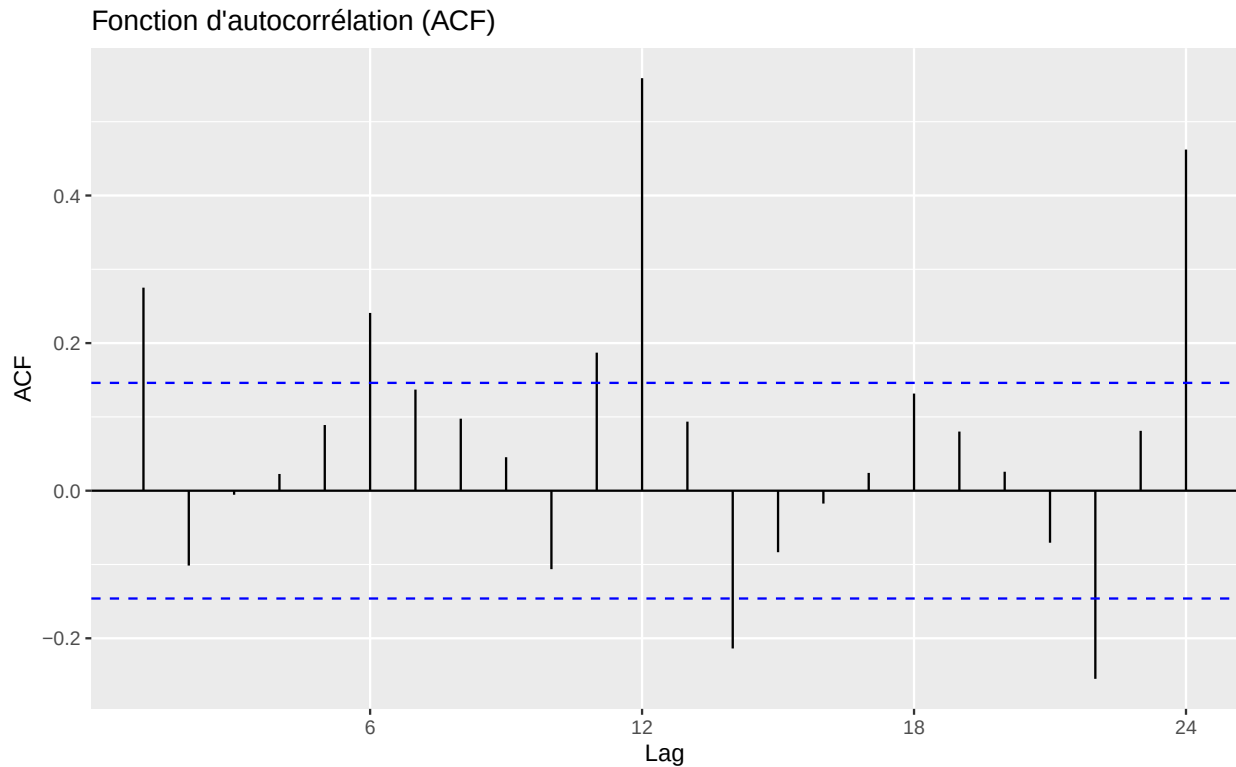
Dans l'ensemble, ce graphique confirme que les niveaux d'accidents varient selon les mois de l'année, ce qui renforce l'hypothèse d'une composante saisonnière dans la série.

6.12 Analyse des fonctions d'autocorrélation

6.12.1 Fonction d'autocorrélation (ACF)

La fonction d'autocorrélation permet de mesurer la dépendance entre les observations séparées par différents retards et constitue une étape essentielle pour l'identification du modèle SARIMA.

```
ggAcf(accidents_ts) +  
  ggtitle("Fonction d'autocorrélation (ACF)")
```



La fonction d'autocorrélation met en évidence plusieurs coefficients significatifs dépassant les bornes de confiance, notamment aux retards 1, 6, 11, 12, 14, 22 et 24 mois.

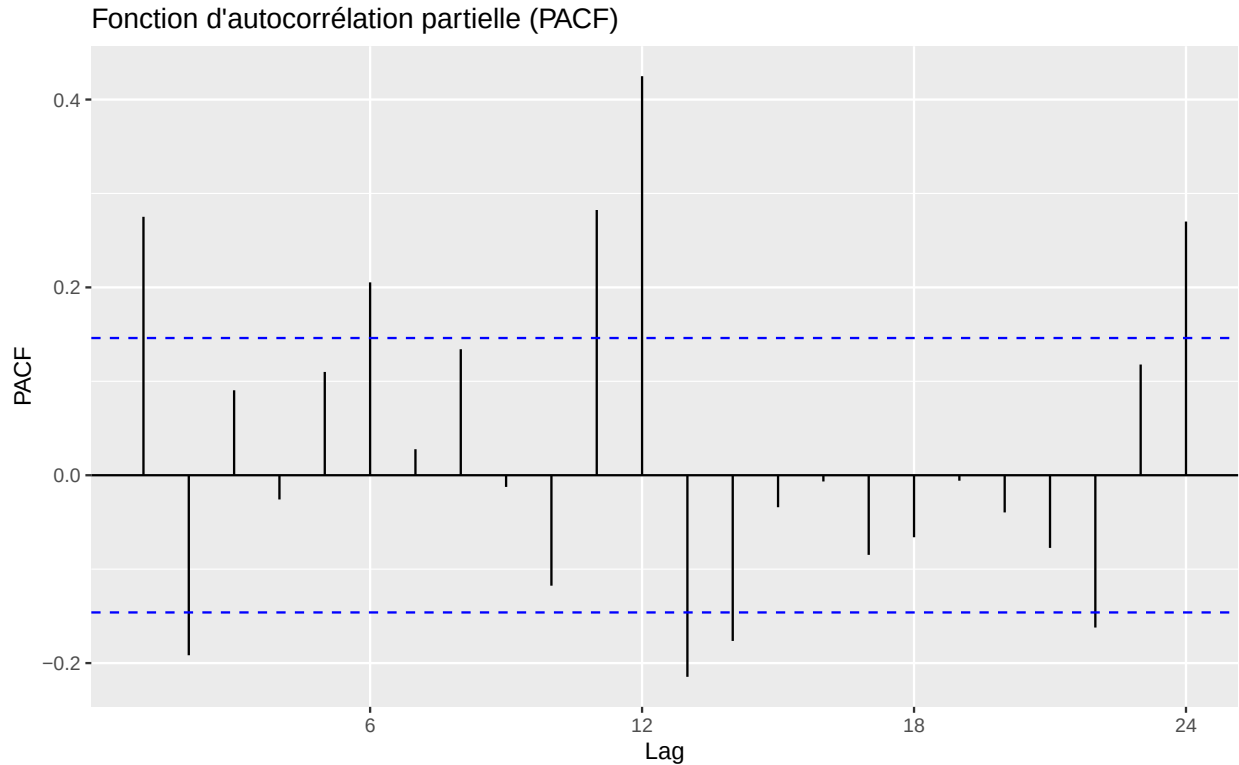
Les pics observés aux retards 12 et 24 mois sont particulièrement marqués, traduisant une forte dépendance annuelle dans la série. Cette structure confirme l'existence d'une saisonnalité de période 12 mois ainsi qu'une dépendance temporelle persistante entre les observations.

Ces résultats justifient l'utilisation d'un modèle SARIMAX intégrant des composantes autorégressives et saisonnières.

6.12.2 Fonction d'autocorrélation partielle (PACF)

La fonction d'autocorrélation partielle permet d'identifier les retards présentant une influence directe sur la série après prise en compte des dépendances intermédiaires.

```
ggPacf(accidents_ts) +  
  ggtitle("Fonction d'autocorrélation partielle (PACF)")
```



La fonction d'autocorrélation partielle présente plusieurs pics significatifs, notamment aux retards 1, 2, 6, 10, 11, 12, 13, 14, 22 et 24 mois.

Le pic observé au retard 12 est particulièrement important, ce qui confirme l'influence de la composante saisonnière annuelle déjà mise en évidence par l'ACF. Les autres retards significatifs traduisent également l'existence d'une dépendance directe entre certaines observations successives.

L'analyse conjointe de l'ACF et de la PACF fournit ainsi des indications utiles pour le choix des ordres autorégressifs et saisonniers du modèle SARIMAX.

7 Analyse exploratoire des variables exogènes

Cette étape vise à analyser les variables exogènes susceptibles d'améliorer la qualité des prévisions du modèle SARIMAX. L'objectif est d'étudier leur évolution temporelle, leur distribution, leurs relations mutuelles ainsi que leur niveau de corrélation afin de vérifier leur pertinence avant leur intégration dans le modèle.

7.1 Sélection des variables exogènes

Les variables explicatives retenues sont :

```
vars_exogenes <- c(
  "pct_nuit",
  "pct_meteo",
  "pct_intersection",
  "pct_autoroute",
  "pct_pieton",
  "pct_moto"
)
```

Ces variables décrivent les principales caractéristiques mensuelles des accidents et seront utilisées comme

variables exogènes du modèle SARIMAX.

7.2 Création des matrices d'apprentissage et de test

Afin d'évaluer les performances prédictives du modèle, les données sont séparées en un jeu d'apprentissage (2007–2019) et un jeu de test (2020–2021).

```
xreg_train <- sarimax_sql %>%
  filter(annee < 2020) %>%
  select(all_of(vars_exogenes)) %>%
  as.matrix()

xreg_test <- sarimax_sql %>%
  filter(annee >= 2020) %>%
  select(all_of(vars_exogenes)) %>%
  as.matrix()

# Standardisation des variables exogènes

xreg_train_scaled <- scale(xreg_train)

xreg_test_scaled <- scale(
  xreg_test,
  center = attr(xreg_train_scaled, "scaled:center"),
  scale = attr(xreg_train_scaled, "scaled:scale")
)
```

7.3 Contrôle de cohérence des données

Les dimensions, la structure et la cohérence des matrices sont vérifiées afin de s'assurer que les données sont correctement préparées pour la modélisation.

Les dimensions obtenues sont les suivantes :

```
dim(xreg_train)
#> [1] 156  6
dim(xreg_test)
#> [1] 24  6
```

Le jeu d'apprentissage comporte 156 observations mensuelles, tandis que le jeu de test en contient 24, ce qui correspond aux deux dernières années de la période d'étude.

7.3.1 Valeurs manquantes

```
colSums(is.na(xreg_train))
#>      pct_nuit      pct_meteo pct_intersection      pct_autoroute
#>           0           0           0           0
#>      pct_pieton      pct_moto
#>           0           0
colSums(is.na(xreg_test))
#>      pct_nuit      pct_meteo pct_intersection      pct_autoroute
#>           0           0           0           0
#>      pct_pieton      pct_moto
#>           0           0
```

7.3.2 Variances

```
apply(
  xreg_train,
  2,
  var,
  na.rm = TRUE
)
#>      pct_nuit      pct_meteo pct_intersection      pct_autoroute
#> 0.015945775 0.007545205 0.002601874 0.001203196
#>      pct_pieton      pct_moto
#> 0.002274211 0.001982505
```

7.3.3 Analyse des corrélations entre variables exogènes

Avant la modélisation, il est nécessaire d'examiner les relations existant entre les variables explicatives afin d'identifier d'éventuelles dépendances importantes.

```
round(
  cor(
    xreg_train,
    use = "complete.obs"
  ),
  3
)
#>      pct_nuit pct_meteo pct_intersection pct_autoroute pct_pieton
#> pct_nuit      1.000      0.702      0.024      0.085      0.543
#> pct_meteo      0.702      1.000      0.035      0.051      0.501
#> pct_intersection 0.024      0.035      1.000     -0.621      0.311
#> pct_autoroute    0.085      0.051     -0.621      1.000     -0.325
#> pct_pieton       0.543      0.501      0.311     -0.325      1.000
#> pct_moto       -0.253     -0.215     -0.199      0.065     -0.268
#>      pct_moto
#> pct_nuit      -0.253
#> pct_meteo      -0.215
#> pct_intersection -0.199
#> pct_autoroute    0.065
#> pct_pieton      -0.268
#> pct_moto       1.000
```

Les résultats montrent notamment :

une corrélation relativement forte entre la proportion d'accidents de nuit et les conditions météorologiques défavorables (0,70) ; une corrélation négative entre les intersections et les accidents sur autoroute (-0,62), traduisant leur caractère complémentaire ; des corrélations modérées ou faibles entre les autres variables.

Dans l'ensemble, aucune corrélation n'apparaît suffisamment élevée pour justifier la suppression immédiate d'une variable.

7.4 Evolution temporelle des variables exogènes

L'évolution des variables exogènes est représentée afin d'identifier d'éventuelles tendances ou ruptures au cours de la période d'étude.

```
df_time <- sarimax_sql %>%
  select(all_of(vars_exogenes)) %>%
```

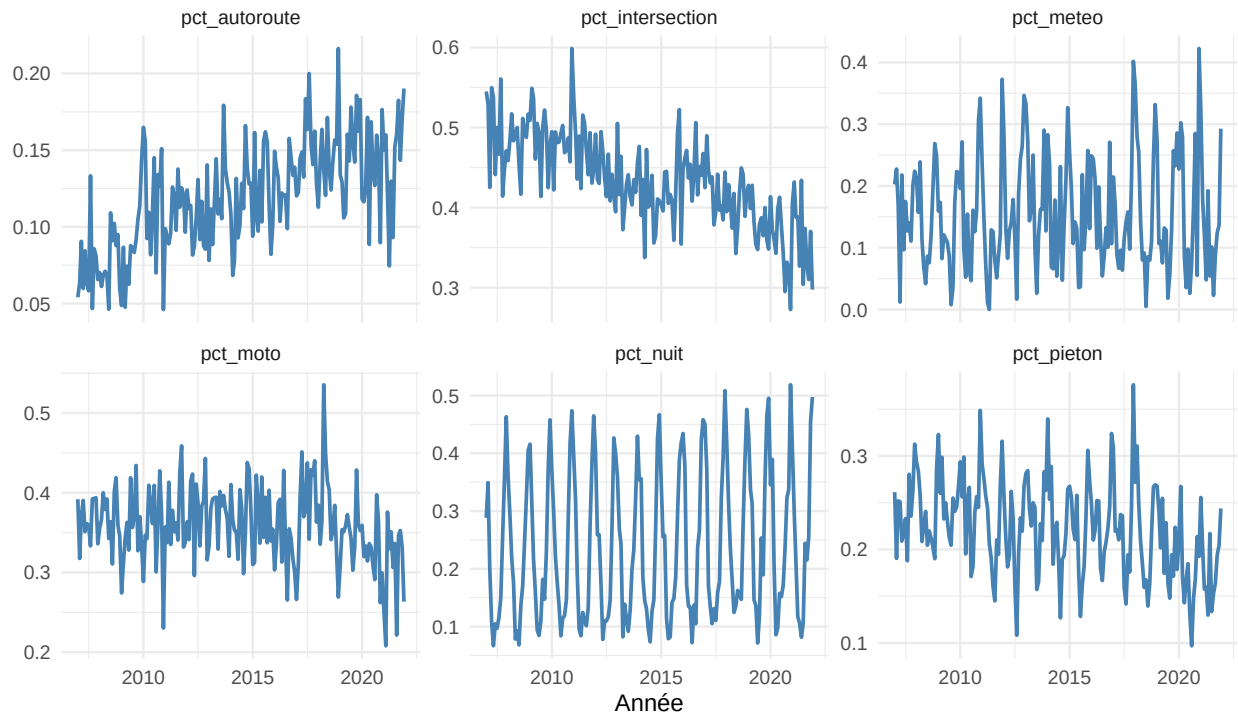
```

mutate(
  date = seq(
    from = as.Date("2007-01-01"),
    by = "month",
    length.out = n()
  )
) %>%
pivot_longer(
  cols = all_of(vars_exogenes),
  names_to = "variable",
  values_to = "valeur"
)

ggplot(
  df_time,
  aes(
    x = date,
    y = valeur
  )
) +
  geom_line(
    colour = "steelblue",
    linewidth = 0.8
  ) +
  facet_wrap(
    ~ variable,
    scales = "free_y",
    ncol = 3
  ) +
  labs(
    title = "Evolution temporelle des variables exogènes",
    x = "Année",
    y = NULL
  ) +
  theme_minimal()

```

Evolution temporelle des variables exogènes



Les graphiques montrent que les variables exogènes n'évoluent pas toutes de la même manière au cours de la période d'étude.

La proportion d'accidents sur autoroute (`pct_autoroute`) présente une tendance globale à la hausse, passant d'environ 0,06–0,08 au début de la période à des valeurs proches de 0,15–0,18 en fin de période, malgré des fluctuations mensuelles.

À l'inverse, la proportion d'accidents survenant en intersection (`pct_intersection`) montre une tendance progressive à la baisse, avec des valeurs proches de 0,50 au début de la série qui diminuent progressivement vers environ 0,35–0,40 à partir de 2020.

La variable `pct_meteo`, représentant la proportion d'accidents survenus sous des conditions météorologiques défavorables, présente d'importantes fluctuations d'un mois à l'autre, sans tendance de fond clairement identifiable.

La proportion d'accidents impliquant un deux-roues motorisé (`pct_moto`) oscille autour de 35 % sur l'ensemble de la période. Après une légère hausse jusqu'en 2018 environ, une diminution est observable au cours des dernières années.

La proportion d'accidents de nuit (`pct_nuit`) présente une forte saisonnalité, avec un cycle annuel très régulier marqué par des pics et des creux qui se répètent chaque année. Cette structure confirme le caractère saisonnier déjà observé lors de l'analyse de la série des accidents.

Enfin, la proportion d'accidents impliquant au moins un piéton (`pct_pieton`) fluctue autour de 20 à 25 %, sans tendance marquée sur l'ensemble de la période, bien que quelques variations ponctuelles apparaissent selon les années.

Dans l'ensemble, ces résultats montrent que les variables exogènes présentent des comportements temporels variés : certaines évoluent progressivement au cours du temps (`pct_autoroute` et `pct_intersection`), d'autres sont principalement caractérisées par des fluctuations mensuelles (`pct_meteo`, `pct_pieton` et `pct_moto`), tandis que `pct_nuit` présente une saisonnalité particulièrement marquée. Ces caractéristiques justifient leur prise en compte comme variables explicatives dans le modèle SARIMAX.

7.5 Distribution des variables exogènes

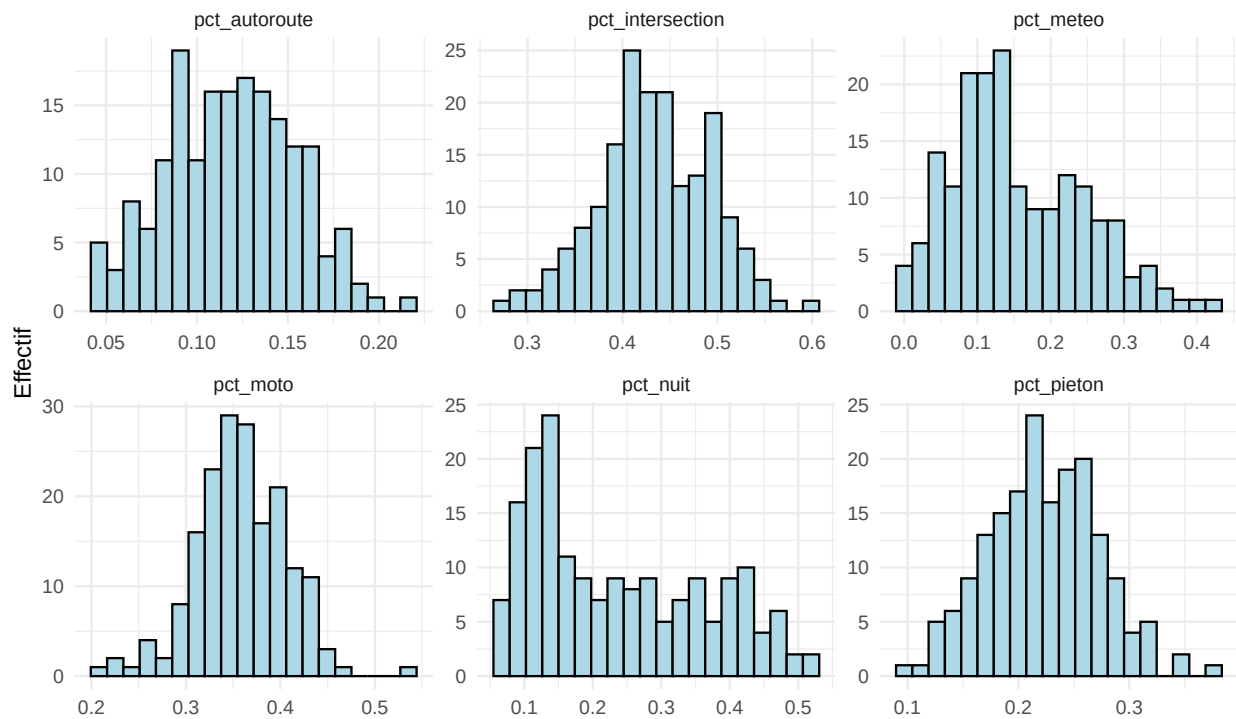
La distribution de chaque variable est examinée afin d'évaluer leur dispersion et d'identifier d'éventuelles valeurs atypiques.

```
df_exogenes <- sarimax_sql %>%  
  select(all_of(vars_exogenes)) %>%  
  pivot_longer(  
    cols = everything(),  
    names_to = "variable",  
    values_to = "valeur"  
  )
```

Histogrammes

```
ggplot(  
  df_exogenes,  
  aes(x = valeur)  
) +  
  geom_histogram(  
    bins = 20,  
    fill = "lightblue",  
    color = "black"  
  ) +  
  facet_wrap(  
    ~ variable,  
    scales = "free",  
    ncol = 3  
  ) +  
  labs(  
    title = "Distribution des variables exogènes",  
    x = NULL,  
    y = "Effectif"  
  ) +  
  theme_minimal()
```

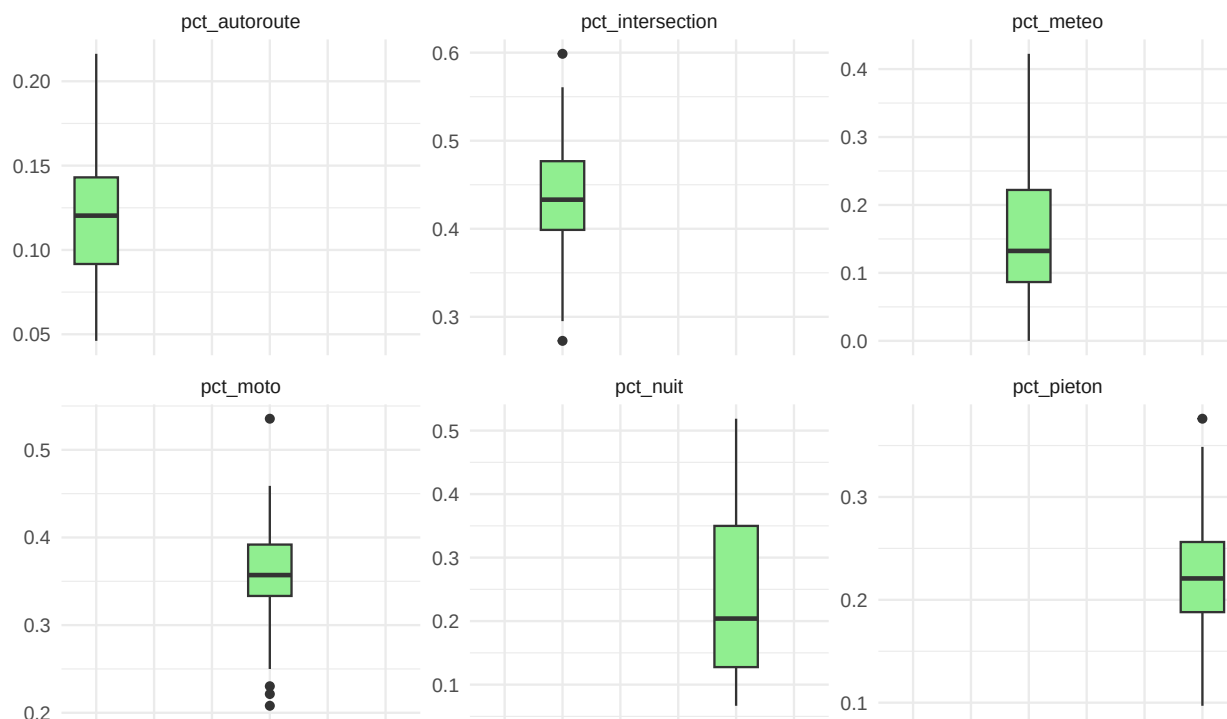
Distribution des variables exogènes



Boxplots

```
ggplot(
  df_exogenes,
  aes(
    x = variable,
    y = valeur
  )
) +
  geom_boxplot(
    fill = "lightgreen"
  ) +
  facet_wrap(
    ~ variable,
    scales = "free_y",
    ncol = 3
  ) +
  labs(
    title = "Boxplots des variables exogènes",
    x = NULL,
    y = NULL
  ) +
  theme_minimal() +
  theme(
    axis.text.x = element_blank(),
    axis.ticks.x = element_blank()
  )
)
```

Boxplots des variables exogènes



Les distributions des différentes variables exogènes présentent des profils variés. Les variables `pct_autoroute`, `pct_intersection`, `pct_moto` et `pct_pieton` sont globalement unimodales et relativement concentrées autour de leur moyenne. La variable `pct_meteo` est davantage dispersée, avec une légère asymétrie vers les valeurs élevées, tandis que `pct_nuit` présente une distribution plus étalée, reflétant les importantes variations saisonnières observées précédemment.

Les boxplots confirment cette dispersion. Quelques valeurs atypiques apparaissent pour certaines variables, notamment `pct_intersection`, `pct_moto` et `pct_pieton`, mais leur nombre reste limité. La variable `pct_nuit` présente la plus forte variabilité interquartile, ce qui est cohérent avec son comportement fortement saisonnier. Dans l'ensemble, aucune valeur extrême ne semble suffisamment importante pour justifier un traitement particulier avant la modélisation.

7.6 Analyse des corrélations

Matrice de corrélations

La matrice de corrélations permet d'évaluer les relations linéaires entre les variables exogènes et d'identifier d'éventuelles dépendances importantes.

```
correlations <- cor(
  sarimax_sql[, c(
    "nb_accidents",
    vars_exogenes
  )],
  use = "complete.obs"
)

round(correlations, 3)
#>               nb_accidents pct_nuit pct_meteo pct_intersection pct_autoroute
#> nb_accidents           1.000   0.108   0.052             0.077         -0.041
```

```

#> pct_nuit          0.108    1.000    0.701        -0.031    0.131
#> pct_meteo         0.052    0.701    1.000        -0.011    0.090
#> pct_intersection  0.077   -0.031   -0.011         1.000   -0.618
#> pct_autoroute     -0.041    0.131    0.090        -0.618    1.000
#> pct_pieton        0.265    0.508    0.466         0.417   -0.344
#> pct_moto          0.388   -0.255   -0.168         0.014   -0.045
#>                pct_pieton pct_moto
#> nb_accidents      0.265    0.388
#> pct_nuit          0.508   -0.255
#> pct_meteo         0.466   -0.168
#> pct_intersection  0.417    0.014
#> pct_autoroute     -0.344   -0.045
#> pct_pieton        1.000   -0.120
#> pct_moto          -0.120    1.000

```

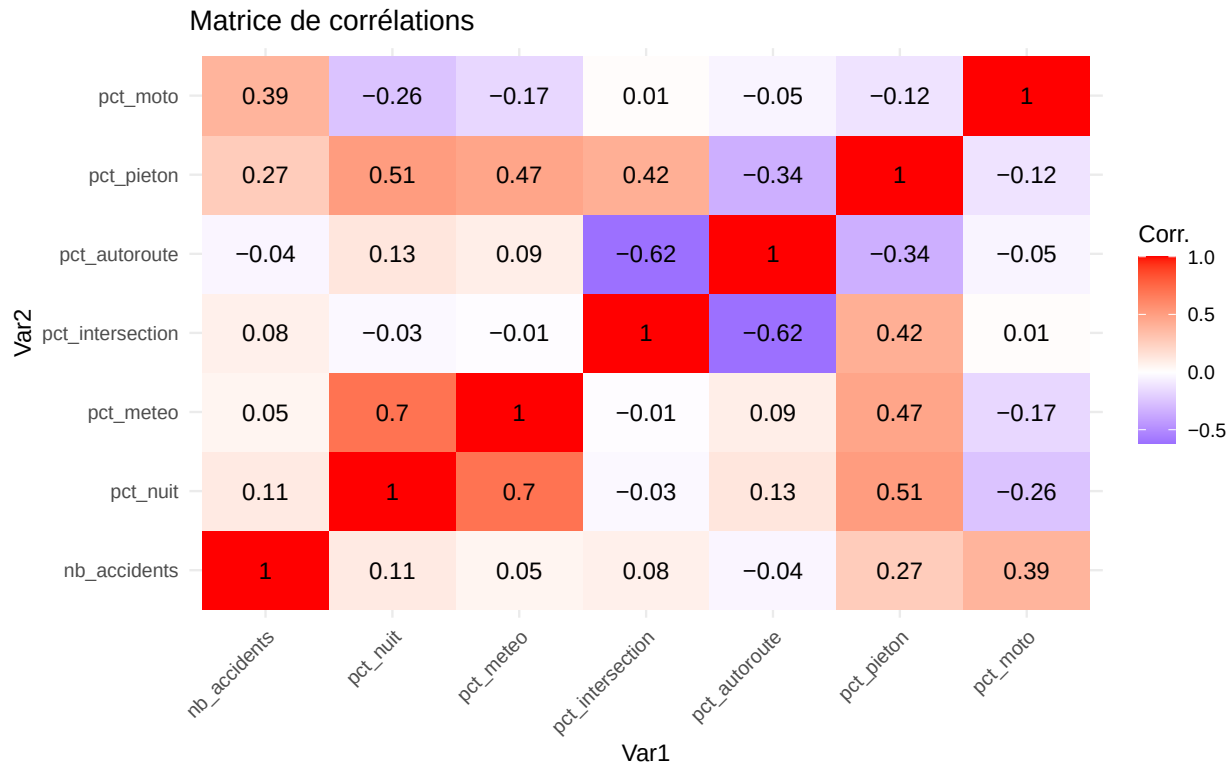
Afin de faciliter l'interprétation, la matrice est représentée sous forme de carte thermique.

```
cor_df <- reshape2::melt(correlations)
```

```

ggplot(
  cor_df,
  aes(
    x = Var1,
    y = Var2,
    fill = value
  )
) +
  geom_tile() +
  geom_text(
    aes(label = round(value, 2)),
    size = 4
  ) +
  scale_fill_gradient2(
    low = "blue",
    mid = "white",
    high = "red",
    midpoint = 0
  ) +
  labs(
    title = "Matrice de corrélations",
    fill = "Corr."
  ) +
  theme_minimal() +
  theme(
    axis.text.x = element_text(
      angle = 45,
      hjust = 1
    )
  )

```



```
ggsave("graphics/07_Matrice_de_corrélations.png", width = 8, height = 5, dpi = 300)
```

Les corrélations entre le nombre mensuel d'accidents et les variables exogènes restent globalement faibles à modérées.

La relation la plus élevée avec la variable cible est observée pour pct_moto ($r = 0,388$), suivie de pct_pieton ($r = 0,265$). À l'inverse, pct_autoroute présente une très faible corrélation négative avec le nombre d'accidents ($r = -0,041$), tandis que pct_nuit, pct_meteo et pct_intersection sont très faiblement corrélées à la variable réponse.

Entre variables explicatives, plusieurs relations plus marquées apparaissent. La proportion d'accidents de nuit (pct_nuit) est fortement corrélée à la proportion d'accidents survenus sous des conditions météorologiques défavorables (pct_meteo) ($r = 0,701$). Une corrélation négative relativement importante est également observée entre pct_intersection et pct_autoroute ($r = -0,618$), ce qui traduit le caractère complémentaire de ces deux types de localisation. Les autres corrélations demeurent faibles à modérées.

Dans l'ensemble, la matrice met en évidence des relations cohérentes entre certaines variables explicatives, sans révéler de dépendances suffisamment fortes pour remettre en cause leur utilisation simultanée dans le modèle SARIMAX.

7.6.1 Vérification de la multicollinéarité

La présence d'une éventuelle multicollinéarité est évaluée à l'aide du facteur d'inflation de variance (VIF).

```
vif_modele <- lm(
  nb_accidents ~ .,
  data = cbind(
    nb_accidents = sarimax_sql$nb_accidents,
    sarimax_sql[, vars_exogenes]
  )
)
```

```

vif_resultats <- car::vif(vif_modele)

round(vif_resultats, 2)
#>      pct_nuit      pct_meteo pct_intersection      pct_autoroute
#>      2.39      2.06      1.79      1.77
#>      pct_pieton      pct_moto
#>      2.01      1.07

max(vif_resultats)
#> [1] 2.389364

```

Les facteurs d'inflation de variance (VIF) sont compris entre 1,07 (pct_moto) et 2,39 (pct_nuit). Toutes les valeurs restent largement inférieures aux seuils généralement retenus pour caractériser une multicolinéarité problématique (5 ou 10).

Ces résultats indiquent que les corrélations observées entre certaines variables explicatives, notamment entre pct_nuit et pct_meteo, ne sont pas suffisamment importantes pour entraîner une instabilité des estimations du modèle. Les variables exogènes peuvent donc être conservées simultanément dans le modèle SARIMAX sans risque de multicolinéarité préoccupante.

8 Analyse de la stationnarité

Avant l'estimation d'un modèle SARIMAX, il est nécessaire de vérifier que la série temporelle satisfait l'hypothèse de stationnarité. Cette propriété garantit que les caractéristiques statistiques de la série demeurent stables au cours du temps et conditionne la validité des modèles ARIMA. L'analyse repose à la fois sur une inspection graphique et sur des tests statistiques complémentaires.

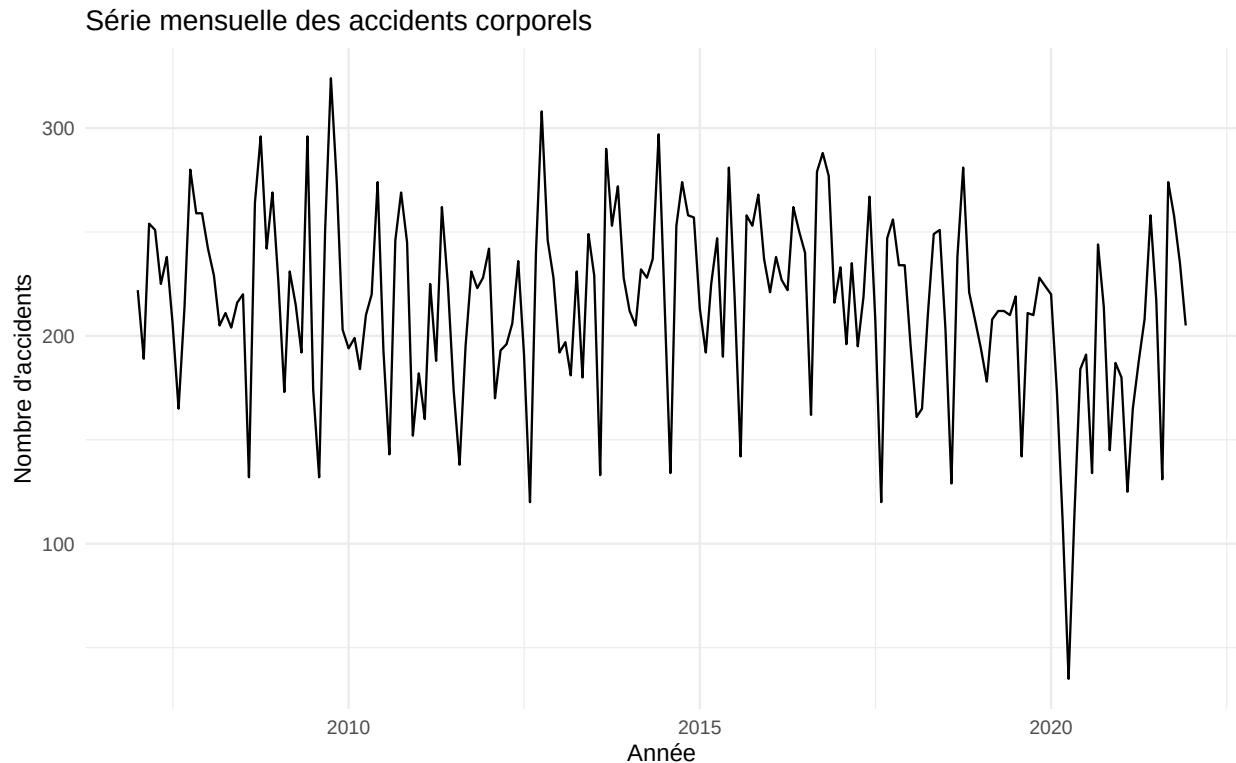
8.1 Représentation graphique

Une première évaluation est réalisée à partir de la représentation graphique de la série temporelle.

```

autoplot(accidents_ts) +
  labs(
    title = "Série mensuelle des accidents corporels",
    x = "Année",
    y = "Nombre d'accidents"
  ) +
  theme_minimal()

```



La représentation graphique montre que le nombre mensuel d'accidents fluctue fortement tout au long de la période d'étude. La série oscille généralement entre 150 et 300 accidents par mois, avec plusieurs pics et creux marqués. Une baisse particulièrement importante est observée en 2020, où le nombre d'accidents chute exceptionnellement à environ 35 accidents sur un mois avant de remonter progressivement.

La série ne présente pas de tendance linéaire clairement marquée sur l'ensemble de la période, mais les fluctuations observées ainsi que les variations récurrentes entre les mois suggèrent la présence d'une composante saisonnière déjà mise en évidence lors de l'analyse exploratoire. L'évaluation visuelle ne permet toutefois pas de conclure à la stationnarité de la série, ce qui justifie le recours à des tests statistiques.

8.2 Test de Dickey-Fuller augmenté (ADF)

Le test de Dickey-Fuller augmenté (ADF) permet d'évaluer l'hypothèse nulle de présence d'une racine unitaire, correspondant à une série non stationnaire.

```
adf.test(accidents_ts)
#>
#> Augmented Dickey-Fuller Test
#>
#> data: accidents_ts
#> Dickey-Fuller = -4.079, Lag order = 5, p-value = 0.01
#> alternative hypothesis: stationary
```

Le test de Dickey-Fuller augmenté fournit une statistique de test de $-4,079$ avec une p-value inférieure à $0,01$. L'hypothèse nulle de présence d'une racine unitaire est donc rejetée au seuil de 5% .

Ces résultats indiquent que la série peut être considérée comme stationnaire selon le test ADF. Autrement dit, aucune preuve statistique ne permet de conclure à l'existence d'une racine unitaire dans la série du nombre mensuel d'accidents.

8.3 Test KPSS

Le test KPSS est utilisé en complément du test ADF. Contrairement à ce dernier, son hypothèse nulle correspond à une série stationnaire.

```
kpss.test(accidents_ts)
#>
#> KPSS Test for Level Stationarity
#>
#> data: accidents_ts
#> KPSS Level = 0.55942, Truncation lag parameter = 4, p-value = 0.02828
```

Le test KPSS fournit une statistique de 0,559 avec une p-value de 0,028, inférieure au seuil de 5 %. L'hypothèse nulle de stationnarité est donc rejetée.

Contrairement au test ADF, le test KPSS suggère que la série présente une légère non-stationnarité. Les deux tests conduisent ainsi à des conclusions différentes, ce qui est fréquent lorsque la série présente simultanément une saisonnalité marquée, des fluctuations importantes ou une rupture ponctuelle, comme celle observée en 2020.

Compte tenu de ces résultats, il est prudent d'envisager une différenciation lors de la construction du modèle SARIMAX afin de garantir la stationnarité de la série.

8.4 Détermination du nombre de différenciations

Le nombre de différenciations ordinaires et saisonnières nécessaires est estimé automatiquement.

```
cat("Nombre de différenciations ordinaires :", ndiffs(accidents_ts), "\n")
#> Nombre de différenciations ordinaires : 1

cat("Nombre de différenciations saisonnières :", nsdiffs(accidents_ts), "\n")
#> Nombre de différenciations saisonnières : 1
```

Les fonctions automatiques recommandent une différenciation ordinaire ($d = 1$) et une différenciation saisonnière ($D = 1$).

Ces recommandations sont cohérentes avec les résultats des tests de stationnarité. Bien que le test ADF indique une série stationnaire, le test KPSS met en évidence une légère non-stationnarité, tandis que l'analyse exploratoire a montré l'existence d'une composante saisonnière marquée. L'application d'une différenciation ordinaire et d'une différenciation saisonnière permettra ainsi d'obtenir une série plus conforme aux hypothèses des modèles SARIMAX tout en conservant l'information utile à la prévision.

9 Construction et sélection des modèles SARIMA

Après avoir vérifié la stationnarité de la série et analysé les fonctions d'autocorrélation (ACF) et d'autocorrélation partielle (PACF), plusieurs modèles SARIMA sont estimés afin d'identifier la spécification la plus adaptée. L'identification initiale repose sur les résultats de l'analyse exploratoire (section 6.11), puis différents modèles sont comparés selon leur qualité d'ajustement, leurs diagnostics résiduels et leurs performances de prévision.

9.1 Constitution des jeux d'apprentissage et de test

La série temporelle est divisée en deux périodes :

un échantillon d'apprentissage couvrant la période de janvier 2007 à décembre 2019, utilisé pour estimer les modèles ; un échantillon de test couvrant la période de janvier 2020 à décembre 2021, réservé à l'évaluation des performances prédictives.

Les variables exogènes sont découpées selon la même période afin de garantir leur cohérence avec la variable cible.

```
accidents_train <- window(
  accidents_ts,
  end = c(2019, 12)
)

accidents_test <- window(
  accidents_ts,
  start = c(2020, 1)
)
```

9.2 Identification automatique d'un modèle de référence

Une première identification est réalisée à l'aide de la fonction `auto.arima()`, qui recherche automatiquement la combinaison de paramètres minimisant les critères d'information.

```
modele_sarima_auto <- auto.arima(
  accidents_train,
  seasonal = TRUE,
  stepwise = FALSE,
  approximation = FALSE,
)

arimaorder(modele_sarima_auto)
```

#>	<i>p</i>	<i>d</i>	<i>q</i>	<i>P</i>	<i>D</i>	<i>Q</i>	<i>Frequency</i>
#>	1	0	1	2	1	0	12

Une première identification est réalisée à l'aide de la fonction `auto.arima()`, qui recherche automatiquement la combinaison de paramètres minimisant le critère d'information AIC. Le modèle retenu est SARIMA(1,0,1)(2,1,0)12. Cette spécification constitue un point de départ pour la construction et la comparaison d'autres modèles candidats. Les ordres proposés sont cohérents avec les dépendances temporelles et la saisonnalité observées lors de l'analyse exploratoire.

Bien que `auto.arima()` propose le modèle SARIMA(1,0,1)(2,1,0)12, plusieurs spécifications voisines ont été estimées afin d'évaluer si une structure alternative pouvait offrir de meilleures performances en termes d'ajustement, de diagnostic des résidus et de prévision sur l'échantillon de test.

9.3 Estimation du modèle de référence

À partir de cette première identification et des résultats de l'analyse exploratoire, un premier modèle SARIMA(2,0,1)(0,1,2)12 est estimé afin de servir de modèle de référence pour les comparaisons ultérieures.

```
modele_sarima <- Arima(
  accidents_train,
  order = c(2,0,1),
  seasonal = list(
    order = c(0,1,2),
    period = 12
  )
)
```

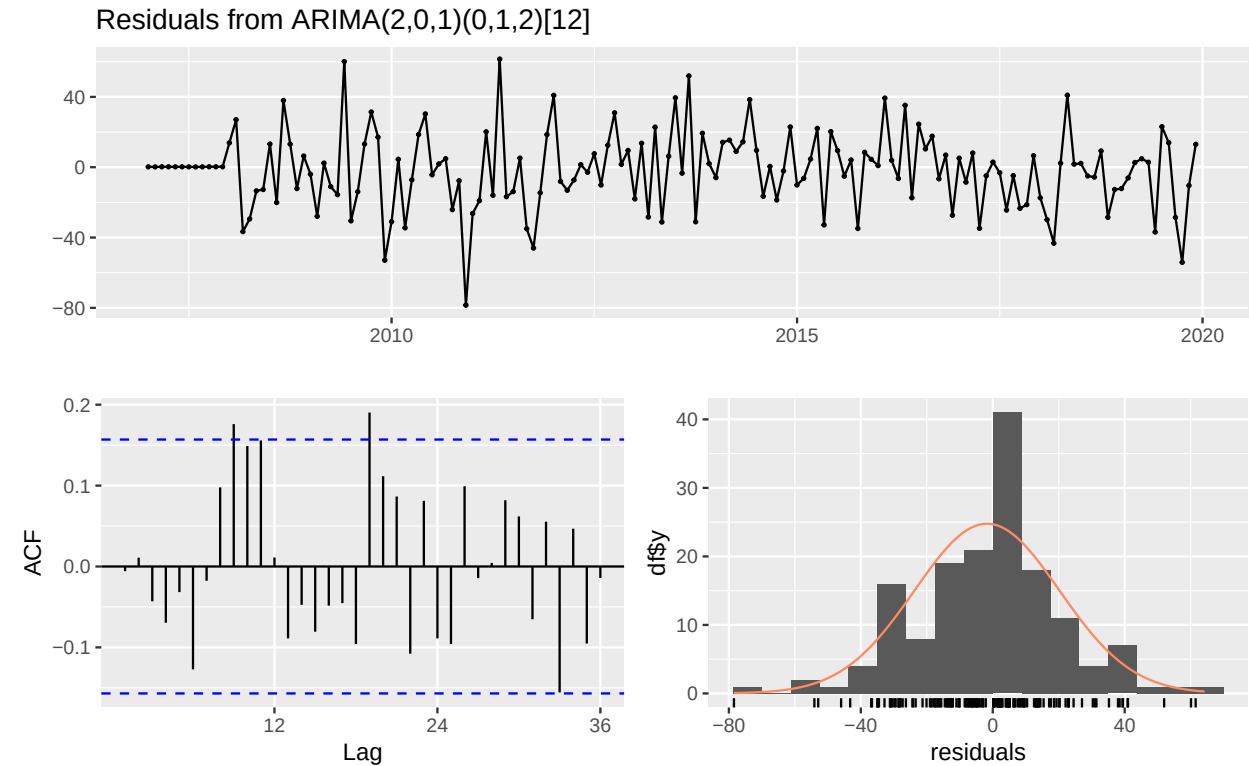
9.4 Diagnostic des résidus

Les résidus du modèle sont analysés afin de vérifier qu'ils se comportent comme un bruit blanc, condition nécessaire à la validité du modèle.

Les diagnostics reposent notamment sur :

le test de Ljung-Box, qui vérifie l'absence d'autocorrélation résiduelle ; le test de Jarque-Bera, qui évalue la normalité des résidus ; l'examen graphique des fonctions ACF et PACF des résidus.

```
checkresiduals(modele_sarima)
```



```
#>
#>  Ljung-Box test
#>
#> data:  Residuals from ARIMA(2,0,1)(0,1,2)[12]
#> Q* = 38.926, df = 19, p-value = 0.004516
#>
#> Model df: 5.   Total lags used: 24

Box.test(
  residuals(modele_sarima),
  lag = 24,
  fitdf = length(coef(modele_sarima)),
  type = "Ljung-Box"
)
#>
#>  Box-Ljung test
#>
#> data:  residuals(modele_sarima)
#> X-squared = 38.926, df = 19, p-value = 0.004516

jarque.bera.test(
  residuals(modele_sarima)
)
```

```
#>
#> Jarque Bera Test
#>
#> data: residuals(modele_sarima)
#> X-squared = 5.0415, df = 2, p-value = 0.0804
```

Les diagnostics montrent que les résidus sont globalement centrés autour de zéro et présentent une variance relativement stable au cours du temps. L'histogramme suggère une distribution proche de la normalité, ce qui est confirmé par le test de Jarque–Bera ($p=0,080$), qui ne permet pas de rejeter l'hypothèse de normalité des résidus au seuil de 5 %.

En revanche, le test de Ljung–Box est significatif ($p=0,0045$), indiquant la présence d'une autocorrélation résiduelle. Cette observation est également visible sur l'ACF des résidus, où quelques pics dépassent les bandes de confiance, notamment autour de certains retards saisonniers. Les résidus ne constituent donc pas un bruit blanc parfait, ce qui suggère que le modèle SARIMA(2,0,1)(0,1,2)₁₂ ne capture pas entièrement la structure de dépendance de la série. Cette constatation justifie la comparaison avec d'autres spécifications.

9.5 Comparaison des modèles candidats

Plusieurs spécifications du modèle SARIMA sont ensuite estimées afin d'identifier la structure offrant le meilleur compromis entre qualité d'ajustement et parcimonie

```
mod1 <- Arima(
  accidents_train,
  order = c(1,0,1),
  seasonal = list(order = c(0,1,2), period = 12)
)

mod2 <- Arima(
  accidents_train,
  order = c(2,0,2),
  seasonal = list(order = c(0,1,2), period = 12)
)

mod3 <- Arima(
  accidents_train,
  order = c(2,0,1),
  seasonal = list(order = c(1,1,2), period = 12)
)

mod4 <- Arima(
  accidents_train,
  order = c(2,0,1),
  seasonal = list(order = c(0,1,1), period = 12)
)
```

Les différents modèles sont comparés à l'aide du critère d'information d'Akaike (AIC), qui permet d'évaluer le compromis entre qualité d'ajustement et parcimonie. Plus la valeur de l'AIC est faible, meilleur est le modèle.

```
library(knitr)

comparaison_aic <- data.frame(
  Modèle = c(
    "SARIMA(2,0,1)(0,1,2)",
    "SARIMA(1,0,1)(0,1,2)",
```

```

    "SARIMA(2,0,2)(0,1,2)",
    "SARIMA(2,0,1)(1,1,2)",
    "SARIMA(2,0,1)(0,1,1)"
  ),
  AIC = round(c(
    AIC(modele_sarima),
    AIC(mod1),
    AIC(mod2),
    AIC(mod3),
    AIC(mod4)
  ), 2)
)

kable(
  comparaison_aic,
  caption = "Comparaison des modèles SARIMA selon le critère AIC",
  align = c("l", "c")
)

```

Table 1: Comparaison des modèles SARIMA selon le critère AIC

Modèle	AIC
SARIMA(2,0,1)(0,1,2)	1352.99
SARIMA(1,0,1)(0,1,2)	1351.19
SARIMA(2,0,2)(0,1,2)	1363.22
SARIMA(2,0,1)(1,1,2)	1354.81
SARIMA(2,0,1)(0,1,1)	1351.29

Les résultats montrent que le modèle SARIMA(1,0,1)(0,1,2) obtient la valeur d'AIC la plus faible (1351,19), ce qui en fait le meilleur modèle selon ce critère.

Le modèle SARIMA(2,0,1)(0,1,1) présente un AIC très proche (1351,29), tandis que le modèle de référence SARIMA(2,0,1)(0,1,2) est légèrement moins performant (1352,99). À l'inverse, les modèles SARIMA(2,0,2)(0,1,2) et SARIMA(2,0,1)(1,1,2) affichent des valeurs d'AIC plus élevées et apparaissent donc moins adaptés aux données.

9.6 Validation sur l'échantillon test

Les trois modèles les plus performants selon le critère AIC ont ensuite été évalués sur l'échantillon de test (2020-2021) à l'aide des indicateurs RMSE, MAE et MAPE.

```

pred_sarima <- forecast(
  modele_sarima,
  h = length(accidents_test)
)

pred_mod1 <- forecast(
  mod1,
  h = length(accidents_test)
)

pred_mod4 <- forecast(
  mod4,

```

```

h = length(accidents_test)
)

perf <- rbind(
  "SARIMA(2,0,1)(0,1,2)" =
    accuracy(pred_sarima, accidents_test)[2,],

  "SARIMA(1,0,1)(0,1,2)" =
    accuracy(pred_mod1, accidents_test)[2,],

  "SARIMA(2,0,1)(0,1,1)" =
    accuracy(pred_mod4, accidents_test)[2,]
)

comparaison_test <- data.frame(
  Modèle = rownames(perf),
  RMSE = round(perf[, "RMSE"], 2),
  MAE = round(perf[, "MAE"], 2),
  MAPE = round(perf[, "MAPE"], 2)
)

kable(
  comparaison_test,
  caption = "Performances des modèles sur l'échantillon de test",
  align = c("l", "c", "c", "c")
)

```

Table 2: Performances des modèles sur l'échantillon de test

	Modèle	RMSE	MAE	MAPE
SARIMA(2,0,1)(0,1,2)	SARIMA(2,0,1)(0,1,2)	53.21	35.72	38.70
SARIMA(1,0,1)(0,1,2)	SARIMA(1,0,1)(0,1,2)	53.46	35.90	38.87
SARIMA(2,0,1)(0,1,1)	SARIMA(2,0,1)(0,1,1)	53.24	35.76	38.81

Les performances prédictives des trois modèles sont très proches. Le modèle SARIMA(2,0,1)(0,1,2) obtient le RMSE le plus faible (53,21) ainsi que le MAE le plus faible (35,72) et le MAPE le plus faible (38,70 %). Les écarts avec les deux autres modèles restent toutefois très faibles (moins de 0,3 unité sur le RMSE et moins de 0,2 % sur le MAPE), ce qui indique que leurs capacités de prévision sont quasiment équivalentes.

Dans la suite de l'étude, le modèle SARIMA(2,0,1)(0,1,2) est retenu comme modèle de référence, en raison de ses performances légèrement supérieures sur les données de test.

10 Construction des modèles SARIMAX

Afin d'identifier les variables exogènes les plus pertinentes, toutes les combinaisons possibles des variables explicatives sont d'abord évaluées sur le modèle SARIMA de référence. Les combinaisons les plus prometteuses sont ensuite réestimées sur plusieurs structures SARIMA candidates afin d'identifier la spécification SARIMAX offrant les meilleures performances d'ajustement et de prévision.

10.1 Recherche des variables exogènes

Afin d'identifier les variables exogènes les plus pertinentes, toutes les combinaisons possibles des variables explicatives sont évaluées. Chaque combinaison est intégrée au modèle SARIMA de référence, puis les

modèles obtenus sont comparés à partir du critère d'information d'Akaike (AIC) et du RMSE calculé sur l'échantillon de test.

```
vars <- colnames(xreg_train_scaled)

results <- data.frame()

for(k in 1:length(vars)){

  combs <- combn(
    vars,
    k,
    simplify = FALSE
  )

  for(v in combs){

    fit <- try(
      Arima(
        accidents_train,
        order = c(2,0,1),
        seasonal = list(
          order = c(0,1,2),
          period = 12
        ),
        xreg = xreg_train_scaled[
          ,
          v,
          drop = FALSE
        ]
      ),
      silent = TRUE
    )

    if(!inherits(fit,"try-error")){

      pred <- forecast(
        fit,
        h = length(accidents_test),
        xreg = xreg_test_scaled[
          ,
          v,
          drop = FALSE
        ]
      )

      results <- rbind(
        results,
        data.frame(
          variables = paste(
            v,
            collapse = " + "
          ),
          AIC = AIC(fit),
```

```

    RMSE = accuracy(
      pred,
      accidents_test
    )[2,"RMSE"]
  )
}
}
}

results <- results[
  order(results$RMSE),
]

top10 <- head(results, 10)

top10
#>
#> 20          variables      AIC    RMSE
#> 36      pct_autoroute + pct_moto 1355.906 51.52861
#> 6          pct_moto 1356.533 51.56896
#> 18      pct_moto 1353.933 51.62723
#> 34      pct_intersection + pct_moto 1354.145 51.75460
#> 15      pct_meteo + pct_intersection + pct_moto 1354.944 51.76411
#> 54      pct_meteo + pct_moto 1354.610 51.78659
#> 55      pct_meteo + pct_intersection + pct_pieton + pct_moto 1356.851 51.91164
#> 40      pct_meteo + pct_autoroute + pct_pieton + pct_moto 1358.392 52.07990
#> 53      pct_intersection + pct_pieton + pct_moto 1356.002 52.10581
#> 53      pct_meteo + pct_intersection + pct_autoroute + pct_moto 1356.920 52.15352

```

Le classement des modèles selon le RMSE montre que les meilleures performances prédictives sont obtenues avec des modèles intégrant principalement les variables `pct_moto` et `pct_autoroute`. La combinaison `pct_autoroute + pct_moto` obtient le RMSE le plus faible (51,53), tandis que l'ajout de `pct_meteo` conduit à une performance très proche (RMSE = 51,57). La variable `pct_moto` apparaît dans l'ensemble des dix meilleures combinaisons, ce qui suggère qu'elle constitue la variable exogène la plus informative pour expliquer les variations mensuelles du nombre d'accidents. Les variables `pct_autoroute`, `pct_meteo` et `pct_intersection` améliorent légèrement les performances lorsqu'elles sont combinées avec `pct_moto`, alors que `pct_pieton` est moins fréquemment présente parmi les meilleurs modèles.

10.2 Comparaison des structures SARIMAX

```

modeles <- list(

  SARIMA_201_012 = list(
    order = c(2,0,1),
    seasonal = c(0,1,2)
  ),

  SARIMA_101_012 = list(
    order = c(1,0,1),
    seasonal = c(0,1,2)
  ),

```

```

SARIMA_201_011 = list(
  order = c(2,0,1),
  seasonal = c(0,1,1)
),

SARIMA_201_112 = list(
  order = c(2,0,1),
  seasonal = c(1,1,2)
)
)

comparaison <- data.frame()

for(i in 1:nrow(top10)){

  vars_i <- unlist(
    strsplit(
      top10$variables[i],
      " \\+ "
    )
  )

  for(nom_modele in names(modeles)){

    spec <- modeles[[nom_modele]]

    fit <- Arima(
      accidents_train,
      order = spec$order,
      seasonal = list(
        order = spec$seasonal,
        period = 12
      ),
      xreg = xreg_train_scaled[
        ,
        vars_i,
        drop = FALSE
      ]
    )

    pred <- forecast(
      fit,
      h = length(accidents_test),
      xreg = xreg_test_scaled[
        ,
        vars_i,
        drop = FALSE
      ]
    )

    comparaison <- rbind(
      comparaison,
      data.frame(

```

```

    modele = nom_modele,
    variables = top10$variables[i],
    AIC = AIC(fit),
    RMSE = accuracy(
      pred,
      accidents_test
    )[2, "RMSE"]
  )
}
}

```

10.3 Comparaison des modèles selon l'AIC

Le critère d'information d'Akaike est utilisé afin de comparer les différents modèles estimés. Un AIC plus faible traduit un meilleur compromis entre qualité d'ajustement et complexité du modèle.

```

comparaison_AIC <- comparaison |>
  arrange(AIC)

head(comparaison_AIC, 20)
#>      modele variables
#> 1 SARIMA_101_012 pct_moto
#> 2 SARIMA_101_012 pct_intersection + pct_moto
#> 3 SARIMA_201_011 pct_moto
#> 4 SARIMA_201_011 pct_intersection + pct_moto
#> 5 SARIMA_101_012 pct_meteo + pct_moto
#> 6 SARIMA_101_012 pct_meteo + pct_intersection + pct_moto
#> 7 SARIMA_201_011 pct_meteo + pct_moto
#> 8 SARIMA_201_011 pct_meteo + pct_intersection + pct_moto
#> 9 SARIMA_201_012 pct_moto
#> 10 SARIMA_101_012 pct_autoroute + pct_moto
#> 11 SARIMA_201_012 pct_intersection + pct_moto
#> 12 SARIMA_101_012 pct_intersection + pct_pieton + pct_moto
#> 13 SARIMA_201_011 pct_autoroute + pct_moto
#> 14 SARIMA_201_012 pct_meteo + pct_moto
#> 15 SARIMA_201_011 pct_intersection + pct_pieton + pct_moto
#> 16 SARIMA_101_012 pct_meteo + pct_autoroute + pct_moto
#> 17 SARIMA_201_012 pct_meteo + pct_intersection + pct_moto
#> 18 SARIMA_101_012 pct_meteo + pct_intersection + pct_pieton + pct_moto
#> 19 SARIMA_201_011 pct_meteo + pct_autoroute + pct_moto
#> 20 SARIMA_101_012 pct_meteo + pct_intersection + pct_autoroute + pct_moto
#>      AIC      RMSE
#> 1 1352.117 51.83328
#> 2 1352.271 52.01877
#> 3 1352.339 51.76830
#> 4 1352.699 52.14172
#> 5 1352.934 51.99141
#> 6 1353.180 51.93364
#> 7 1353.190 51.88774
#> 8 1353.654 52.16056
#> 9 1353.933 51.62723
#> 10 1354.057 51.62685
#> 11 1354.145 51.75460

```

```
#> 12 1354.160 52.00540
#> 13 1354.333 51.72833
#> 14 1354.610 51.78659
#> 15 1354.642 52.32983
#> 16 1354.796 51.69278
#> 17 1354.944 51.76411
#> 18 1355.121 52.22354
#> 19 1355.166 51.79707
#> 20 1355.176 52.21631
```

Les modèles présentant les plus faibles valeurs d'AIC correspondent principalement aux structures SARIMA(1,0,1)(0,1,2) et SARIMA(2,0,1)(0,1,1) associées à la variable `pct_moto`, seule ou combinée avec `pct_intersection`. Le plus faible AIC est obtenu pour le modèle SARIMA(1,0,1)(0,1,2) utilisant uniquement `pct_moto` (AIC = 1352,12). Les écarts d'AIC entre les meilleurs modèles restent toutefois très faibles (inférieurs à trois unités), ce qui indique que plusieurs spécifications offrent une qualité d'ajustement comparable sur les données d'apprentissage. Le critère AIC ne permettant pas à lui seul de juger des performances prédictives, une comparaison sur l'échantillon de test est également réalisée.

10.4 Comparaison des modèles selon le RMSE

Les modèles sont ensuite comparés à partir du RMSE obtenu sur l'échantillon de test. Contrairement à l'AIC, cet indicateur mesure directement la qualité des prévisions sur des observations non utilisées lors de l'estimation.

```
comparaison <- comparaison[
  order(comparaison$RMSE),
]

head(comparaison,20)
#>      modele                                variables      AIC
#> 1  SARIMA_201_012                pct_autoroute + pct_moto 1355.906
#> 5  SARIMA_201_012      pct_meteo + pct_autoroute + pct_moto 1356.533
#> 2  SARIMA_101_012                pct_autoroute + pct_moto 1354.057
#> 9  SARIMA_201_012                                pct_moto 1353.933
#> 6  SARIMA_101_012      pct_meteo + pct_autoroute + pct_moto 1354.796
#> 3  SARIMA_201_011                pct_autoroute + pct_moto 1354.333
#> 13 SARIMA_201_012                pct_intersection + pct_moto 1354.145
#> 17 SARIMA_201_012      pct_meteo + pct_intersection + pct_moto 1354.944
#> 11 SARIMA_201_011                                pct_moto 1352.339
#> 4  SARIMA_201_112                pct_autoroute + pct_moto 1357.864
#> 21 SARIMA_201_012                pct_meteo + pct_moto 1354.610
#> 7  SARIMA_201_011      pct_meteo + pct_autoroute + pct_moto 1355.166
#> 10 SARIMA_101_012                                pct_moto 1352.117
#> 12 SARIMA_201_112                                pct_moto 1355.921
#> 23 SARIMA_201_011                pct_meteo + pct_moto 1353.190
#> 8  SARIMA_201_112      pct_meteo + pct_autoroute + pct_moto 1358.717
#> 25 SARIMA_201_012      pct_meteo + pct_intersection + pct_pieton + pct_moto 1356.851
#> 18 SARIMA_101_012      pct_meteo + pct_intersection + pct_moto 1353.180
#> 22 SARIMA_101_012                pct_meteo + pct_moto 1352.934
#> 34 SARIMA_101_012      pct_intersection + pct_pieton + pct_moto 1354.160
#>      RMSE
#> 1  51.52861
#> 5  51.56896
#> 2  51.62685
```

```

#> 9 51.62723
#> 6 51.69278
#> 3 51.72833
#> 13 51.75460
#> 17 51.76411
#> 11 51.76830
#> 4 51.77176
#> 21 51.78659
#> 7 51.79707
#> 10 51.83328
#> 12 51.84924
#> 23 51.88774
#> 8 51.89665
#> 25 51.91164
#> 18 51.93364
#> 22 51.99141
#> 34 52.00540

comparaison[1,]
#>      modele      variables      AIC      RMSE
#> 1 SARIMA_201_012 pct_autoroute + pct_moto 1355.906 51.52861

```

Le classement selon le RMSE met en évidence que les meilleures performances de prévision sont obtenues avec le modèle SARIMA(2,0,1)(0,1,2) utilisant les variables pct_autoroute et pct_moto, avec un RMSE de 51,53. Les modèles intégrant pct_meteo en complément de ces deux variables obtiennent également des performances très proches (RMSE = 51,57). De manière générale, les écarts de RMSE entre les meilleurs modèles demeurent faibles (moins de 0,5 unité), ce qui montre que plusieurs spécifications présentent une capacité de prévision similaire. En revanche, le classement obtenu selon le RMSE diffère de celui fondé sur l'AIC : le modèle présentant le meilleur ajustement sur les données d'apprentissage n'est pas celui qui offre les meilleures performances de prévision sur l'échantillon de test. Cette différence souligne l'importance d'évaluer les modèles à l'aide d'indicateurs de prévision en complément des critères d'information.

10.5 Modèles candidats retenus

Les modèles présentant les meilleures performances selon le RMSE sont retenus pour l'étape de validation. Ils feront l'objet d'une validation croisée afin de vérifier leur capacité de généralisation avant la sélection du modèle final.

```

# Trois meilleurs modèles selon le RMSE
modeles_candidats <- comparaison |>
  dplyr::arrange(RMSE) |>
  dplyr::slice(1:3)

knitr::kable(
  modeles_candidats,
  digits = 3,
  caption = "Modèles candidats retenus pour la validation"
)

```

Table 3: Modèles candidats retenus pour la validation

modele	variables	AIC	RMSE
SARIMA_201_012	pct_autoroute + pct_moto	1355.906	51.529

modele	variables	AIC	RMSE
SARIMA_201_012	pct_meteo + pct_autoroute + pct_moto	1356.533	51.569
SARIMA_101_012	pct_autoroute + pct_moto	1354.057	51.627

Les trois modèles présentant les plus faibles valeurs du RMSE sont retenus comme modèles candidats pour la phase suivante de validation. Ils correspondent à deux structures SARIMA différentes utilisant principalement les variables pct_autoroute et pct_moto, avec ou sans la variable pct_meteo. Leurs performances étant très proches, une évaluation complémentaire permettra de déterminer le modèle offrant le meilleur compromis entre qualité d'ajustement, précision des prévisions et robustesse.

11 Validation du modèle retenu

Après la sélection des modèles candidats, une validation est réalisée afin d'évaluer leur capacité de généralisation. Cette étape comprend une validation croisée de type Expanding Window, suivie de l'estimation du modèle final sur l'ensemble des données disponibles et d'une analyse de ses résidus. L'objectif est de retenir le modèle offrant les meilleures performances prédictives tout en vérifiant que les hypothèses du modèle SARIMAX sont raisonnablement satisfaites.

11.1 Validation croisée (Expanding Window)

Une validation croisée de type Expanding Window est réalisée afin d'évaluer les performances prédictives des modèles dans un contexte proche de leur utilisation réelle. À chaque itération, le modèle est estimé sur un historique progressivement enrichi avant de produire une prévision à un horizon d'un mois.

11.1.1 Construction de la matrice des variables exogènes

Les variables exogènes des échantillons d'apprentissage et de test sont regroupées dans une matrice unique afin de faciliter la mise en œuvre de la validation croisée.

```
xreg_scaled <- rbind(
  xreg_train_scaled,
  xreg_test_scaled
)
```

11.1.2 Fonction de validation croisée (prévision à un pas)

Une fonction est définie afin d'estimer successivement le modèle sur des fenêtres d'apprentissage de taille croissante et de calculer le RMSE associé aux prévisions réalisées à un pas.

```
cv_rmse <- function(vars){

  erreurs <- c()

  # Nombre minimal d'observations ( 5 ans de données mensuelles)
  # afin d'obtenir une estimation stable du modèle SARIMA.
  debut_cv <- 61

  for(i in debut_cv:(length(accidents_ts) - 1)){

    # Série d'apprentissage

    y_train <- window(
      accidents_ts,
```

```

    end = time(accidents_ts)[i]
  )

  # Observation à prédire

  y_test <- accidents_ts[i + 1]

  x_train <- xreg_scaled[
    1:i,
    vars,
    drop = FALSE
  ]

  x_test <- xreg_scaled[
    i + 1,
    vars,
    drop = FALSE
  ]

  # Estimation du modèle
  fit <- try(
    Arima(
      y_train,
      order = c(2, 0, 1),
      seasonal = list(
        order = c(0, 1, 2),
        period = 12
      ),
      xreg = x_train
    ),
    silent = TRUE
  )

  # Prédiction à un pas

  if(!inherits(fit, "try-error")){

    pred <- forecast(
      fit,
      h = 1,
      xreg = x_test
    )

    erreurs <- c(
      erreurs,
      y_test - pred$mean[1]
    )
  }
}

if(length(erreurs) == 0){
  return(NA_real_)
}

```

```

}

sqrt(mean(erreurs^2))
}

```

11.1.3 Comparaison des modèles candidats

Les modèles candidats sont comparés à partir du RMSE obtenu lors de la validation croisée. Cette comparaison permet de sélectionner le modèle offrant les meilleures performances prédictives sur l'ensemble de la période étudiée.

```

resultats_cv <- data.frame(
  Modele = c(
    "pct_moto",
    "pct_autoroute + pct_moto",
    "pct_meteo + pct_autoroute + pct_moto"
  ),
  RMSE_CV = c(
    cv_rmse("pct_moto"),
    cv_rmse(c("pct_autoroute", "pct_moto")),
    cv_rmse(c("pct_meteo", "pct_autoroute", "pct_moto"))
  )
)

knitr::kable(
  resultats_cv,
  digits = 3,
  caption = "Résultats de la validation croisée"
)

```

Table 4: Résultats de la validation croisée

Modele	RMSE_CV
pct_moto	29.391
pct_autoroute + pct_moto	29.825
pct_meteo + pct_autoroute + pct_moto	29.975

Les résultats de la validation croisée montrent que le modèle intégrant uniquement pct_moto obtient le plus faible RMSE (29,39), devant les modèles utilisant pct_autoroute + pct_moto (29,83) et pct_meteo + pct_autoroute + pct_moto (29,98). Les écarts entre ces performances restent toutefois relativement faibles, ce qui indique que les trois spécifications présentent une capacité prédictive comparable. Le modèle utilisant uniquement pct_moto est finalement retenu au regard de son meilleur compromis entre précision de prévision et simplicité.

11.2 Estimation du modèle final

Le modèle sélectionné à l'issue de la validation croisée est ensuite réestimé sur l'ensemble des données disponibles afin d'obtenir les estimations finales des paramètres qui seront utilisées pour les analyses de diagnostic et les prévisions.

```

modele_final <- Arima(
  accidents_ts,
  order = c(2, 0, 1),
  seasonal = list(

```

```

    order = c(0, 1, 2),
    period = 12
  ),
  xreg = xreg_scaled[, "pct_moto", drop = FALSE]
)

resume_modele <- data.frame(
  Critere = c("AIC"),
  Valeur = c(AIC(modele_final))
)

knitr::kable(
  resume_modele,
  digits = 3,
  caption = "Critère d'information du modèle final"
)

```

Table 5: Critère d'information du modèle final

Critere	Valeur
AIC	1620.248

```

coef_modele <- data.frame(
  Paramètre = names(coef(modele_final)),
  Estimation = coef(modele_final)
)

knitr::kable(
  coef_modele,
  digits = 4,
  caption = "Estimation des paramètres du modèle final"
)

```

Table 6: Estimation des paramètres du modèle final

	Paramètre	Estimation
ar1	ar1	1.1757
ar2	ar2	-0.2099
ma1	ma1	-0.8405
sma1	sma1	-0.9452
sma2	sma2	0.1090
pct_moto	pct_moto	6.1894

Les estimations obtenues correspondent aux paramètres du modèle final ajusté sur l'ensemble des observations disponibles. Ce modèle constitue la version définitive utilisée pour les diagnostics et les prévisions. Les coefficients associés aux variables exogènes permettent d'apprécier leur contribution à l'explication des variations mensuelles du nombre d'accidents.

11.3 Vérification des résidus

L'analyse des résidus permet de vérifier que le modèle retenu satisfait les principales hypothèses des modèles SARIMAX. En particulier, les résidus doivent être assimilables à un bruit blanc, c'est-à-dire ne présenter ni

autocorrélation résiduelle ni structure systématique.

```
ljung <- Box.test(
  residuals(modele_final),
  lag = 24,
  type = "Ljung-Box",
  fitdf = length(coef(modele_final))
)

diagnostic <- data.frame(
  Test = "Ljung-Box",
  Statistique = as.numeric(ljung$statistic),
  "p-value" = ljung$p.value
)

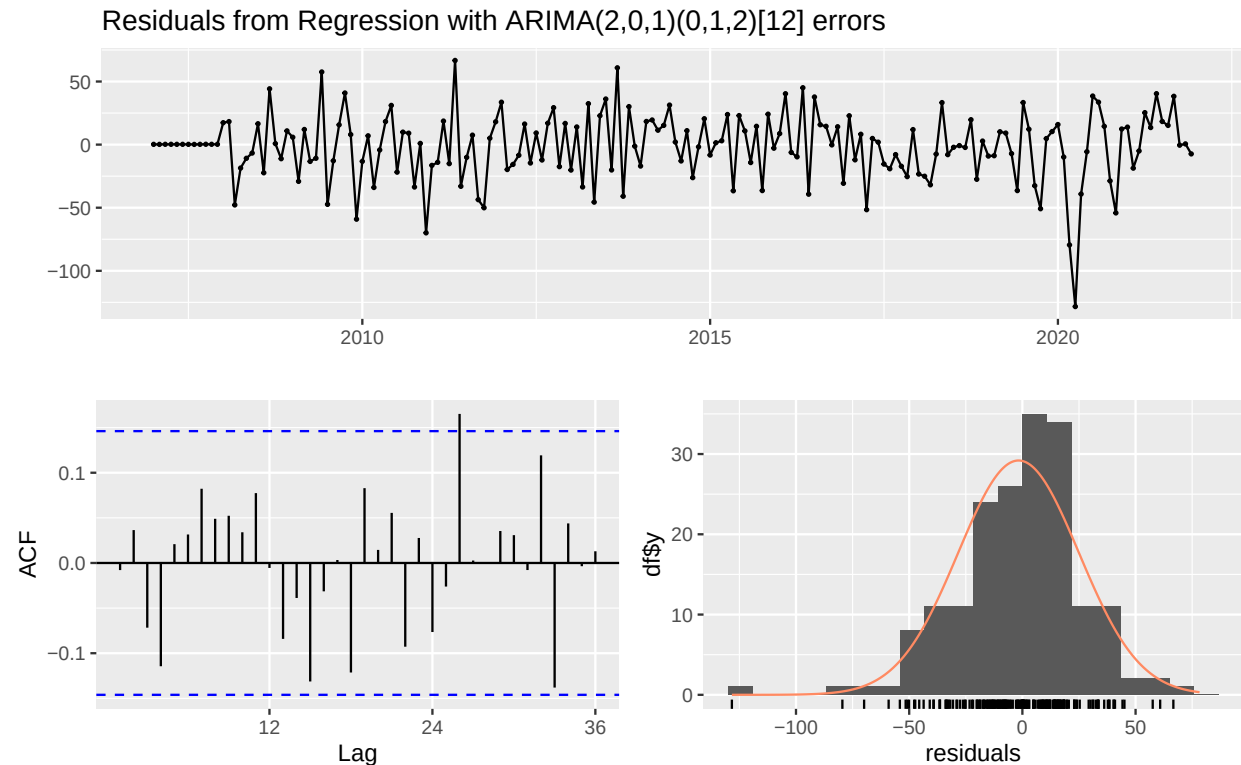
knitr::kable(
  diagnostic,
  digits = 4,
  caption = "Résultat du test de Ljung-Box"
)
```

Table 7: Résultat du test de Ljung-Box

Test	Statistique	p.value
Ljung-Box	21.0994	0.2744

```
# Diagnostic graphique des résidus

checkresiduals(modele_final)
```



```
#>
#> Ljung-Box test
#>
#> data: Residuals from Regression with ARIMA(2,0,1)(0,1,2)[12] errors
#> Q* = 21.099, df = 19, p-value = 0.3313
#>
#> Model df: 5. Total lags used: 24
```

Le test de Ljung-Box permet d'évaluer l'absence d'autocorrélation résiduelle. Une p-value supérieure au seuil de significativité indique que l'hypothèse de bruit blanc ne peut pas être rejetée, ce qui traduit un ajustement satisfaisant du modèle. Les graphiques de diagnostic permettent de compléter cette analyse en vérifiant la distribution des résidus et l'absence de structure résiduelle apparente.

12 Validation externe sur les données BAAC 2022

Cette dernière étape consiste à évaluer les performances du modèle SARIMAX sur un jeu de données indépendant de celui utilisé pour son estimation. Les données BAAC de l'année 2022 sont utilisées comme base de validation externe afin de vérifier la capacité du modèle à généraliser ses prévisions sur de nouvelles observations.

12.1 Importation des fichiers BAAC 2022

Les fichiers BAAC correspondant à l'année 2022 sont importés afin de constituer la base de données utilisée pour la validation externe du modèle

```
# Lecture des fichiers BAAC 2022
carac <- read_csv2(
  "caracteristiques-2022.csv",
  show_col_types = FALSE
```

```
)

veh <- read_csv2(
  "vehicules-2022.csv",
  show_col_types = FALSE)
```

12.2 Construction de la base de validation

Les accidents corporels survenus dans le département des Hauts-de-Seine sont sélectionnés puis les informations nécessaires sont fusionnées afin de reconstruire une base comparable à celle utilisée pour l'apprentissage.

```
carac92 <- carac %>%
  filter(dep == "92") %>%
  rename(Num_Acc = Accident_Id)

accidents_moto <- veh %>%
  filter(catv %in% c(30, 31, 32, 33, 34)) %>%
  distinct(Num_Acc) %>%
  mutate(moto = 1)

base92 <- carac92 %>%
  left_join(accidents_moto, by = "Num_Acc") %>%
  mutate(
    moto = ifelse(is.na(moto), 0, 1),
    mois = as.integer(mois)
  )
```

La base de validation obtenue reprend la même structure que celle utilisée lors de l'estimation du modèle. Cette homogénéité garantit que la variable exogène est calculée selon la même méthodologie que celle utilisée lors de l'estimation du modèle, assurant ainsi la cohérence entre les phases d'apprentissage et de validation.

12.3 Construction de la variable exogène mensuelle 2022

La variable exogène retenue dans le modèle final est calculée mensuellement selon la même méthodologie que celle appliquée aux données historiques.

```
xreg_2022 <- base92 %>%
  group_by(mois) %>%
  summarise(
    nb_accidents = n(),
    pct_moto = mean(moto, na.rm = TRUE),
    .groups = "drop"
  ) %>%
  arrange(mois)
```

La proportion mensuelle d'accidents impliquant une moto est calculée pour chacun des mois de l'année 2022.

12.4 Standardisation de la variable

La variable exogène est standardisée à partir des paramètres calculés sur les données d'apprentissage afin d'assurer la cohérence entre les différentes phases de modélisation.

```
centre <- unname(attr(xreg_train_scaled, "scaled:center")["pct_moto"])
ecart <- unname(attr(xreg_train_scaled, "scaled:scale")["pct_moto"])

xreg_2022_scaled <- scale(
```

```
xreg_2022[, "pct_moto", drop = FALSE],
  center = centre,
  scale = ecart
)

colnames(xreg_2022_scaled) <- "pct_moto"
```

L'utilisation des mêmes paramètres de centrage et de réduction que ceux employés lors de l'estimation évite toute fuite d'information provenant des données de validation et garantit la comparabilité de la variable exogène avec celle utilisée lors de l'estimation du modèle.

12.5 Vérification de la cohérence des données

Plusieurs contrôles sont réalisés afin de vérifier la compatibilité des données de validation avec le modèle estimé.

```
# Vérification des dimensions
length(accidents_ts)
#> [1] 180
nrow(xreg_scaled)
#> [1] 180
nrow(xreg_2022_scaled)
#> [1] 12

# Vérification des noms des variables exogènes

colnames(xreg_scaled)
#> [1] "pct_nuit"          "pct_meteo"          "pct_intersection" "pct_autoroute"
#> [5] "pct_pieton"         "pct_moto"
colnames(xreg_2022_scaled)
#> [1] "pct_moto"

# Vérification des variables attendues par le modèle final

coef(modele_final)
#>      ar1      ar2      ma1      sma1      sma2  pct_moto
#> 1.1756828 -0.2098717 -0.8405345 -0.9452274 0.1089551 6.1894373

# Vérification du nombre de variables exogènes

ncol(xreg_2022_scaled)
#> [1] 1
```

Les contrôles confirment que la matrice des variables exogènes de validation est cohérente avec celle utilisée lors de l'estimation du modèle. Le modèle final ne comporte qu'une seule variable exogène, `pct_moto`, et la matrice `xreg_2022_scaled` présente bien le même nom de variable ainsi que le nombre de colonnes attendu. Les données sont ainsi compatibles avec la fonction de prévision `forecast()`.

12.6 Prévision des accidents en 2022

Le modèle SARIMAX retenu est utilisé pour produire les prévisions mensuelles du nombre d'accidents corporels sur l'année 2022 à partir des variables exogènes observées.

```
prevision_2022 <- forecast(
  modele_final,
```

```
h = 12,
xreg = xreg_2022_scaled
)
```

12.7 Comparaison entre les prévisions et les observations

Les valeurs prédites sont comparées aux observations réellement enregistrées afin d'évaluer la qualité des prévisions produites par le modèle.

```
obs_2022 <- c(
  162, 173, 219, 178,
  262, 276, 228, 127,
  258, 235, 197, 204
)

validation_2022 <- data.frame(
  Mois = factor(month.abb, levels = month.abb),
  Observe = obs_2022,
  Prevision = round(as.numeric(prevision_2022$mean))
)

validation_2022 <- transform(
  validation_2022,
  Erreur = Observe - Prevision,
  Erreur_abs = abs(Observe - Prevision),
  Erreur_pct = 100 * (Observe - Prevision) / Observe
)

knitr::kable(
  validation_2022,
  digits = 1,
  caption = "Comparaison entre les observations et les prévisions en 2022"
)
```

Table 8: Comparaison entre les observations et les prévisions en 2022

Mois	Observe	Prevision	Erreur	Erreur_abs	Erreur_pct
Jan	162	213	-51	51	-31.5
Feb	173	181	-8	8	-4.6
Mar	219	201	18	18	8.2
Apr	178	175	3	3	1.7
May	262	210	52	52	19.8
Jun	276	234	42	42	15.2
Jul	228	209	19	19	8.3
Aug	127	129	-2	2	-1.6
Sep	258	244	14	14	5.4
Oct	235	243	-8	8	-3.4
Nov	197	235	-38	38	-19.3
Dec	204	220	-16	16	-7.8

Le tableau met en évidence que le modèle reproduit globalement l'évolution mensuelle des accidents corporels en 2022. Les prévisions sont particulièrement proches des observations pour les mois de février, avril, août,

septembre et octobre. En revanche, des écarts plus importants apparaissent en janvier, mai, juin et novembre, où le modèle sous-estime ou surestime davantage le nombre d'accidents. Malgré ces différences ponctuelles, les prévisions suivent correctement la dynamique générale de la série.

12.8 Évaluation des performances prédictives

Les performances du modèle sont quantifiées à l'aide de plusieurs indicateurs complémentaires : le MAE, le RMSE et le MAPE.

```
MAE <- mean(validation_2022$Erreur_abs)
RMSE <- sqrt(mean(validation_2022$Erreur^2))
MAPE <- mean(abs(validation_2022$Erreur_pct))

cat("MAE =", round(MAE, 2), "\n")
#> MAE = 22.58
cat("RMSE =", round(RMSE, 2), "\n")
#> RMSE = 28.56
cat("MAPE =", round(MAPE, 2), "%\n")
#> MAPE = 10.58 %
```

Les indicateurs obtenus sont les suivants : un MAE de 22,58 accidents, un RMSE de 28,56 accidents et un MAPE de 10,58 %. Ces résultats indiquent qu'en moyenne les prévisions s'écartent d'une vingtaine d'accidents des valeurs observées, avec une erreur relative moyenne d'environ 11 %. Cette précision peut être considérée comme satisfaisante pour une série mensuelle d'accidents corporels présentant une variabilité importante. Le RMSE, légèrement supérieur au MAE, traduit la présence de quelques erreurs plus marquées sur certains mois, notamment en janvier et en mai.

12.9 Visualisation des prévisions

Une représentation graphique est réalisée afin de comparer visuellement les valeurs observées et les valeurs prédites sur l'ensemble de l'année 2022.

```
ggplot(validation_2022, aes(x = Mois, group = 1)) +
  geom_line(aes(y = Prevision, colour = "Prévision"),
            linewidth = 1.3) +
  geom_point(aes(y = Prevision, colour = "Prévision"),
             size = 3) +
  geom_line(aes(y = Observe, colour = "Observé"),
            linewidth = 1.3) +
  geom_point(aes(y = Observe, colour = "Observé"),
             size = 3) +

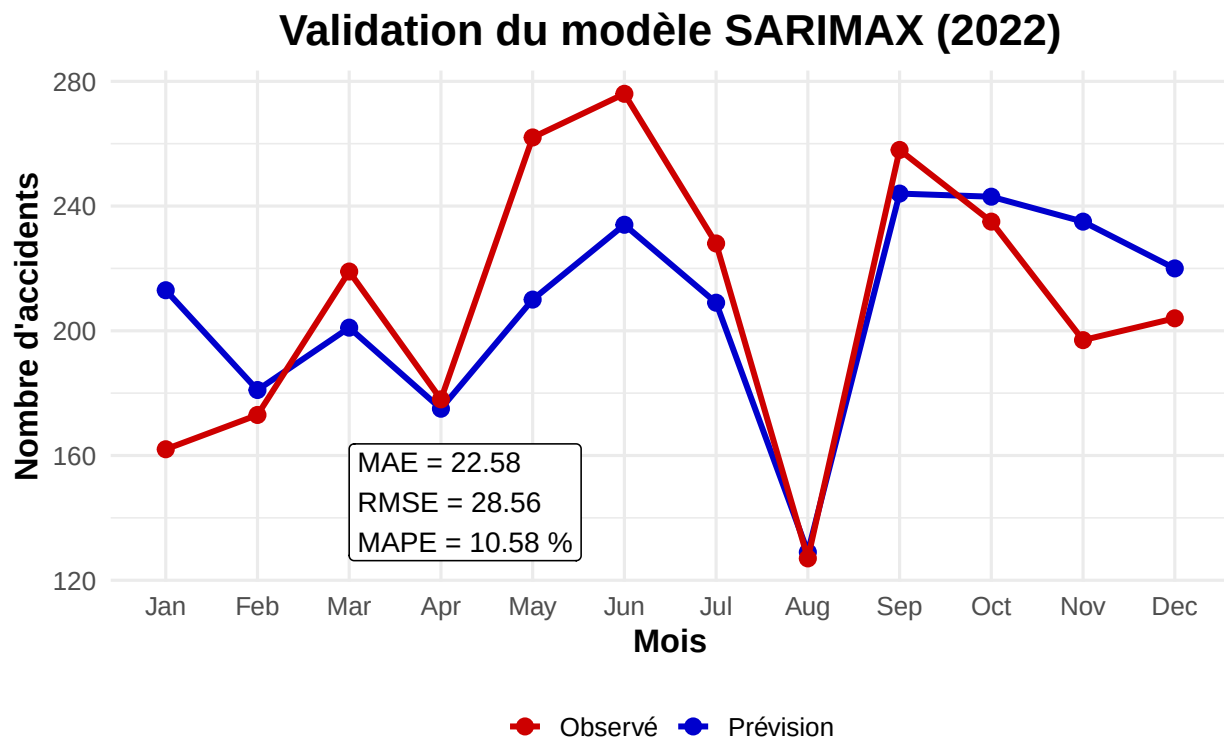
  scale_colour_manual(
    name = NULL,
    values = c(
      "Prévision" = "blue3",
      "Observé" = "red3"
    )
  ) +
  annotate(
    "label",
    x = 3,
    y = 145,
    hjust = 0,
    size = 4.5,
```

```

label = paste0(
  "MAE = ", sprintf("%.2f", MAE),
  "\nRMSE = ", sprintf("%.2f", RMSE),
  "\nMAPE = ", sprintf("%.2f %%", MAPE)
),
fill = "white",
colour = "black"
) +
labs(
  title = "Validation du modèle SARIMAX (2022)",
  x = "Mois",
  y = "Nombre d'accidents"
) +
theme_minimal(base_size = 15) +
theme(

  plot.title = element_text(
    face = "bold",
    size = 20,
    hjust = 0.5
  ),
  axis.title = element_text(
    face = "bold",
    size = 15
  ),
  legend.position = "bottom",
  legend.text = element_text(size = 12)
)

```



```
ggsave("graphics/08_Visualisation_des_prévisions.png", width = 8, height = 5, dpi = 300)
```

Le modèle SARIMAX(2,0,1)(0,1,2)₁₂ intégrant la variable pct_moto présente des performances prédictives satisfaisantes, avec un MAPE de 10,58 % sur l'échantillon de validation externe 2022. Néanmoins, l'analyse graphique montre une sous-estimation systématique des pics de sinistralité observés en mai et juin.

Cette limite structurelle provient de la nature de la variable exogène : il s'agit d'une proportion relative et non d'une mesure de l'exposition absolue au risque. En période printanière, l'augmentation du volume global de circulation des deux-roues motorisés (kilomètres parcourus) accroît l'exposition, un phénomène que la seule part relative des motos ne peut pas capter. Ce manque d'information se traduit par un lissage des prévisions lors des fortes variations saisonnières.

Pour corriger ce biais, une amélioration du modèle consisterait à introduire des variables de pure exposition. L'intégration d'indices de trafic routier en Île-de-France ou de données météorologiques (ensoleillement, températures moyennes) permettrait d'agir comme des variables d'approximation (proxies) de l'intensité réelle de circulation.